Topics in Computational Geometric Vision

Aaron Wetzler

Topics in Computational Geometric Vision

Research Thesis

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy

Aaron Wetzler

Submitted to the Senate
of the Technion — Israel Institute of Technology Elul
5777 Haifa August 2017

This research was carried out under the supervision of Prof. Ron Kimmel, in the Faculties of Computer Science and Electrical Engineering.

Some results in this thesis have been published as articles by the author and research collaborators in conferences and journals during the course of the author's doctoral research period, the most up-to-date versions of which being:

Roberto Mecca, Ariel Tankus, Aaron Wetzler, and Alfred M Bruckstein. A direct differential approach to photometric stereo with perspective viewing. *SIAM Journal on Imaging Sciences*, 7(2):579–612, 2014.

Roberto Mecca, Aaron Wetzler, Alfred M Bruckstein, and Ron Kimmel. Near field photometric stereo with point light sources. *SIAM Journal on Imaging Sciences*, 7(4):2732–2770, 2014.

Roy Or-El, Rom Hershkovitz, Aaron Wetzler, Guy Rosman, Alfred M Bruckstein, and Ron Kimmel. Real-time depth refinement for specular objects. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4378–4386, 2016.

Roy Or-El, Guy Rosman, Aaron Wetzler, Ron Kimmel, and Alfred M Bruckstein. Rgbd-fusion: Real-time high precision depth recovery. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5407–5416, 2015.

Gautam Pai, Aaron Wetzler, and Ron Kimmel. Learning invariant representations of planar curves. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

Ron Slossberg, Aaron Wetzler, and Ron Kimmel. Freehand laser scanning using mobile phone. In *British Machine Vision Conference, 2015*, 2015.

Ron Slossberg, Aaron Wetzler, and Ron Kimmel. Deep stereo matching with dense CRF priors. *arXiv preprint arXiv:1612.01725*, 2016.

Aaron Wetzler, Yonathan Aflalo, Anastasia Dubrovina, and Ron Kimmel. The Laplace-Beltrami operator: a ubiquitous tool for image and shape processing. In *International Symposium on Mathematical Morphology and Its Applications to Signal and Image Processing*, pages 302–316, 2013.

Aaron Wetzler, Rom Hershkovitz, and Ron Kimmel. The Patch-Beltrami framework. submitted, 2017.

Aaron Wetzler and Ron Kimmel. Efficient Beltrami flow in patch-space. In *International Conference on Scale Space and Variational Methods in Computer Vision*, pages 134–143, 2011.

Aaron Wetzler and Ron Kimmel. Multi-GPU training for tera-feature scale, randomized decision trees. submitted, 2017.

Aaron Wetzler and Ron Kimmel. Winqi: A system for 6d localization and SLAM augmentation using wideangle optics and coded light beacons. *arXiv preprint arXiv:1708.05625*, 2017.

Aaron Wetzler, Ron Kimmel, Alfred M Bruckstein, and Roberto Mecca. Close-range photometric stereo with point light sources. In *3DV 2014*, 2014.

Aaron Wetzler, Guy Rosman, and Ron Kimmel. Patch-space Beltrami denoising of 3D point clouds. In *Electrical & Electronics Engineers in Israel (IEEEI), 2012 IEEE 27th Convention of*, pages 1–5, 2012.

Aaron Wetzler, Ron Slossberg, and Ron Kimmel. Rule of thumb: Deep derotation for improved fingertip detection. In *British Machine Vision Conference, 2015*, 2015.

Eric Yudin, Aaron Wetzler, Matan Sela, and Ron Kimmel. Improving 3D facial action unit detection with intrinsic normalization. In *Proceedings of the 1st International Workshop on DIFFerential Geometry in Computer Vision for Analysis of Shapes, Images and Trajectories (DIFF-CV 2015)*, pages 5.1–5.10, 2015.

The generous financial help of the Technion, ERC, Jacobs-Qualcomm foundation, and Mel-Berlin foundation is gratefully acknowledged

Contents

List of Figures

Abstract **1**

1 Introduction **3**

1.1 A 3D world . . . 3

1.2 Removing noise . . . 5

1.3 Comprehension: going beyond the data . . . 5

1.4 In summary . . . 6

I RECONSTRUCTING GEOMETRY **9**

2 Mobile reconstruction **11**

2.1 Stereo devices . . . 11

2.2 Voxel carving . . . 12

2.3 Laser scanning . . . 13

2.4 Beam scanner . . . 14

2.5 Conclusion . . . 15

3 Photometric stereo **17**

3.1 Perspective On Shape from Shading and Photometric Stereo . . . 17

3.2 The Endoscopic Perspective Set-Up . . . 19

3.3 A New Close-Range PS Model . . . 20

3.3.1 Two types of light attenuation . . . 21

3.3.2 $EPPS_2$ model . . . 22

3.4 Direct Surface Reconstruction Using Images with Missing Parts . . . 23

3.5 W-$EPPS_n$ with Almost no Boundary Condition . . . 24

3.5.1 Steering the characteristic field . . . 25

3.5.2 Numerical scheme . . . 26
3.6 Experimental Results . . . 26
3.6.1 Synthetic cases . . . 27
3.6.2 Real cases . . . 29
3.7 Conclusions . . . 30

II GEOMETRIC DENOISING 33

4 The Patch-Beltrami Framework 35
4.1 Introduction . . . 35
4.2 The Patch-Beltrami framework . . . 38
4.2.1 The Beltrami filter . . . 38
4.2.2 Patch space metric . . . 40
4.3 Denoising over regular domains . . . 41
4.3.1 Numerical scheme . . . 42
4.3.2 Parameter tuning . . . 44
4.3.3 Experimental results . . . 48
4.3.4 Residual noise . . . 49
4.3.5 Non-Gaussian denoising . . . 49
4.3.6 Discussion . . . 52
4.4 Extension to point clouds . . . 53
4.4.1 Redefining the update for point-clouds . . . 54
4.4.2 Local neighborhood . . . 55
4.4.3 Frame estimation . . . 56
4.4.4 Updating along the normal . . . 57
4.4.5 Robust normal estimation . . . 57
4.4.6 Results . . . 57
4.5 Operator tuning: a deep perspective . . . 59
4.5.1 Explicit schemes as recurrent tunable architectures . . . 60
4.5.2 Implementation . . . 63
4.5.3 Experiments . . . 64
4.6 Discussion and conclusions . . . 70

III CLASSIFYING DEPTH **73**

5 Randomized decision trees on tera-feature scale data for vision applications **75**

5.1 Motivation . . . 75
5.2 A brief overview of decision trees . . . 77
5.3 Past work on randomized tree building . . . 79
5.4 Novel approach to randomized tree induction . . . 81
5.4.1 Data partitioning . . . 84
5.4.2 Histograms for optimal splits . . . 85
5.5 Experiments . . . 87
5.6 Runtime inference . . . 93
5.6.1 FPGA pixel flow . . . 93
5.7 Conclusions . . . 95

6 Fingertip detection and identification **97**

6.1 Introduction . . . 97
6.2 Building HandNet: Creation and annotation . . . 99
6.3 Hand localization . . . 101
6.4 Fingertip detection . . . 102
6.4.1 Random decision tree . . . 103
6.4.2 Convolutional neural network . . . 104
6.4.3 RDT and CNN input . . . 105
6.5 Derotation . . . 106
6.5.1 Orientation regression . . . 106
6.5.2 DeROT: Designing a derotation method . . . 107
6.5.3 Derotation with PCA and Procrustes . . . 108
6.6 Experiments . . . 108
6.6.1 Evaluation protocol and data . . . 108
6.6.2 Discussion . . . 109
6.7 Going beyond fingertips to full hand pose . . . 110
6.8 Conclusions and future work . . . 113

7 Discussion and Conclusions **115**

Hebrew Abstract **i**

List of Figures

2.1 Stereo iPhones . . . 12
2.2 Voxel carving setup . . . 13
2.3 Laser scanning on a mobile device . . . 14
2.4 Mobile structured light scanner . . . 14

3.1 Endoscopic perspective setup . . . 20
3.2 Light intensity model for surface mounted LEDs . . . 22
3.3 Reconstruction with shadows . . . 27
3.4 Reconstruction under strong light focus . . . 28
3.5 Comparison of reconstruction methods . . . 29
3.6 Reconstructions of real objects . . . 30

4.1 Examples of Patch-Beltrami framework image denoising . . . 36
4.2 Computational graph for Equation 4.17 . . . 42
4.3 Visualization of RGB cloud of noisy and denoised Baboon . . . 43
4.4 Behavior of $PSNR$ relative to β, image resolution and image content . . . 45
4.5 Contour maps higlighting denoising behavior of different Patch-Beltrami parameters . . . 45
4.6 Linear log-space relationship between $\beta^{-}2$ and σ . . . 46
4.7 Different iteration numbers for denoising of Lena's eye . . . 47
4.8 Method noise . . . 49
4.9 Removal of aliasing and block textures . . . 50
4.10 Comparison with BM3D . . . 51
4.11 Scanned face depth map denoising . . . 52
4.12 Example cloud patches . . . 56
4.13 PCA versus robust normal estimation . . . 58
4.14 Point cloud denoising of a scanned depth map . . . 60
4.15 Rendering of shapes denoised by the point cloud Patch-Beltrami method . . . 61
4.16 Computation graph for the Deep Patch-Beltrami . . . 61

4.17 Blender design tool example for DPB. 64
4.18 Montage of first stage learnt filters 67
4.19 Comparison of Beltrami, Patch-Beltrami and Deep Patch-Beltrami filtering for different iterations for zoomed in Cox eye image 68
4.20 Comparison of Beltrami, Patch-Beltrami and Deep Patch-Beltrami filtering for different iterations for full Cox image 69
4.21 Venn diagram noise comparison 69

5.1 Data preloading pattern for multiple features 82
5.2 Forward-reverse indexing 84
5.3 Sample reordering 85
5.4 Histogram kernel probing 87
5.5 Comparing tree decisions required to accuracy and dataset size 89
5.6 Training time speed-down relative to using *int8* datatype 90
5.7 Impact of different number of tested thresholds 90
5.8 Tree level training time 91
5.9 Training time speed-up relative to using 1 GPU 92
5.10 Impact of preloading features 92
5.11 Impact of number of classes on training time 93

6.1 Examples from HandNet test set detections 98
6.2 HandNet data capture setup 99
6.3 Available HandNet data annotations after calibration 100
6.4 Circular normalized coordinates 102
6.5 Examples of hand detections in a raw depth stream 102
6.6 Understanding derotation 103
6.7 Fingertip feature layout 105
6.8 Network layout for hand orientation regression 106
6.9 Network layout for fingertip heat map detector 106
6.10 Prediction of hand rotation coefficients 107
6.11 Synthetic and real examples of DeROT 107
6.12 Precision and recall for thumb detection 110
6.13 Accumulated location heatmap for fingertips 111
6.14 Results of matching and then fitting a source pose to a target pose 112

Abstract

You don't need eyes to see, you need vision

Maxi Jazz, Faithless

In this dissertation we explore various geometric elements along the computational vision pipeline. By combining geometric principles for shape analysis with modern sensing techniques, large-scale datasets and powerful computational architectures we show various new ways of enabling computers to better perceive, interpret and comprehend the geometry of the world around them. Specifically we explore the topics of reconstruction, filtering and semantic processing within the context of computational geometric vision. We start by briefly covering approaches for reconstructing geometry on mobile devices. We then develop a method of performing efficient photometric stereo which can model non-linear and near-source lighting setups while avoiding directly computing normal fields.

Another aspect of processing which we focus on is that of denoising. Image and shape data which is obtained through modern cameras is often noisy. We contribute a new patch based data denoising framework called the Patch-Beltrami filter for denoising grayscale and color images and extend it to depth fields and 3D meshes. We then modify the approach by reformulating the differential operators used as trainable kernels in a deep neural network and unrolling the update step through time. We demonstrate state of the art results and highlight the fact that other PDE based methods could take advantage of the same basic idea.

In the last part of this work we discuss the problem of localizing and identifying self similar geometric objects in a complex visual space. We specifically focus on identifying fingertips on articulating hands observed by depth cameras. We describe how we used high accuracy magnetic sensors to annotate large quantities of training data. To perform learning efficiently we turn to randomized trees and contribute a new approach for efficiently mapping the training of a random decision tree on billions of training samples with trillions of data points to a single multi-GPU computing node. Similarly for inference we describe a novel pipelined FPGA hardware implementation.

Chapter 1

Introduction

This work aims at pushing forward the boundaries of human understanding of how we can provide computers with the ability to interpret and comprehend the geometry of the world around us. To do this we need to build up a set of theoretical and computational tools which will enable our exploration into mostly unexplored and non-unified territory. We take significant inspiration from the now successful approach of using deep neural networks and very large databases for textual and 2D imaging domains and apply this to the realm of 3D shapes and their representations. Our line of research is part of a diversion from a broader approach in which geometrical analysis is considered to be a fundamental truth of how shapes should behave and be treated. However to truly embrace the scientific approach as the process of discovering how the world works we will need to take a less purist view. In fact, the famous mathematician Henri Poincaré once said "Geometry is not true, it is advantageous". To some, this may sound like a sort of blasphemy as many hold the principles of geometry to be almost divine. However it is precisely in this light that we embarked on our scientific endeavor. In other words the world is messy but geometry provides a good lens through which to observe it. By combining geometric principles for shape analysis with modern sensing techniques, very large datasets and powerful computational architectures we show in this work various new ways of enabling our electrical counterparts to perceive the world around them.

1.1 A 3D world

The physical world in which we live and interact is comprised of three spatial dimensions. However the information which passes through our eyes and those of camera sensors is just a projection from this higher dimensional 3D space to a lower dimensional, lossy 2D raw representation. In some sense both the our biology and the technology we develop are limited in their capacity to fully represent the intrinisc geometric structure of the world around us. Despite this limitation, as a species we have invested extensive resources into producing and consuming flat screen and sensor technologies which enable at least an

approximate representation format through the use of high pixel density display screens and photo-sensor grids for use in cameras. This augmented expansion of our collective capacity for sensing the world has recently transferred itself to the realm of geometric depth sensing and manifested in popular scanning devices like Kinect [127] and RealSense [16]. Indeed, during the course of the research carried out here we have observed a dramatic increase in the number and variety of depth cameras available as well as the different methods that one can use to obtain depth and 3D information from the environment. From medicine, robotics, geography all the way to entertainment and direct computer interfaces the range of applications that can use depth and 3D information span almost the entire range of human endeavors.

Given the current advanced state of depth reconstruction it is interesting that there are still a large number of use-cases which can benefit from either improved methodologies or new approaches. Recent years have seen a dramatic increase in the available computing power as well as the mobility of this computational capacity. In Chapter 2 of Part I of this manuscript we briefly cover a number of different experiments we performed for producing depth and 3D structure on mobile devices such as smartphones. It also provides insight into the wide variety of techniques available for reconstructing the geometry of the world . Indeed the methods are as varied as the applications themselves but each typically has a set of properties which define its overall behavior.

One such property is whether a method is direct or indirect. Direct methods gather all the information they need to produce a first estimate of true geometrical quantities in one pass over the data. They may use further passes to refine this estimate but direct methods essentially produce all the information required after all the available data has been consumed and processed and sometimes well before. Some examples of the direct approach would be structured light systems [119, 37, 11, 153, 16] and time-of-flight cameras [61, 51, 107]. The opposite to these approaches is the family of indirect methods. These typically require first processing all the available data to first determine an indirect quantity such as illumination direction, surface normals or image focus of an imaged scene and then integrating this information under various natural and known constraints to arrive at the final result. Well known examples of this are photometric stereo [64, 149, 146, 102], shape from shading [102, 146, 149] and shape from defocus [48, 120]. Part I of this work continues in Chapter 3 to describe our work in developing a unique PDE variation of shape from photometric stereo under both close range perspective camera viewing and nearby point light sources. Our method diverges from the typical indirect photometric stereo approaches and instead uses an iterative approach to directly yield an estimated depth of the observed scene. The PDE approach also naturally yields a way to handle occlusions and we explore the method's behavior under situations of different noise and missing data conditions.

1.2 Removing noise

The multitude of imaging sensors available are all limited in some way by their engineering and the laws of physics. The result is that there are many different sources of noise that find their way into the output. Specifically in a vision pipeline low levels of light, electrical cross talk, heat, sensor layouts, compression artifacts and other confounding factors can all impact the final quality of the data produced and decades of research have been focused on how to partially or completely eliminate these issues. Deciding how to approach the problem of image denoising is dependent on constraints on the level of computational availability and the desired denoising efficacy. The most straightforward solution to image denoising is to treat the image as a noisy signal and perform low pass filtering using either simple neighborhood averaging or Gaussian blur. This linear filtering ignores the content of the image and treats it purely as a 2D signal with frequency components that can be understood and manipulated by tools from Fourier anlysis. The disadvantage however is that these techniques have no in-built capacity to account for structures such as edges and corners that appear in natural images. This has directly motivated the development of more advanced techniques which can overcome the inherrent limitations of basic blurring operations. The major succesful approaches for image denoising include median [27] and bilateral filtering [135], sparse-land models [45], wavelet coefficient shrinkage [40, 23], collaborative patch filtering [17, 34], anisotropic diffusion models[109, 128, 137] and more recently deep neural architectures [28]. During the course of our research we have developed a new framework for denoising which we call the Patch Beltrami framework. In Part II of this thesis we demonstrate how we take a principled geometric approach to dealing with denoising in both regular domains such as images and irregular domains such as point clouds. The framework enables performing denoising using a geometrically inspired approach, yields results comparable to state of the art and takes advantage of the tools and perspectives enabled by deep learning.

1.3 Comprehension: going beyond the data

The guiding motivation for the work in this thesis is the idea that good quality, multimodal data inputs can be used to enable more effective human computer interaction. After speach, the most powerful instrument for indicating human intention is through the use of our hands. The human hand is a highly efficient instrument that has evolved over millions of years to impart an evolutionary advantage by enabling the use of tools and non-verbal human-human communication. The modern computing era has necessitated that in order to impart our intention to a machine we must project our multi-digit dexterous capabilities down to two degrees of freedom. In other words we are slaves to a virtual pointer moving around a mostly flat environment. More recently with the advent of touch enabled surfaces our capacity for directing a computer to do our bidding has been dramatically enhanced. It is clear however that despite this

improvement in usability, the interaction with the machines that pervade our lives is still a poor and limited imitation of that between two people. In the modern computing age the onus is thus now on researchers to develop more natural and reliable interaction modalities that take advantage of advances in computing power. The successful construction of a fully functional hand pose estimation system for close range interaction will have far reaching implications. It is therefore precisely the problem of localization and pose estimation that we are interested in directing our attention towards.

To do so we will need to create and access a vast amount of information that represents as closely as possible the full span of potential hand motion and poses. Enabling machine comprehenseion of the essence of this data without slowing down effective research and development is difficult. While there are many different potential approaches to do so decision trees and their randomized variants have proven particularly popular because of their simplicty and ease of both implementation and analysis. In Part III of this thesis we tackle the above challenges by first describing in Chapter 5 a novel mapping of a randomized tree learning paradigm to parallel hardware and then exploring how this and other powerful learning mechanisms can be used to allow human like semantic understanding of parts of a hand as captured by a depth camera. In Chapter 6 we describe a novel approach to compacting the space of hand poses by partially mitigating the effects of hand rotation which simplifies the space on which we need to run our learning procedures. We also cover our approach for semi-automatically annoptating a large corpus of hand poses with accurate fingertip labels. The end result of our efforts is that our pipeline can learn on billions of training examples on a single modern GPU. The learnt model can be applied at above realtime speeds on commodity hardware to enable effective fingertip detection which is a building block to fully enabling human computer interactions.

1.4 In summary

This thesis focuses on exploring many elements of computational geometric vision. It can be argued that until recently, geometrical analysis has been considered to be a fundamental truth about how shapes should behave and be treated. But to truly embrace the scientific approach as the process of discovering how the world works, will require taking a less purist view. Henri Poincaré himself once said that “Geometry is not true, it is advantageous”. In other words, the world is messy, but geometry provides a good lens through which to observe it. By combining geometric principles for shape analysis with very large datasets and powerful computational architectures we believe that we have a greater chance of enabling our electronic counterparts to perceive and comprehend us and the world around them. It is precisely in this light that we embarked on the research described in the following manuscript.

Part I

RECONSTRUCTING GEOMETRY

Chapter 2

Mobile reconstruction

The significant advancements in mobile computing and the ubiquitous availability of mobile photographic hardware in smartphones and other consumer devices implies that future reconstruction algorithms may benefit from having a mobile specific focus. Recent years have indeed seen an increase in the ammount of academic and industrial activity focused around embedding both depth sensing devices and improved reconstruction algorithms in small devices such as handheld tablets and smrtphpnes. During the course of the research for this thesis we explored a number of different potential approaches to performing mobile based reconstruction directly on the device. Although we will not discuss any of the approaches at length, it is worth covering each one briefly to give context to the problem of 3D reconstruction in a primarily human focused use case where people or every day objects are the scanning targets.

Geometry capture processes can be split into active or passive. Active methods shine light or other waveforms which can be sensed by a receiving sensor whereas passive methods absorb ambient light or signals from the environment. Active methods are more challenging to construct but are typically more accurate as they can control the information broadcast to the scene so as to maximize their capacity to receive it back and process it efficiently. We will now describe two passive approaches and then two active approaches for performing mobile geometry capture.

2.1 Stereo devices

Stereo reconstruction is one of the oldest techniques in use by the vision community for obtaining depth information about a given scene [multiple refs]. Using two 2D images of the scene taken simultaneously from different views corresponding points in both imagestogether with internal parameters of each camera can be used to restore the geometry of the observed object. To explore the possibility of using stereo shape reconstruction in a mobile scenario we rigidly joined two iPhone devices and callibrated their intrinisic and extrinisic parameters using [12]. Figure 2.1 illustrates the setup. Unlike typical stereo systems where

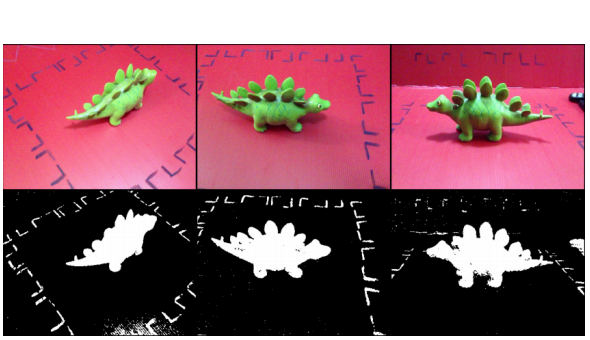
cameras share a common processing clock or are completely static while the scene is being captured, the use case we explored involved two devices with no common timing mechanism and typically hand held during capture resulting in somewhat unstable imaging conditions. To compensate we developed an average round trip time synchronization system and used Semi Global Block Matching [63] to compute a disparity map which we converted to a metrically correct mesh using the callibration information. The low quality of the resulting disparity maps together with the complexity of ensuring the correctness of the callibration system leads us to the conclusion that stereo systems are not good under mostly uncontrolled jolting and motion when handheld. A succesful system would require a common sensor substrate with a common lensing system and careful consideration of the effects of temperature and vibration.

Figure 2.1: Two iPhones rigidly connected to enable a consistent frame of reference while moving. Setting up the callibration between the two phones requires synchronizing their clocks so that capture is performed at the same time.

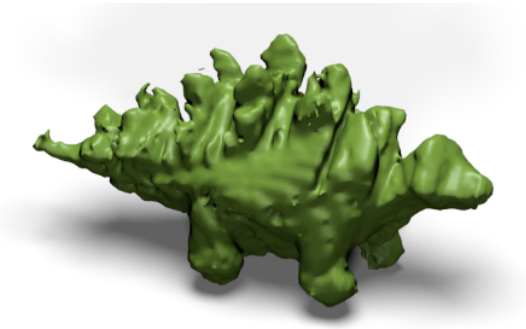
2.2 Voxel carving

Among the many approaches to 3D reconstruction, voxel carving [79] stands out as one of the more straightforward. However the simplicty comes with two restrictions. The first is that the object must be easy to segment out from a known background and the second is that the reconstruction accuracy will be limited for concave shapes or those with holes because reconstruction is based on the visual hull images by a camera from multiple directions. A virtual cube of voxels is gradually carved into the shape of the object by removing non silhouette pixels which project into a ray in space and imply that there is no voxel occupancy where the ray is. A further challenge wih this approach is that one needs to obtain the location of the camera with respect to the object in a common world frame so as to correctly project rays through the volume. We solved this by using the PointCloud SDK which was a commercially available 6dof SLAM software package in 2014 but has since been discontinued. We obtained the geometry from

the resulting voxel volume using the classic marching cubes method [88]. Example segmentations from our method can be seen in figure 2.2(a) and a result is shown in figure 2.2(b). Although this method is not optimal nor generally applicable it may indeed be useful in future body reconstruction approaches when when combined with semantic human segmentation methods as a body has few visible cavities and the visual hull is an accurate representation of the human form.



(a) Top: input images captured on device, bottom: resulting segmentation masks

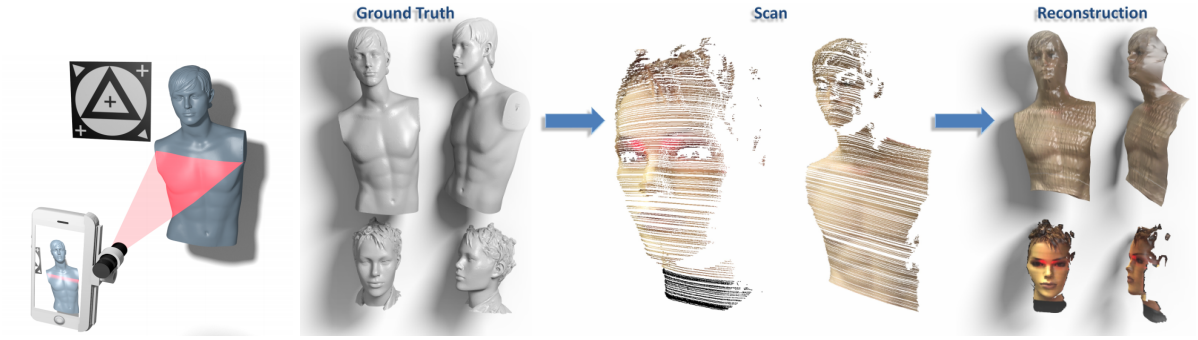


(b) Color rendering of reconstructed mesh after Marching Cubes

Figure 2.2: Setup and result of our voxel carving approach for mobile voxel reconstruction. Noisy segmentations are non destructive provided no data has been incorrectly segmented as background. The noisy is typically carved away with more images. Incorrect holes in even one image of the segmentation however results in incorrect holes carved in the result.

2.3 Laser scanning

Laser line based scanning is a method which has become popular in industrial applications for 3D object capture as it is very precise and repeatable. Our work in [114] demonstrates a novel approach for mobile phone scanning which utilizes a smart-phone and cheap laser pointer with a cylindrical lens which produces a line pattern attached to the phone using a 3D printed adapter. Non-linear multi-scale line filtering is used to detect the center of the projected laser beam in each frame with sub-pixel accuracy. The line location coupled with the estimated phone position and orientation in 3D space, obtained from publicly available SLAM libraries and marker tracking, permits us to perform a 3D reconstruction of a point cloud of the observed objects. Color and texture are extracted for every point along the scanned line point by projecting the reconstructed points back onto previous keyframed images. The proposed method is validated by comparing the reconstruction error to the ground truth obtained from an industrial laser scanner. Figure 2.3(a) shows the setup while figure 2.3(b) shows qualitative results. A well callibrated, fast scanning laser that is eyesafe coupled with an embedded processor could indeed be a viable approach for widespread distribution on a smartphone or tablet.



(a) Capture setup of laser unit attached to device.

(b) Qualitative results of scanning output. Note the final reconstruction is performed offline to improve the rendering quality.

Figure 2.3: This figure illustrates the setup and result of our laser scanning approach for mobile laser reconstruction. The marker is used for creating a world reference frame. The initial scanlines from the resulting laser scan yield 3D structure which is then integrated in a surface completion pipeline.

2.4 Beam scanner

Structured light scanning has become increasingly popular in recent years because of the availability of relatively cheap projection systems. By using the embedded pico projector of the Samsung Galaxy Beam smartphone we were able to produce a fully operational coded light mobile scanner based on the patterns and reconstruction process described in [117]. Figure 2.4 illustrates our setup. To the best of our knowledge this was the first use of a commodity smartphone device using an inbuilt projector for structured light reconstruction on a mobile device. While our initial results were neither fast nor accurate the method seems particularly well suited for close range scanning. It is clear however that different surfaces with different reflectivity will yield different reconstructions and thus surfaces such as hair or metal are likely to be problematic.

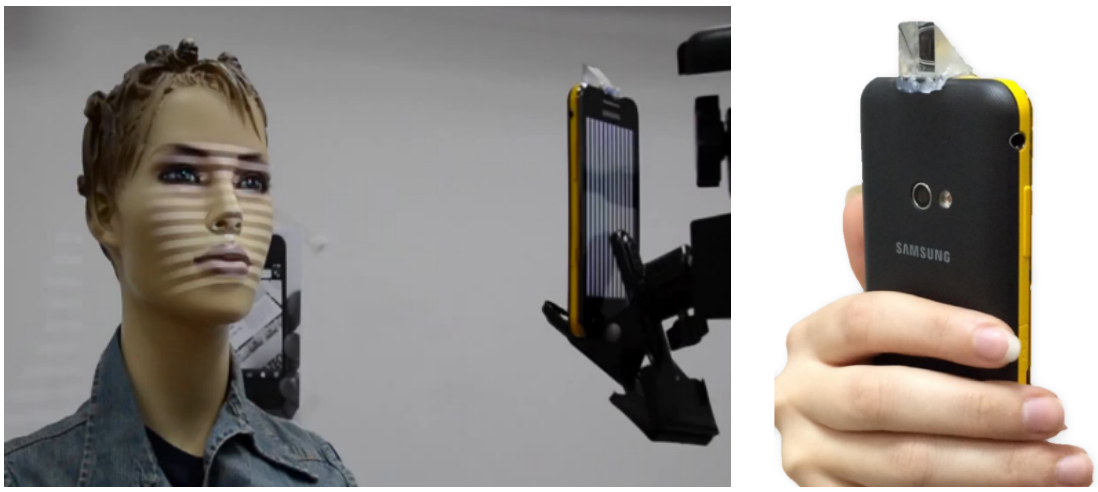


Figure 2.4: Setup of the structured light scanner using the Samsung Beam. Notice that the device cannot be handheld because of prototype setups limitation in capture speed. Modern structred light scanners such as the Intel RealSense can achieve framerates of $60 fps$ or higher.

2.5 Conclusion

In this chapter we have briefly covered a number of different possible approaches for implementing mobile based 3D scanning systems. While each of the methods we exlored has both advantages and disadvantages it is interesting to note that various large companies such as Apple, Intel, Google and Facebook with large research and development teams have all chosen to use active coded light reconstruction approaches in research and in real products.

Chapter 3

Photometric stereo

Shape recovery based on shading variations of a lighted object was recently revisited with improvements that allow for the photometric stereo approach to serve as a competitive alternative for other shape reconstruction methods. However, most efforts of using photometric stereo tend to ignore some factors that are relevant in practical applications. The approach we consider tackles the photometric stereo reconstruction in the case of near-field imaging which means that both camera and light sources are close to the imaged object. The known challenges that characterize the problem involve perspective viewing geometry, attenuation of light and possibly missing regions. Here, we pay special attention to the question of how to faithfully model these aspects and by the same token design an efficient and robust numerical solver. We present a well-posed mathematical representation that integrates the above assumptions into a single coherent model. The surface reconstruction in our near-field scenario can then be executed efficiently in linear time. The merging strategy of the irradiance equations provided for each light source allows us to consider a characteristic expansion model which enables the direct computation of the surface. We evaluate several types of light attenuation models with nonuniform albedo and noise on synthetic data using four virtual sources. We also demonstrate the proposed method on surface reconstruction of real data using three images, each one taken with a different light source.

3.1 Perspective On Shape from Shading and Photometric Stereo

Since the seminal Shape from Shading paper by B.K.P. Horn [64], new models have been introduced in order to extend the range of shape recovery problems that can be solved by Shape from Shading (SfS) methods [102, 146, 149]. A particularly important direction of research has been the transition from the assumption of orthographic viewing geometry [76, 90, 65] to the more realistic one of perspective cameras [112].

Several papers reported using Perspective SfS (PSfS) methods applied to endoscopic image analysis

[149, 146, 102]. Deguchi *et al*let@tokeneonedot [102] introduced the perspective viewing shape estimation for objects located close to the camera, taking into account a realistic endoscopic model for close light source illumination (i.e. ideally attached to the optical center). Later, Prados et al. [112] studied the same formulation under a *differential* point of view and, using a model based on Partial Differential Equations (PDEs), they concluded that their formulation for the endoscopic perspective shape from shading problem is well-posed. A recent paper by Breuß et al. [15] shows that there might be ambiguity in the Prados et al. model. In fact, near-field Endoscopic Perspective (EP) shape reconstruction remains an open problem if only a single image is considered.

In this paper we deal with the case of calibrated Photometric Stereo (PS), where multiple images of the same scene are taken under different illumination conditions (with known light positions) when scanner and object are close to each other. While there are methods for uncallibrated PS such as Basri *et al*let@tokeneonedot [10, 145] we instead focus on a known callibrated light and setup. For a recent overview about calibrated (and uncalibrated) PS methods see [106]. In the EP framework Wu et al. [146] studied the multi-image EP problem by considering two light sources placed off the optical center. They develop a model based on the radiance information obtained by simultaneously illuminating an object with two different light sources. They then recover the surface by considering a single irradiance equation for the sum of Lambertian reflectance functions of the two different light sources. The use of this reflectance function results in a loss of information. In order to avoid this problem and issues related to unknown albedo, they use a photometric calibration. Surface recovery is performed within a variational framework that involves high computational complexity compared to alternative direct methods [89]. The shape from EP problem solved via a PS technique using n images (EPPS$_n$) is a problem initially addressed by Iwahori *et al*let@tokeneonedot [68], Clark [29] and more recently Migita *et al*let@tokeneonedot [96]. Collins and Bartoli [30] solve the close-range PS problem with weak considerations on the parametrization of the variables like unknown surface, light sources and solving the problem of the light attenuation with a-priori light calibration procedure. They also use a prior for a reflectance model, adding patches on the inspected object even if the surface is assumed to be Lambertian. In particular, their mathematical formulation is based on the usual double step procedure where an energy functional is minimized (which allows the computation of the surface derivatives) and thereafter the surface is recovered [53, 2]. Let us emphasize that their energy is based on the sum of Lambertian irradiance equations and this is a sort of contradictory procedure with respect to the scientific trend using photometric ratios [144, 89, 21] which yield more suitable problems. For example the ratio approach effectively makes the problem independent from the albedo.

Recently Parot et al. [108] studied the EPPS$_n$ by using the classic approach to PS and using a heuristic filtering process. They solve for the normal field to the surface as if the light sources were distant and

then filter the directional gradients depending on the frequencies. The depth map is then computed using a multi grid solver for the Poisson equation. The applicability of this method is very limited since they calibrate the uniform light directions assuming *reasonable* distance between the object and the scanner. This allows to exaggerate surface features but not reconstruct the actual surface depths.

Here, we present a new mathematical formulation for the EPPS$_n$ by considering the endoscopic perspective presented in [102] and using the perspective parametrization used in [91] based on non-linear PDEs from photometric ratios. Such an extension introduces several realistic features which make the new model mathematically interesting. The formulation of a new mathematical model based on quasi-linear PDEs is followed by an efficient finite difference upwind numerical scheme for the direct approximation of the surface. This scheme is simple to implement, highly parallelizable and converges efficiently. The new model we describe can also successfully handle images with missing data. Usually the image acquisition procedure for PS includes nuisance factors such as occlusions and shadows. Here we do not focus on occlusions or shadows detection, but we show that our problem successfully reconstructs surfaces when images have missing parts. Furthermore, a significant advantage of our model is the direct computation of the 3D surface without explicit computation of the surface normals for the purpose of integration. Non-linearities resulting from our realistic lighting model yield a normal field that depends on the depth of the surface as well as the direction of the light source. However, this does not pose a problem in our framework because we avoid directly computing the normal field and using it in the numerical scheme.

In section 3.2 we recall a well known parametrization for the surface under close camera observation. The new mathematical model is introduced in section 3.3 where we show the differential model including non-linear light attenuation. Section 3.4 introduces the theoretical formulation of the new differential approach for three images which can be easily extended to $n \geq 3$. Using this formulation we will show how it is possible to overcome the problem of handling images with missing regions. In section 3.5 we solve the problem of EPPS$_3$ and EPPS$_4$ when there are images with missing parts and an unknown albedo. A portion of that section is also devoted to the explanation of the up-wind scheme used in the numerical tests. Finally, in section 3.6 the experimental results on synthetic and real data are presented.

3.2 The Endoscopic Perspective Set-Up

In order to give all the necessary ingredients to understand the geometry behind the model we start by considering the parametrization of the surface Σ (see figure 3.1) given in [112] up to an unknown function z from the image domain $\overline{\Omega}_p = \Omega_p \cup \partial\Omega_p$ to $\mathbb{R}$ such that:

$$\mathbf{M}(x,y) = [\xi(x,y), \eta(x,y), \zeta(x,y)] := \left[-x\frac{z(x,y)}{f}, -y\frac{z(x,y)}{f}, z(x,y) \right]. \tag{3.1}$$

Here, $f > 0$ is the focal length of the camera, $\zeta < -f < 0$, and the triple $[\xi(x,y), \eta(x,y), \zeta(x,y)] = [\xi, \eta, \zeta]$ are the real world coordinates (with respect to the image coordinates). The parametrization Equation (3.1) is based on the pinhole camera model and is due to the specific perspective viewing geometry as seen in figure 3.1 where the camera is placed at the origin C of the coordinate system $C\xi\eta\zeta$ (namely at the optical center) [112].

We recall that an outgoing vector normal to the surface Σ is

$$\overline{\mathbf{n}}(x,y) = \frac{z}{f^2}\big[f\nabla z(x,y), z(x,y) + (x,y)\cdot\nabla z(x,y)\big], \tag{3.2}$$

and since the irradiance equation depends on the unit normal, we take $\mathbf{n}(x,y) = \frac{\overline{\mathbf{n}}(x,y)}{|\overline{\mathbf{n}}(x,y)|}$ as a unit length normal vector.

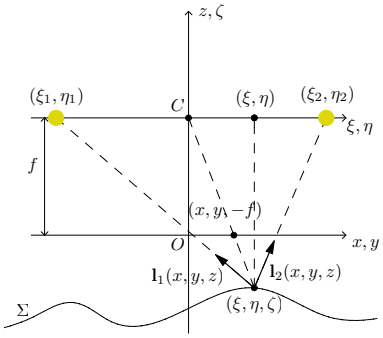


Figure 3.1: In the perspective world (i.e. the image coordinate system given by $Oxyz$) the light reflected at image point (x,y) comes from the real point $[\xi, \eta, \zeta]$ of the surface.

We consider the well-known irradiance equation for Lambertian surfaces, given by the cosine law by the following inner product

$$I(x,y) = \rho(x,y)(\mathbf{n}(x,y)\cdot\mathbf{l}(x,y,\ldots)), \tag{3.3}$$

where $I : \overline{\Omega}_p \to [0,1]$ is the image function, $\rho(x,y)$ is the unknown albedo and $\mathbf{l}(x,y,\ldots)$ is the light source direction. Let us emphasize that the light direction $\mathbf{l}$ incident to the surface depends explicitly on the image points (x,y) and on other factors $(\ldots)$ we shall specify in the next section, since the endoscopic formulation assumes close light source illumination.

3.3 A New Close-Range PS Model

In much of the literature dealing with Perspective SfS [112, 102], a single light source is considered to be placed at the optical center. This assumption is unrealistic when considering a camera close to the inspected object. Since PS uses several light sources, we consider a considerably more realistic placement (ξ_j, η_j) on the optical plane, that is $\zeta = 0$. The coplanarity of the sources is not a necessary constraint but

we adopt it here to simplify our formulation. We define the light directions as

$$\mathbf{l}_j(x,y,z) = \frac{\left[-\frac{\xi_j f}{z} - x, -\frac{\eta_j f}{z} - y, f\right]}{\sqrt{\left(x+\frac{\xi_j f}{z}\right)^2 + \left(y+\frac{\eta_j f}{z}\right)^2 + f^2}} = \frac{\bar{\mathbf{l}}_j(x,y,z)}{q_j(x,y,z)}, \tag{3.4}$$

where

$$q_j(x,y,z) = \sqrt{\left(x+\frac{\xi_j f}{z}\right)^2 + \left(y+\frac{\eta_j f}{z}\right)^2 + f^2}. \tag{3.5}$$

Let us emphasize that for this model, the light directions depend not only on the point (x, y), but since they are displaced from the optical center, they also depend on z. This introduces a non-linearity that does not involve the derivatives of z.

3.3.1 Two types of light attenuation

We now consider two different kinds of light attenuation. The first factor is due to the reduction of light energy proportional to the inverse squared distance between the light source and object. The second factor of attenuation we describe, is a result of a realistic directional lighting model of a real SMD light. However in principle we could choose among many different continuous attenuation models to suit our needs.

Light attenuation due to distance

The standard way to model this attenuation of the light intensity is to compute the distance between the light source and the surface explicitly. For this purpose, let us consider the functions $\mathbf{r}_1(x,y,z)$ and $\mathbf{r}_2(x,y,z)$ as the distances between the point of the surface $[\xi,\eta,\zeta]$ and the respective light source. In this case, since the light sources are shifted with respect to the origin, the distance function is as follows:

$$r_j(x,y,z) = \text{dist}\left((\xi_j,\eta_j,0), \left(-x\frac{z}{f}, -y\frac{z}{f}, z\right)\right) = \\ \frac{z}{f}\sqrt{\left(x+\frac{\xi_j f}{z}\right)^2 + \left(y+\frac{\eta_j f}{z}\right)^2 + f^2} = \frac{z}{f} q_j(x,y,z). \tag{3.6}$$

The attenuation factor can then be written as r_j^{-2}.

Radial attenuation of the light

Many existing light sources are directional. That is to say, they are bright along a principal direction and become less bright at angles further from the main direction. This behavior can be observed in figure 3.2 and can be effectively simulated by multiplication with $\cos(\theta)^\mu$ where μ is the attenuation coefficient and is reminiscent of the specular model for surface reflectance.

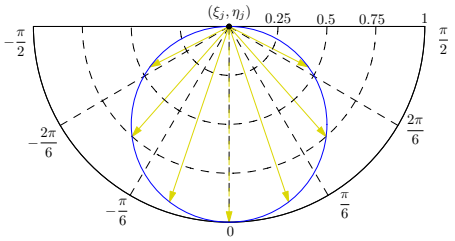


Figure 3.2: The intensity of light for a directional light source pointing downwards. This particular model is directly inspired by SMD LEDs which produce strong light intensity orthogonal to the surface they are placed on and this approaches zero for directions parallel to the surface.

The attenuation factor is easily computable since

$$\cos(\theta)^{\mu}(x,y,z) = (\mathbf{l}_j(x,y,z)\cdot(0,0,1))^{\mu} = \frac{f^{\mu}}{q_j^{\mu}(x,y,z)} \tag{3.7}$$

where $\mathbf{l}_j(x,y,z)$ is the i-th light source placed at (ξ_j,η_j). The light direction is collinear with the viewing direction of the camera which simplifies both our formulation and the experimental setup. Both attenuation effects can be expressed by multiplication of the following factor

$$\mathrm{a}_j(x,y,z) = \frac{f^{\mu}}{r_j^2(x,y,z)q_j^{\mu}(x,y,z)} = \frac{f^{\mu+2}}{z^2 q_j^{\mu+2}(x,y,z)}. \tag{3.8}$$

3.3.2 EPPS$_2$ model

Our model for the SfEPPS$_2$ problem considers the following irradiance equations

$$\begin{aligned} I_j(x,y) =& \rho(x,y)\frac{\bar{\mathbf{l}}_j(x,y,z)\cdot\mathbf{n}(x,y)}{r_j^2(x,y,z)q_j^{\mu+1}(x,y,z)}f^{\mu} = \\ & \rho(x,y)\frac{\bar{\mathbf{l}}_j(x,y,z)\cdot\overline{\mathbf{n}}(x,y)f^{\mu+2}}{z^2 q_j^{\mu+3}(x,y,z)|\overline{\mathbf{n}}(x,y)|} \end{aligned} \tag{3.9}$$

where

$$\bar{\mathbf{l}}_j(x,y,z)\cdot\overline{\mathbf{n}}(x,y) = -\xi_j z_x - \eta_j z_y + \frac{z^2}{f}. \tag{3.10}$$

Now, solving the EPPS$_2$ from a mathematical point of view consists of solving the following system of non-linear PDEs of Hamilton-Jacobi type,

$$\begin{cases} I_1(x,y) = \rho(x,y)\dfrac{\bar{\mathbf{l}}_1(x,y,z)\cdot\overline{\mathbf{n}}(x,y)f^{\mu+2}}{z^2 q_1^{\mu+3}(x,y,z)|\overline{\mathbf{n}}(x,y)|} & \text{on } \Omega_p \\ I_2(x,y) = \rho(x,y)\dfrac{\bar{\mathbf{l}}_2(x,y,z)\cdot\overline{n}(x,y)f^{\mu+2}}{z^2 q_2^{\mu+3}(x,y,z)|\overline{\mathbf{n}}(x,y)|} & \text{on } \Omega_p \\ z(x,y) = g(x,y) & \text{on } \partial\Omega_p \end{cases}. \tag{3.11}$$

Our strategy to solve such a problem is to merge the irradiance equations (3.11) by noting that the

non-vanishing quantity $\frac{\rho(x,y)f^{\mu+2}}{|\overline{\mathbf{n}}(x,y)|z^2}$ is present in both of them. We merge such equations as follows

$$\underbrace{\frac{I_1(x,y)q_1^{\mu+3}(x,y,z)}{\bar{\mathbf{l}}_1(x,y,z)\cdot\overline{\mathbf{n}}(x,y)} = \overbrace{\frac{\rho(x,y)f^{\mu+2}}{|\overline{\mathbf{n}}(x,y)|z^2}}^{\text{Second equation of (3.11)}}}_{\text{First equation of (3.11)}} = \frac{I_2(x,y)q_2^{\mu+3}(x,y,z)}{\bar{\mathbf{l}}_2(x,y,z)\cdot\overline{\mathbf{n}}(x,y)}$$

getting

$$\frac{I_1(x,y)q_1^{\mu+3}(x,y,z)}{-\xi_1 z_x - \eta_1 z_y + \frac{z^2}{f}} = \frac{I_2(x,y)q_2^{\mu+3}(x,y,z)}{-\xi_2 z_x - \eta_2 z_y + \frac{z^2}{f}}. \tag{3.12}$$

Let us emphasize that this way of merging the irradiance equations, allows us to eliminate the non-linearity with respect to the partial derivatives of z contained in $|\overline{\mathbf{n}}(x,y)|$. This makes the resulting problem completely independent from the albedo and the strategy is an incremental development over [144], where the ratio idea originated.

After some algebra, we obtain the following first-order quasi-linear PDE

$$\begin{aligned} z_x\Big(I_1(x,y)q_1^{\mu+3}(x,y,z)\xi_2 - I_2(x,y)q_2^{\mu+3}(x,y,z)\xi_1\Big) + \\ z_y\Big(I_1(x,y)q_1^{\mu+3}(x,y,z)\eta_2 - I_2(x,y)q_2^{\mu+3}(x,y,z)\eta_1\Big) = \\ \frac{z^2}{f}\Big(I_1(x,y)q_1^{\mu+3}(x,y,z) - I_2(x,y)q_2^{\mu+3}(x,y,z)\Big), \end{aligned} \tag{3.13}$$

resumed as follows

$$\begin{cases} \mathbf{b}(x,y,z)\cdot\nabla z(x,y) = s(x,y,z), & \text{on } \Omega_p \\ z(x,y) = g(x,y) & \text{on } \partial\Omega_p, \end{cases} \tag{3.14}$$

where $g(x,y)$ is the Dirichlet boundary condition,

$$\begin{aligned} \mathbf{b}(x,y,z) = \Big[&I_1(x,y)q_1^{\mu+3}(x,y,z)\xi_2 - I_2(x,y)q_2^{\mu+3}(x,y,z)\xi_1, \\ &I_1(x,y)q_1^{\mu+3}(x,y,z)\eta_2 - I_2(x,y)q_2^{\mu+3}(x,y,z)\eta_1\Big] \end{aligned}$$

and

$$s(x,y,z) = \frac{z^2}{f}\Big(I_1(x,y)q_1^{\mu+3}(x,y,z) - I_2(x,y)q_2^{\mu+3}(x,y,z)\Big).$$

3.4 Direct Surface Reconstruction Using Images with Missing Parts

If we have three images then we can consider the set of unique image pairs and linearly combine them following the same procedure as [91]. We are thus able to define the EPPS$_n$ problem by exploiting the linearity of the basic differential formulation (3.14) and reducing it to a single quasi-linear PDE which

can handle missing regions in a natural fashion.

Since we want to exploit the PS technique, we assume that each pixel is illuminated in at least two images thereby avoiding reduction to a EPPS_1 problem. This means that we need a minimum of two light sources to proceed with our method.

In order to complete the theoretical analysis, we note that the uniqueness of the weak (i.e. Lipschitz, $z \in \text{Lip}(\overline{\Omega}_p)$) solution for the differential problem (3.14) can be proved ([89]). Here an important comment must be made regarding the light attenuation. In the previous section we introduced the new model (3.14) with two specific light attenuations because for real tests we used SMD lights with these photometric characteristics. However, well-posedness is guaranteed even for general attenuation functions (3.8). In the following sections we show how we can still reconstruct the surface from three images, even if they have non-overlapped missing parts, treating the new differential problem with a general attenuation function $a_j(x, y, z)$.

3.5 W-EPPS$_n$ with Almost no Boundary Condition

In this section we focus on the applicability of our model. In section 3.4 we extended the EPPS_n model by supposing the knowledge of the boundary condition $g(x, y)$. Clearly such a hypothesis compromises the use of that model for many real applications. It is therefore important to find a way to solve the EPPS_n problem while removing the requirement for a-priori knowledge of the boundary condition.

Researchers facing the SfS, PSfS or the EPPS problems often adopt a two-stage strategy where one first computes the surface normals all over the domain and then integrates the normal field in order to perform surface recovery. Usually the normal computation is done by considering a linear system computed using the irradiance equations. Let us emphasise that when close light sources are taken into account the usual linearity of the Lambertian reflectance equation is lost. In this particular case, since the light sources are not placed at the optical center, the normalization of the light direction and the further light attenuations introduce a non-linearity with respect to z. In other words, given n images, it is very hard to compute the unknown z_x, z_y and z just by solving the following system of irradiance equations:

$$\begin{cases} I_1 = \rho(x,y)\mathrm{a}_1(x,y,z)\dfrac{\bar{\mathbf{l}}_1(x,y,z)\cdot\mathbf{n}(x,y)}{q_1(x,y,z)} \\ I_2 = \rho(x,y)\mathrm{a}_2(x,y,z)\dfrac{\bar{\mathbf{l}}_2(x,y,z)\cdot\mathbf{n}(x,y)}{q_2(x,y,z)} \\ \vdots \\ I_n = \rho(x,y)\mathrm{a}_n(x,y,z)\dfrac{\bar{\mathbf{l}}_n(x,y,z)\cdot\mathbf{n}(x,y)}{q_n(x,y,z)} \end{cases} . \tag{3.15}$$

The approach in this work is to compute z without completely eliminating the non-linearity. In fact, the

quasi-linear PDE of Equation (3.14) still contains a non-linear component, but the problem of recovering z can be successfully achieved directly by solving a quasi-linear PDE. Furthermore, partial images clearly represent a loss of information in the image set and several authors have approached the problem of surface recovery with occlusions [62, 22]. However we are not aware of attempts to overcome the problem of missing parts within the framework of the endoscopic problem. We take the above issues into consideration and use them to help design a numerical strategy for reconstructing the surface using our model. The strategy involves determining the depth of only a single arbitrarily placed initial seed point within the reconstruction domain and robustly manipulating the path of the characteristics from that point. We do this in order to let the information travel in the most convenient directions for the whole domain.

3.5.1 Steering the characteristic field

On the way to defining a numerical strategy we will need to manipulate the path along which the information travels. To do this we will exploit the following result:

Theorem *Let* $\mathbf{b}_t(x, y, z)$ *be the vector field of any pair of images where* $t \in \binom{[n]}{2}$[1]. *Then,* $\forall t_1, t_2 \in \binom{[n]}{2}$, $\forall (u, v) \in \Omega_p$ *and* $\forall z \in$ *Lip*$(\overline{\Omega}_p)$ *we have:*

$$\mathbf{b}_{t_1}(x, y, z) \cdot \mathbf{b}_{t_2}(x, y, z) \neq \pm |\mathbf{b}_{t_1}(x, y, z)||\mathbf{b}_{t_2}(x, y, z)|. \tag{3.16}$$

In other words, this theorem states that two different vector fields $\mathbf{b}_{t_1}$ and $\mathbf{b}_{t_2}$ can not be parallel. We can adopt the same fast marching strategy of [91] steering the direction of the characteristics for the case when $n = 3$.

We enumerate the steps in our characteristic steering method here:

1. fix the exact depth value to z for a point, in our case it shall be towards the center of the image domain adding all of that point's neighbors to a list of pixels to be visited;

2. traverse the list of pixels to be visited and update the value for z for each one by the scheme (3.18) derived in the next section;

3. for each newly visited pixel add its unvisited neighbors to the list of pixels to be visited;

4. in case of (non-overlapped) missing parts in the images, we can change the wavefront propagation direction in order to *surround* the shadow sets (i.e. computing the boundary condition) and then solve (3.14) with the appropriate pair of images;

5. the above steps are repeated until the L^∞ discrete norm of the difference between the last two elements of the approximating sequence is smaller than some predetermined threshold.

[1] $\binom{[n]}{2}$ is the set of pairs of integer indices with no repetition. For example, if $n = 3$, we have $\binom{[3]}{2} = \{(1, 2), (1, 3), (2, 3)\}$.

We remark that in this near-field set up, the convergence of the previous algorithm is not as straightforward as in [91] because here the vector fields $\mathbf{b}(x, y, z)$ depend on z, i.e. they are unknown. However, this aspect does not prevent the scheme from converging efficiently. The fast convergence behaviour of the problem requires a non-trivial proof showing the convergence of the numerical scheme with respect to [89], but we omit it here due to lack of space.

3.5.2 Numerical scheme

We now consider the numerical method that we employ to obtain the experimental results. The scheme considered originates from [89] where a finite difference up-wind scheme is used.

In order to simplify the notation we shall denote $\mathbf{b}(x_i, y_j, z(x_i, y_j))$ by $\mathbf{b}_{i,j}(z_{i,j}) = (b^1_{i,j}(z_{i,j}), b^2_{i,j}(z_{i,j}))$ and $s(x_i, y_j, z(x_i, y_j))$ by $s_{i,j}(z_{i,j})$.

Let us consider the following implicit up-wind scheme:

$$b^1_{i,j}(Z_{i,j})\frac{Z_{i+1,j} - Z_{i-1,j}}{2\Delta} + b^2_{i,j}(Z_{i,j})\frac{Z_{i,j+1} - Z_{i,j-1}}{2\Delta} = s_{i,j}(Z_{i,j}) + |b^1_{i,j}(Z_{i,j})|\frac{Z_{i+1,j} - 2Z_{i,j} + Z_{i-1,j}}{2\Delta} + |b^2_{i,j}(Z_{i,j})|\frac{Z_{i,j+1} - 2Z_{i,j} + Z_{i,j-1}}{2\Delta}. \quad (3.17)$$

The artificial diffusion introduced in the right side of (3.17) allows us to follow the vector field $\mathbf{b}$ by considering the most appropriate discretization for the fist derivative in order to track the characteristic lines. Specifically it consists of a numerical scheme of consistency order equal to one with respect to both partial derivatives.

We can write (3.17) explicitly as follows:

$$Z^{r+1}_{i,j} = \Big(|b^1_{i,j}(Z^r_{i,j})|Z^r_{i-\text{sign}\,(b^1_{i,j}(Z^r_{i,j})),j} + |b^2_{i,j}(Z^r_{i,j})|Z^r_{i,j-\text{sign}\,(b^2_{i,j}(Z^r_{i,j}))} + s_{i,j}(Z^r_{i,j})\Delta\Big)\frac{1}{|b^1_{i,j}(Z^r_{i,j})| + |b^2_{i,j}(Z^r_{i,j})|}. \quad (3.18)$$

3.6 Experimental Results

We now describe the full experimental procedure we followed while investigating the properties of the new model. The numerical schemes were all implemented in Matlab® MEX files in unoptimized C++ using OpenMP for the parallelization and executed in Matlab® using a 2013 Dell Precision M6700 with an Intel i7 CPU clocked at 31 GHz and 321 GB of RAM.

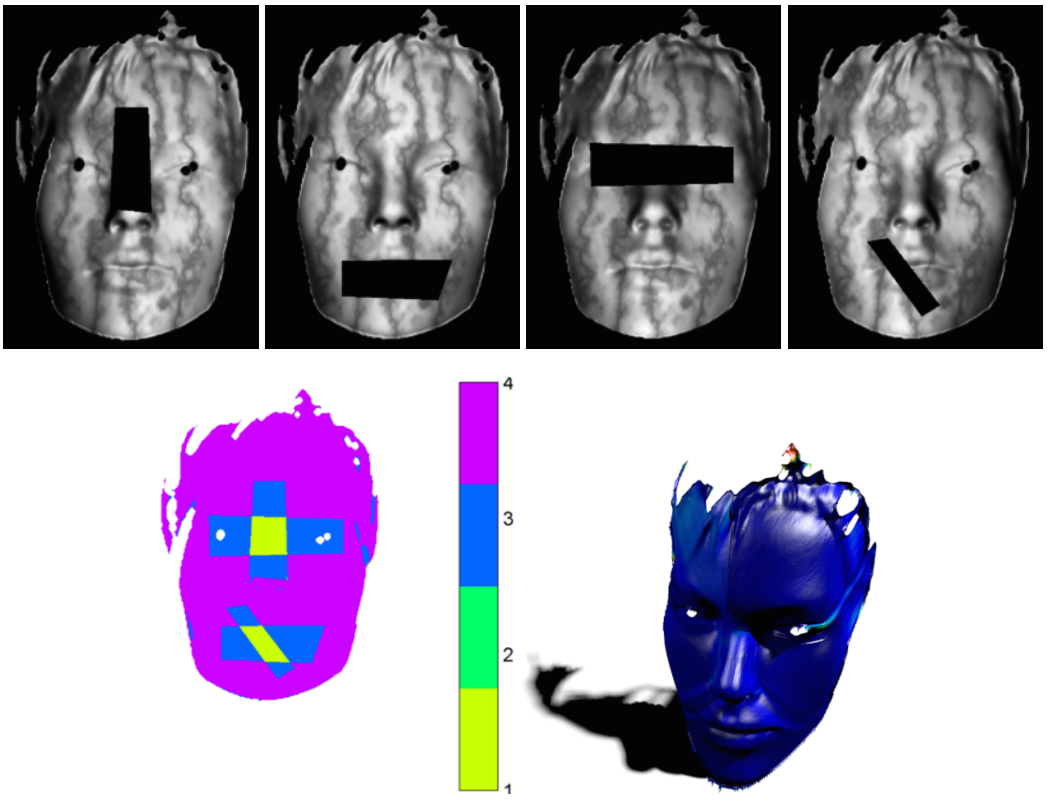


Figure 3.3: The input images with $\mu = 1$ are shown in the first row. The second row has (on the left) the shadow map and (on the right) the approximate 3D shape with the Euclidean error map textured on. $MSE = 0.52\,\mathrm{mm}^2$

3.6.1 Synthetic cases

Eve

We therefore first consider a realistic shape obtained from a mannequin head where real depth data has been obtained from a structured light range scanner. This is stored as a height field over a grid with 4-neighbor connectivity. The mannequin face (known as Eve) is approximately $70\,\mathrm{cm}$ from the virtual camera center. The virtual camera is a perspective pinhole camera and all depth pixels which are background are set to NaN to indicate that they are not part of the mesh. Each pixel is also assigned an albedo which we restrict to be between 0 and 1. For the experiments with Eve we use either a uniform albedo of 1 or a synthetically generated albedo based on Perlin noise. We define four virtual light sources which lie on the camera plane $z = 0$ and are positioned at 90^o intervals at a radius of $4\,\mathrm{cm}$. Each light source is defined by its direction and the non-linear light attenuation coefficient μ as described in section 3.3.1.

Using the above setup we performed the experiment as shown in figure 3.3. Here the central pixel was initialized with the ground truth depth value and the method described in section 3.5.1 was used together with the backward upwind scheme of section 3.5.2.

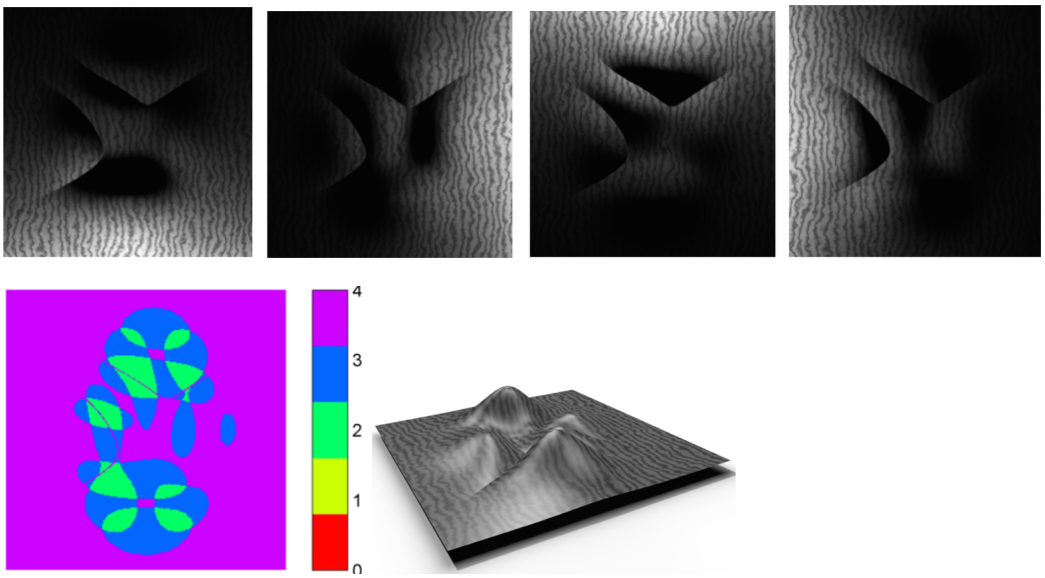


Figure 3.4: The input images with $\mu = 5$ are shown in the first row. The second row has (on the left) the shadow map and (on the right) the approximate 3D shape with recovered albedo textured on. $MSE = 3.75\text{E-}4 units^2$

AbsPeaks, with realistic shadows

Figure 3.4 shows the set of significantly overlapped shadows and light attenuations where $\mu = 5$ for light sources placed at a radial distance of 10 units. This demonstrates that our formulation enables photometric stereo even for shadowed regions and non-parallel-ray light sources.

Comparison to other methods

Let us compare our shape recovery approach to other methods which have not necessarily been designed to handle shadows. We therefore allow all irradiance values and retain the data in double precision floating point format without first converting to 8-bit grayscale. The point light sources are positioned 3 units from the focal point, which, as we have mentioned, is in contrast to the classic assumption of distant light sources. The reasoning behind this placement is to demonstrate the deformation of the shape recovered by other methods when the model of nearby light sources, is not taken into account. We compare our approach with [2] and [43]. The surface normals provided to these two methods are first computed by assuming distant parallel ray light sources where the light direction is computed by averaging between the light source position and an approximate distance from the object (as done in [108]). [2] requires the use of full boundary conditions whereas [43] assumes so called natural boundary conditions which requires no additional information. Figure 3.5 illustrates that the reconstruction by the other two methods becomes considerably worse when we add attenuation of $\mu = 1$, whereas our result remains consistent with an RMSE of 1.95E-2 units.

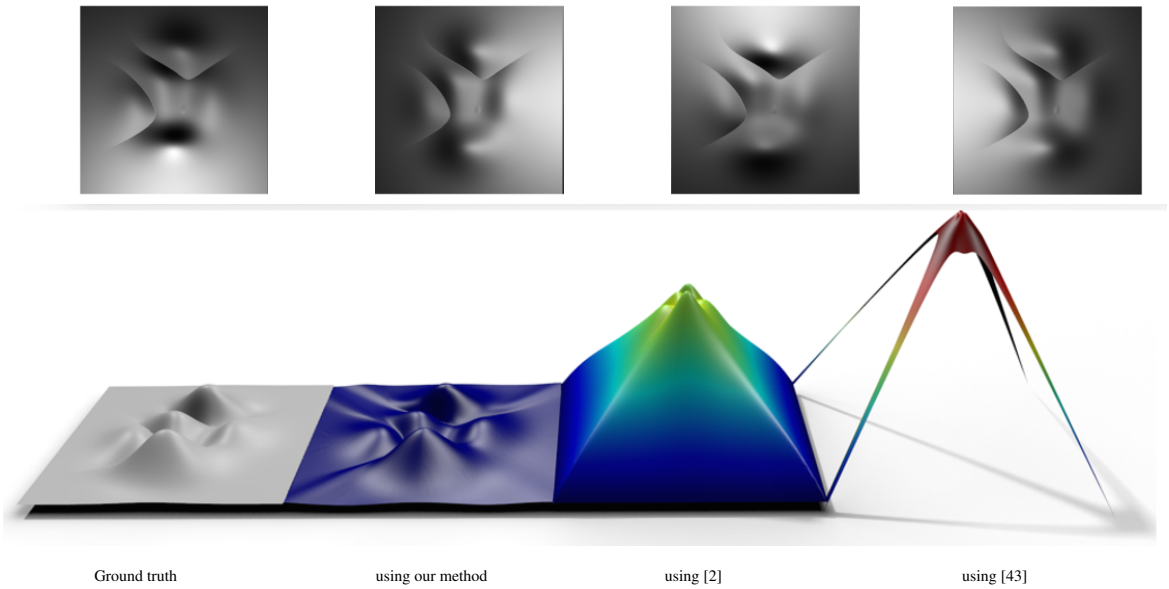


Figure 3.5: **First row** Images from AbsPeaks with $\mu = 1$ and lights positioned 3 units from the focal point. **Second row** From left to right: ground truth shape, reconstruction using our method $MSE = 3.82\text{E-}4 units^2$, the reconstruction using [2] $MSE = 5.19 units^2$, the reconstruction using [43] $MSE = 21.5 units^2$. All reconstructed shapes have their error maps textured on.

3.6.2 Real cases

In the last scenario we use a low cost endoscopic camera synchronized to four individually controlled low power white SMD light sources. The images were taken in a dark environment and a calibration image was acquired without any lights activated. This was then subtracted from all subsequent images to account for ambient lighting. Any pixels with a value less than 20 were marked as shadow. This is a very simple heuristic but useful for our setup of multiple light sources. The intrinsic camera parameters were found using the Bouguet calibration toolbox [12] and all images had the effect of lens distortion removed. The depth to a single point on the object was measured manually. A more practical version could use a small laser pointer calibrated to the camera to extract an initial seed depth but we have not implemented this in these experiments. The processed images for each object can be seen in figure 3.6 with a rendering of a novel view of the same object based on our reconstruction. It is interesting to note that our method was able to successfully reconstruct the thumb and the ridges of the thumb-prints are clearly visible in the 3D render. Furthermore, despite the specularity and non-uniform albedo present in the steps images, the reconstruction is unaffected and preserves the straight lines of the edges of the steps. The hand model is also interesting because it illustrates that our method can handle real world scenarios in which there is a level of noise in the captured images.

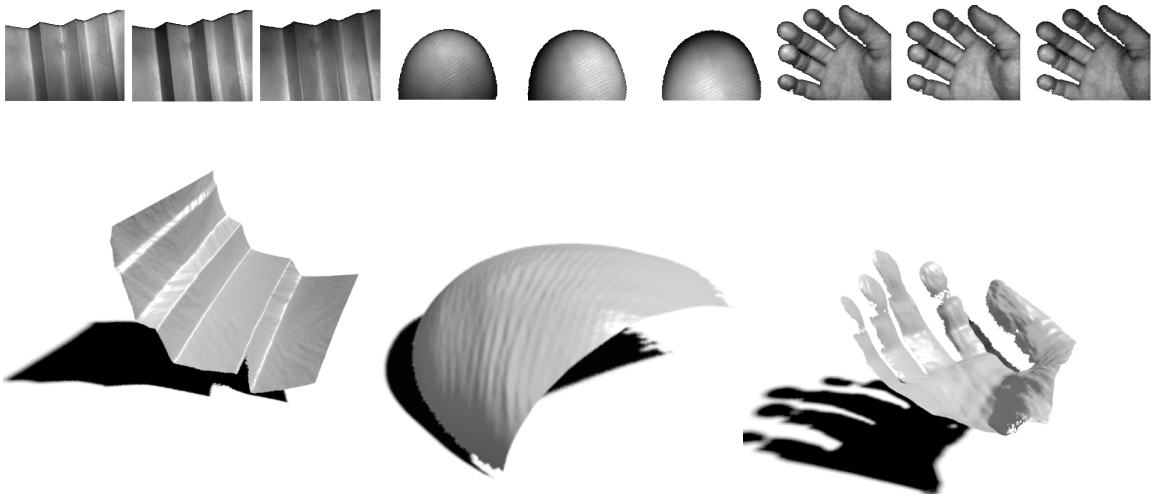


Figure 3.6: Novel views from reconstructions of real objects imaged with the experimental setup described in section 3.6.2. The captured images are shown on top of each reconstruction. The distance between the object and the scanner is: Model stairs $120\,\mathrm{mm}$, Thumb-tip $35\,\mathrm{mm}$, Hand $220\,\mathrm{mm}$

3.7 Conclusions

An efficient model for shape reconstruction from the EPPS$_n$ problem was proposed. The main goal of the proposed model is to define a new differential formulation based on a quasi-linear PDE, where the well-posedness holds even in the presence of images with missing parts. We have shown that PDEs provide a strong way of modeling near-field photometric stereo that can be used to approximate completely general lighting scenarios. The model we presented overcomes the limitations of more classical approaches since the lighting is modeled realistically as a nearby source with fully general illumination. As far as we can tell this currently represents state of the art in physical modeling for any PS method. We are currently investigating even more general models for non-linearities such as specular effects and general BRDF functions. Our current experiments demonstrate that the model gracefully deals with non-linear light attenuations as well as non-uniform surface albedo and missing data. Furthermore, the suggested method handles real world surfaces and produces quantitatively faithful surface reconstructions for nearby objects. The method is highly parallelizeable and future work will attempt an implementation on a kilo-core Graphics Processing Unit to demonstrate that real time endoscopic shape from PS is possible. Current ongoing work includes shrinking the scanning head and performing tracking so that reconstructed depth maps can be fused into a larger surface. Another issue is that of obtaining at least one known ground truth point (assuming a single connected surface). This has not been implemented but we are investigating using a laser dot or line calibrated to the camera.

Part II

GEOMETRIC DENOISING

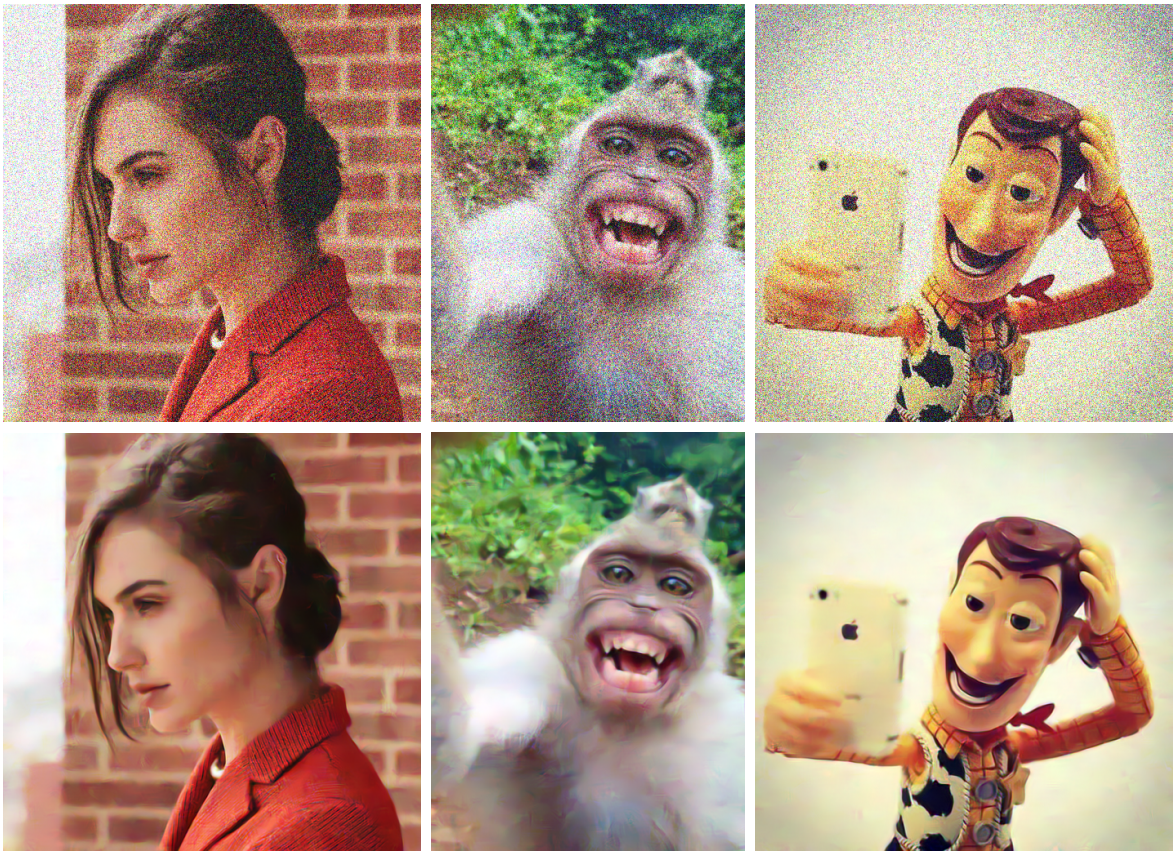
Chapter 4

The Patch-Beltrami Framework

In this chapter we discuss a framework which we have developed for reducing noise in images and an extension to meshes. We base our framework on the original Beltrami filter which treats images as two dimensional manifolds embedded in a joint features-space domain. This way, a color image is considered to be a two dimensional surface embedded in a hybrid spatial-spectral five dimensional $\{x, y, R, G, B\}$ space. Image selective smoothing, often referred to as a denoising filter, amounts to the process of area minimization of the image surface by mean curvature flow. We present our approach to modifying the Beltrami filter by treating local neighboring pixels as the feature-space or in other words a neighborhood aware local descriptor. A distance is defined by the amount of deformation a local patch undergoes while traversing its support in the spatial domain. The question we try to tackle is how to perform patch based denoising bith accurately and efficiently. We demonstrate the performance of the new Patch-Beltrami filter and provide useful implementation considerations for that allow for parameter tuning and efficient implementation on hand-held devices like smart phones. Furthermore we illustrate one possible way in which this method can be extended from the domain of regular grids to unstructured point clouds. Finally, we demonstrate that by treating the Patch-Beltrami operator as a collection of tunable convolution kernels and using natural image data together with a minimization objective we achieve state of the art image denoising while retaining the geometric flavour of the approach.

4.1 Introduction

Following the success of the Non-Local Means denoising method as introduced by Buades et al. in [17] much attention has been devoted to developing various types of patch based denoising techniques. A patch, in terms of an image, is generally considered to be a square region of pixels of fixed size centered at the coordinates of an image pixel. Peyrè in [110] studies patch based manifolds while a more specific analysis of a generalized patch based denoising framework is done by Tschumperlè and Brun in [137].



A B C

Figure 4.1: Examples of Deep Patch-Beltrami denoising for color images. **Top row:** shows images with AWGN at $\sigma = 50$. **Bottom row:** shows the denoised results. A) Gal Gadot, $PSNR = 28.63$dB B) Monkey pickpocket, $PSNR = 27.98$dB C) Woody selfie, $PSNR = 29.82$dB

They show that the NL means [17] and Bilateral [135] filters are isotropic versions of their patch based diffusion framework by choosing a specific patch size and metric. In much the same way Sochen *et al*let@tokeneonedotpresent the Beltrami framework and show in [128] how choices of different metrics can be used to produce filtering methods like the anisotropic diffusion process of Perona and Malik [109] as an example. Anisotropic diffusion was also shown by Barash in [9] to have a strong connection to the Bilateral filter through the adaptive smoothing filter and Elad in [44] demonstrated its connection to other classical filtering techniques. In [116] Maragos and Roussos, explore a generalization of the Beltrami flow using exponentially weighted patches. Initially we will use a similar formulation but instead setting the weights of each neighboring pixel to be one. In this context, the Beltrami framework provides a general and natural substrate for diffusion based image manipulation and naturally extends to higher dimensions. While diffusion methods seemed promising they were typically no match for the non-differential variations which were state of the art for a long time. In fact the most successful non-local patch based denoising method was considered to be the BM3D method of Dabov *et al*let@tokeneonedot [34]. There similar patches are first collected and then denoised together using collaborative Winer filtering and placed back to their original locations. This approach uses the inherrent structure and image statistics to very effectively reduce image noise. The method was so succesful that a number of papers [84, 24] posed the question of whether the field of denoising was perhaps saturating its ability to improve. Levin [84] posited that it seemed unlikely that any method would be able to surpass BM3D by more than a small fraction of a dB. The claim was that assuming a natural image prior the BM3D method was around $0.1\mathrm{dB}$ from the optimal upper bound and this was backed up by empirical evidence on a large corpus of natural image patches. Although the paper was well argued it was soon shown to be off the mark as a number of models such as [28, 122, 59] surpassed the BM3D method by $0.2\mathrm{dB}$ or more[1]. Of particular relevance to us is the method of Chen *et al*let@tokeneonedot [28] as it is most similar in character to our PDE learning based approach which we will discuss in Section 4.5.

The primary motivation for the framework we present in the coming sections is to provide a principled geometric approach to dealing with denoising in both regular domains such as images and irregular domains such point clouds. Part of this work has been previosuly published in a more condensed format [141, 142] and here we cover that work, reconsider elements of it and siginficantly extend the previous approach to produce results which are comparable to state of the art while taking advantage of deep neural architectures.

[1]Since the time at which this manuscript was prepared a number of newer models have appeared in the literature which surpass BM3D by even greater margins.

4.2 The Patch-Beltrami framework

The Beltrami filter [77, 128] models image denoising and smoothing as a process of mean curvature minimization over a high dimensional embedding of an input image. Here we describe the original filter and then how we extend it to treat image patches to form the basis of the Patch-Beltrami framework.

4.2.1 The Beltrami filter

We consider an image to be a $2D$ Riemanian manifold embedded into a $D = d + 2$ dimensional space. For the case of grayscale images $d = 1$ as there is only a single intensity value for every location on the $2D$ manifold. For color images this becomes $d = 3$ as color values are typically represented as RGB intensity tuples. Although in principle we could use another color space such as $YCbCr$ or Lab there is an advantage to using RGB as it more directly relates to image sensor values. We can thus write the map $X : \Sigma \to M$ where X is the mapping of the image manifold Σ into the embedding space feature manifold M. A grayscale embedding map can be written as

$$X(\sigma_1, \sigma_2) = (x(\sigma_1, \sigma_2), y(\sigma_1, \sigma_2), z(\sigma_1, \sigma_2)). \tag{4.1}$$

where $(\sigma_1, \sigma_2) \subset \Sigma$. By specifying that $x = \sigma_1$, $y = \sigma_2$ and $z = I$, then from (4.1) and with a slight abuse of notation we can write the map X as

$$X(x, y) = (x, y, I(x, y)). \tag{4.2}$$

Both Σ and M are Riemannian manifolds and hence are equipped with metric tensors G and H respectively which enable measurement of lengths over each manifold. We require the embedding to preserve the lengths as measured on each manifold to be the same. We can write this constraint explicitly on the arclength differential ds for both the original manifold and it's embedding as

$$ds^2 = \underbrace{(dx\ dy\ dI)\, H \begin{pmatrix} dx \\ dy \\ dI \end{pmatrix}}_{\text{measured in } M} = \underbrace{(dx\ dy)\, G \begin{pmatrix} dx \\ dy \end{pmatrix}}_{\text{measured in } \Sigma}. \tag{4.3}$$

We can equate these and write the result compactly using Einstein notation where repeated upper and lower indices are summed over

$$g_{uv} = \partial_\mu X^p \partial_\nu X^q h_{pq} \quad \mu, \nu = 1..2 \quad p, q = 1..3 \tag{4.4}$$

Here the meaning of $\partial_{\mu,\nu}$ is just the partial derivative with respect to x or y. For the simplest case we have a direct Euclidean embedding in (4.4) where $H = (h_{ij})$ is set as the identity. We use the chain rule $dI = \partial_x I dx + \partial_y I dy$ to determine that for (4.2) the induced metric tensor (g_{uv}) can be seen to be

$$G = \begin{pmatrix} 1 + \partial_x^2 I & \partial_x I \partial_y I \\ \partial_x I \partial_y I & 1 + \partial_y^2 I \end{pmatrix}. \tag{4.5}$$

The original Beltrami Framework proposed the novel use of the Polyakov action functional [128] in order to provide a measure on the manifold defined as

$$S[X^p, g_{\mu\nu}, h_{pq}] = \int d^m \sigma \sqrt{g} g^{\mu\nu} \partial_\mu X^p \partial_\nu X^q h_{pq}(\mathbf{X}). \tag{4.6}$$

This action is invariant under smooth coordinate transformations and, despite it's apparent complexity, turns out to be the area of the embedding surface. We refer the reader to [128] for more specific details and focus here on arriving at the computational aspects of the expression. For the Euclidean embedding in M described above, the total surface area of the grayscale manifold computed from the perspective of the coordinates in Σ simplifies to

$$S\left[X, G\right] = \iint \sqrt{g} dx dy = \iint \sqrt{1 + \partial_x^2 I + \partial_y^2 I} dx dy. \tag{4.7}$$

Here $g = \det(G)$. We can find the first variation of this functional with respect to I which results in the Euler-Lagrange relation given by

$$-\partial_x \left(\frac{\partial_x I}{\sqrt{g}} \right) - \partial_y \left(\frac{\partial_y I}{\sqrt{g}} \right) = 0. \tag{4.8}$$

We recall that S is reparameterization invariant and reorganize the terms of Equation (4.8) while multiplying both sides by $g^{-1/2}$ to yield

$$-\frac{1}{\sqrt{g}} \text{div} \left(\sqrt{g} G^{-1} \nabla I \right) = 0. \tag{4.9}$$

The left side of (4.9) indicates the rate at which the surface area will become smaller as a result of a change in I. Furthermore, this expression can be recognized as the second order differential operator of Beltrami (the extension of the Laplacian to manfiolds) and can be compactly written as $\Delta_g I = 0$. The key idea behind the Beltrami filter is the idea that this can be used to define an explicit geometric flow called the Beltrami flow on the manifold coordinate I

$$I_t = \Delta_g I. \tag{4.10}$$

This can be seen to be a manifold heat diffusion process that creates a generalized manifold scale space. We are be able to traverse this scale space in an explicit Euler forward step computational framework and in its most basic form it produces an effective and intuitive technique for denoising and smoothing grayscale images when using the metric in (4.5). We now describe our key extension of the Beltrami flow that takes advantage of local discrete patch information.

4.2.2 Patch space metric

Over an image domain a patch is a region defined around a given point. So for a discretized image array of pixels we define a patch to be a fixed size square neighbourhood centered at a given pixel. In a sense this means that every pixel is described by the intensity or color values of its neighbors (in a fixed order) instead of a single grayscale value. In fact this "descriptor" perspective is what justifies the use of discrete pixel values within a continuous variational framework. Therefore, using the same idea as before we consider our image to be a $2D$ Riemanian manifold mapped into a much higher dimensional space via $P : \Sigma \to \mathbb{R}^{d(2r+1)^2+2}$ in the form

$$P(x, y) = (x, y, \{I_k(x + i, y + j)\}), \tag{4.11}$$

where $i, j = -r...r$ and $k = 1...d$. Here $r =\in \mathbb{N}$ is known as the window radius, and as noted before d is the number of channels in the image. For example, a single channel image where $d = 1$ and $r = 5$ produces patches of size 11×11 centered about each pixel in the image I. We can see that the above definition reduces to the grayscale embedding (4.2) for $r = 0$ and $d = 1$ as described in the previous section. For $r > 0$ we can see that we are using spatially shifted versions of the original image as embedding coordinates which is essentially a pixel descriptor defined by it's neighbors. From here on we will denote $I_k(x + i, y + j)$ as I_{kij}. Note that we shall write I_{k00} as I_k which is simply the k^{th} color channel of the image I. Given this new patch based setup we will derive a more general induced metric tensor G for the patch embedding based on a similar approach as the Beltrami filter described in the previous section. For that goal we again consider the arclength measurement ds in an Euclidean embedding space with even scaling but this time using the patch values I_{kij} which yields

$$ds^2 = dx^2 + dy^2 + \sum_{kij} (dI_{kij})^2 . \tag{4.12}$$

In reality, the coordinates x and y do not possess the same physical measure as the intensity values of the image so we need to introduce a scaling factor β^2 into the embedding tensor h_{pq} so that

$$h_{pq} = \begin{cases} \delta_{pq} & p,q = 1...2 \\ \beta^2\delta_{pq} & p,q = 3...dr^2+2 \end{cases}, \tag{4.13}$$

where δ_{pq} is the Kronecker delta. Following the same procedure as before we can use the chain rule $dI_{kij} = \partial_x I_{kij}dx + \partial_y I_{kij}dy$ to expand (4.12) and pullback the patch-space metric from the embedding. The new induced metric tensor for the $2D$ image manifold embedded into patch-space is then given by

$$G = \begin{pmatrix} 1+\beta^2\sum_{kij}\partial_x^2 I_{kij} & \beta^2\sum_{kij}\partial_x I_{kij}\partial_y I_{kij} \\ \beta^2\sum_{kij}\partial_x I_{kij}\partial_y I_{kij} & 1+\beta^2\sum_{kij}\partial_y^2 I_{kij} \end{pmatrix}. \tag{4.14}$$

This metric combined with (4.10) gives the Patch-space Beltrami flow as

$$(I_k)_t = \Delta_g I_k = \frac{1}{\sqrt{g}}\text{div}\left(\sqrt{g}G^{-1}\nabla I_k\right). \tag{4.15}$$

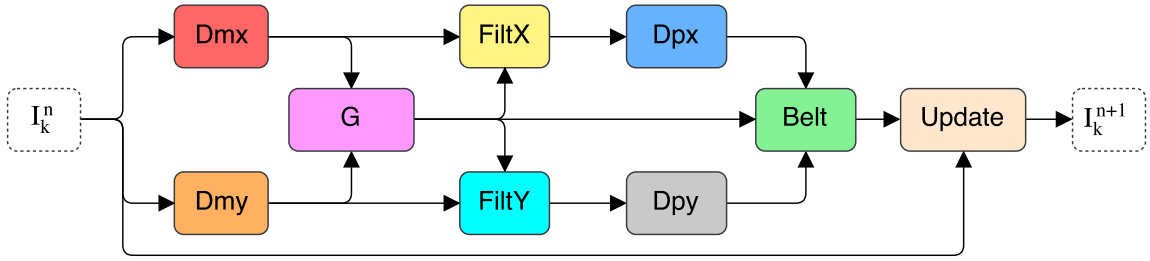
We only define the flow for the original color channels I_k although one can include the other patch-space channels at the cost of increasing the order of computational complexity although we did not find this step to be necessary.

4.3 Denoising over regular domains

The flow produced by (4.15) is simply the original Beltrami filter for $r = 0$. However when the patch-space window radius is larger than 0 the flow provides more local context and results in a far more effective image denoising flow than the regular Beltrami flow in (4.10). We use (4.15) to progress through the scale space on single or multi-channel image manifolds embedded into patch-space. For image denoising the typical measurement of denoising quality is the Peak Signal to Noise Ratio (PSNR) which is defined as

$$\text{PSNR}_I(I_{est}) = 10\log_{10}\left(\frac{255^2}{\text{Avg}\left[(I-I_{est})^2\right]}\right), \tag{4.16}$$

where Avg $[\cdot]$ computes the average and I_{est} is the estimation of the noise-free I. PSNR produces results in dB and although it is not directly related to how people perceive the quality of denoised image it is a good proxy and in this work we aim to maximize it either indirectly or indirectly.



$$I_k^{n+1} = I_k^n + \delta t \left(\frac{1}{\sqrt{g}} D_x^+ \left(\frac{1}{\sqrt{g}} \left(g_{22} D_x^- I_k^n - g_{12} D_y^- I_k^n \right) \right) + \frac{1}{\sqrt{g}} D_y^+ \left(\frac{1}{\sqrt{g}} \left(g_{11} D_y^- I_k^n - g_{21} D_x^- I_k^n \right) \right) \right) \tag{4.17}$$

Figure 4.2: This is the computation graph for Equation 4.17. The equation is shown with each functional component highlighted to show where it is computed in the graph. Best viewed in color.

4.3.1 Numerical scheme

In order to use the geometric flow defined in Equation (4.15) we assume that an input image is sampled as a set of pixels over a regular flat grid with unit spacing between adjacent pixels. We first write out Equation (4.15) explicitly as

$$\begin{aligned} (I_k)_t &= \tfrac{1}{\sqrt{g}}\text{div}\left(\tfrac{1}{\sqrt{g}} \begin{bmatrix} g_{22} & -g_{12} \\ -g_{21} & g_{11} \end{bmatrix} \begin{bmatrix} \partial_x I_k \\ \partial_y I_k \end{bmatrix} \right) \\ &= \tfrac{1}{\sqrt{g}}\partial_x \left(\tfrac{1}{\sqrt{g}} \left(g_{22}\partial_x I_k - g_{12}\partial_y I_k \right) \right) + \\ &\quad \tfrac{1}{\sqrt{g}}\partial_y \left(\tfrac{1}{\sqrt{g}} \left(g_{11}\partial_y I_k - g_{21}\partial_x I_k \right) \right). \end{aligned} \tag{4.18}$$

We now build our numerical scheme using a forward Euler method that uses the following forward and backward discrete difference operators:

$$\begin{aligned} D_x^+ f &= f(x+1,y) - f(x,y), \\ D_y^+ f &= f(x,y+1) - f(x,y), \\ D_x^- f &= f(x,y) - f(x-1,y), \\ D_y^- f &= f(x,y) - f(x,y-1). \end{aligned} \tag{4.19}$$

For the sake of simplicity these spatial operators assume unit spacing and this will also be useful even when the input is non-uniform such as on point clouds which we describe in Section 4.4. We further impose Neumann boundary conditions with the derivative going to 0 for points outside the domain. The Patch-Beltrami flow in (4.18) can be approximated using an explicit first-order Euler step and does not require solving a linear system which, as we shall discuss, makes it amenable to efficient parallization.

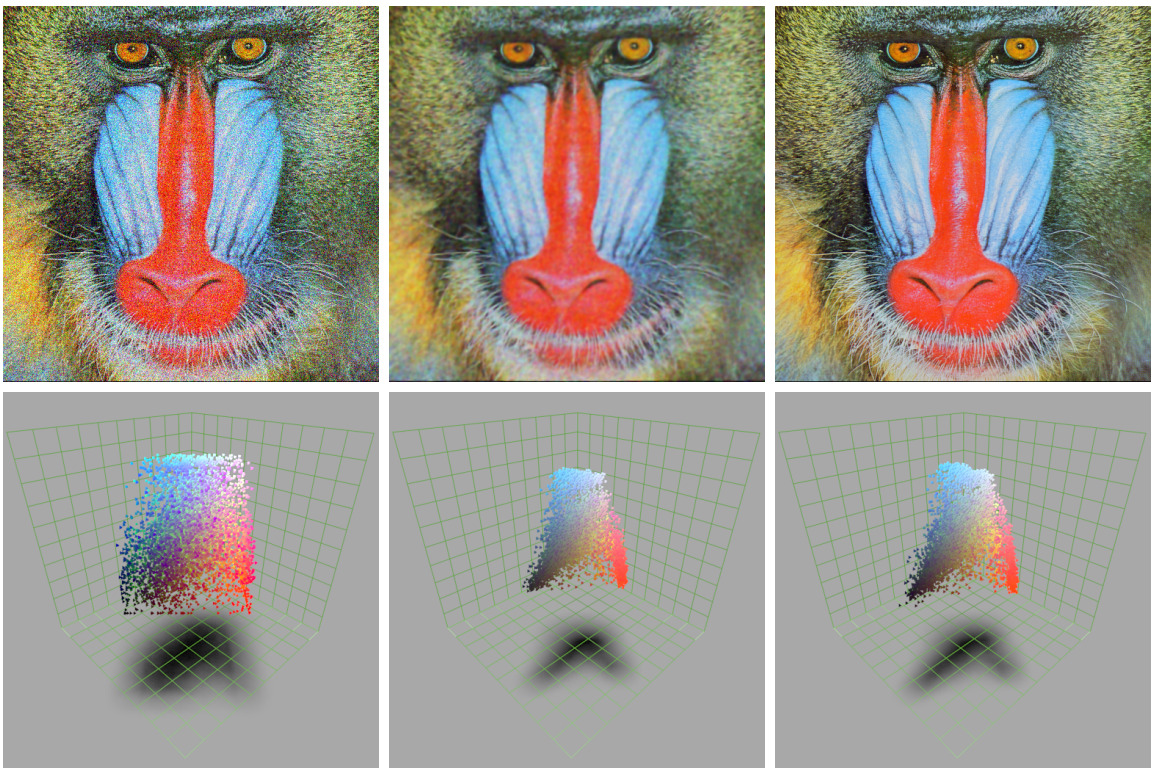


Figure 4.3: Visualization of effect of denoising on the compactness of the RGB triples. **Left column:** Noisy image of Baboon with $\sigma = 40$ and the visualization of it's RGB values in on a 3D axis. **Middle column:** Denoised Baboon image and corresponding RGB cloud. **Right column:** Original noise free image and RGB cloud. It is useful to note the similarity in structure and density of the latter two RGB clouds despite the strong noise in the noisy cloud. This hints at the ideas that colors in an image can be viewed as sitting on a latent manifold and provides further intuition behind the mechanism of the Patch-Beltrami denoising process.

The discrete update recurrence is given by

$$I_k^{n+1} = I_k^n + \delta t \begin{pmatrix} \frac{1}{\sqrt{g}} D_x^+ \left(\frac{1}{\sqrt{g}} \left(g_{22} D_x^- I_k^n - g_{12} D_y^- I_k^n \right) \right) + \\ \frac{1}{\sqrt{g}} D_y^+ \left(\frac{1}{\sqrt{g}} \left(g_{11} D_y^- I_k^n - g_{21} D_x^- I_k^n \right) \right) \end{pmatrix}, \tag{4.20}$$

where the n superscript indicates the iteration number. We set the initial I^0 to be the noisy input image. Using the discrete differential operators the metric tensor components can be approximated as

$$\begin{array}{rcl} g_{11} & = & 1 + \beta^2 \sum_{kij} (D_x^- I_{kij}^n)^2 \\ g_{12/21} & = & \beta^2 \sum_{kij} (D_x^- I_{kij}^n D_y^- I_{kij}^n) \\ g_{22} & = & 1 + \beta^2 \sum_{kij} (D_y^- I_{kij}^n)^2 \\ \frac{1}{\sqrt{g}} & = & (g_{11} g_{22} - g_{12}^2)^{-0.5} \end{array}. \tag{4.21}$$

4.3.2 Parameter tuning

The numerical scheme in (4.20) and (4.21) has a number of parameters which need to be chosen: window radius r, number of iterations N, time step δt and space-color scaling β. We wish to select the values of these parameters that will produce the best $PSNR$ values for images which we assume to have been polluted with additive Gaussian white noise and known standard deviation σ. The original Beltrami filter chooses one common value of β for all images and noise profiles and then traverses a scale space by applying the Beltrami operator iteratively which performs diffusion of image energy over the embedding manifold over time. However this approach is limited in that it does not explicitly aim to improve the $PSNR$ nor does it take into account the runtime complexity. In other words it is not clear how to select β and N. In the Patch-Beltrami framework we take to opposite approach and search for the optimal β accross the different framework parameters and noise profiles while fixing the number of iterations. Doing so stretches the space-color ratio and alters the distances in the embedding space while at the same time maintaining a fixed runtime. In order to determine the parameters which yield optimal $PSNR$ denoising results we chose a small set of representative images and ran a simple grid search over the set of possible parameters limited to ranges which we identified manually. This is performed separately for color and grayscale inputs as we found that the behavior was dependent on the number of image channels. With all parameters held constant except for β we found empirically that the $PSNR$ always has a unique global maximum. Interestingly, the value of β which yields this optimum is almost identical irrespective of the content of the image and the actual image resolution provided that they that have been polluted with the same σ. This behavior is illustrated in Figure 4.4.

We also found that increasing the window radius above $r = 2$ has no benefit for grayscale images and that there is only marginal improvement when increasing the number of iterations above $N = 50$

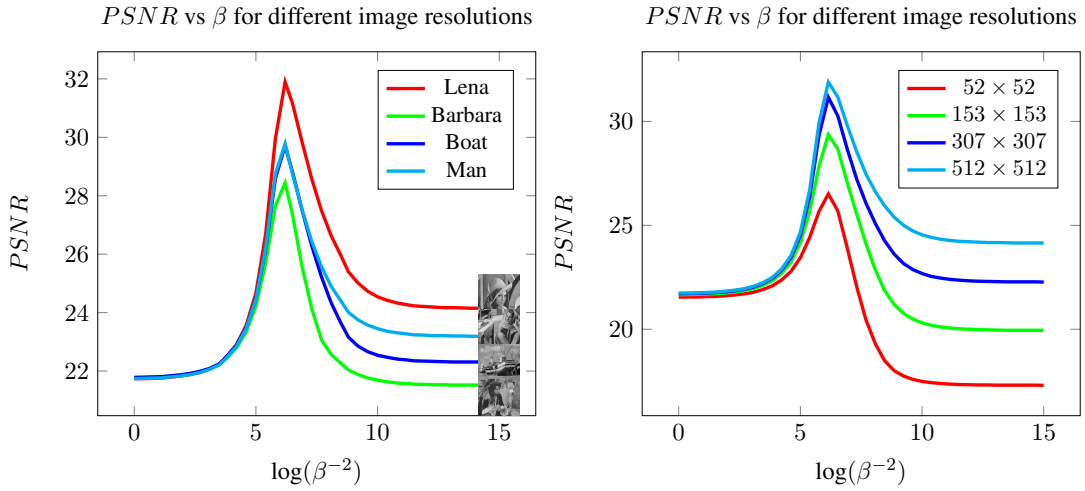


Figure 4.4: Behavior of $PSNR$ relative to β, image resolution and image content. **Left:** This shows the behavior of the $PSNR$ as a function of β for different images of the same resolution. **Right:** This shows the behavior of the $PSNR$ for different image resolutions. These two figures highlight the fact that the optimal value of β is independent of image content and size. The optimal $PSNR$ value can also be seen to be higher for larger images and lower for more visually complex images.

as demonstrated in Table 4.2. Freezing these two parameters and then modifying both β and δt yields an optimal $PSNR$ which is relatively insensitive to δt around the best value of β. This is displayed in Figure 4.5.

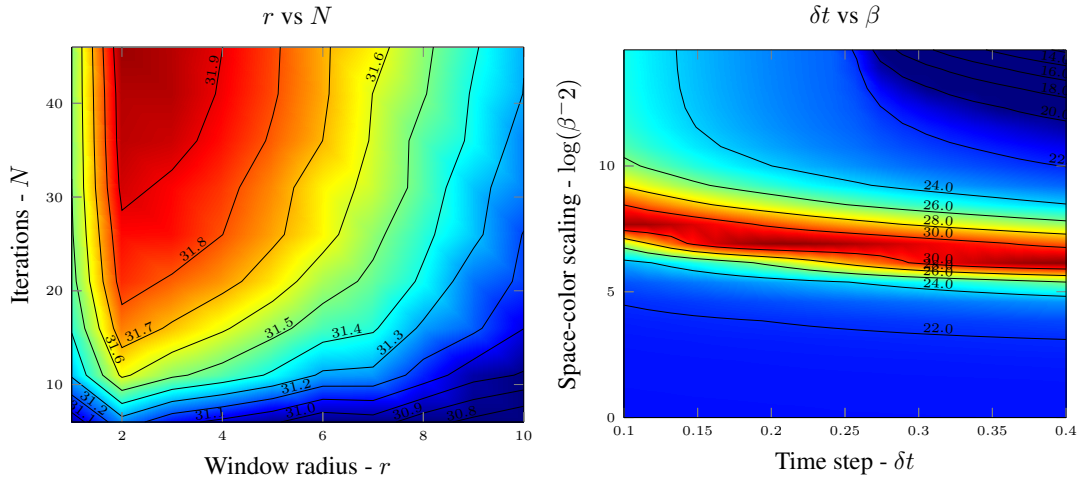


Figure 4.5: Contour maps higlighting denoising behavior of different Patch-Beltrami parameters. High values of $PSNR$ are shown in red while lower values in blue. **Left:** Iteration numbers and window radius both have a strong impact on the $PSNR$. However a radius of $r = 2$ or $r = 3$ is sufficient and more than 50 iterations tends to show no noticeable improvement. **Right:** The specific choice of δt is not particularly important as can be seen in the plot. However the choice of β has a strong effect on the denoising capabilities of the method.

Ultimately we would like a way to directly choose the optimal parameters for a given σ without performing an optimal search every time however the analytical relationship between β and σ is non-trivial. Instead we experimentally determined that the optimal value of β is approximately related to σ by a linear

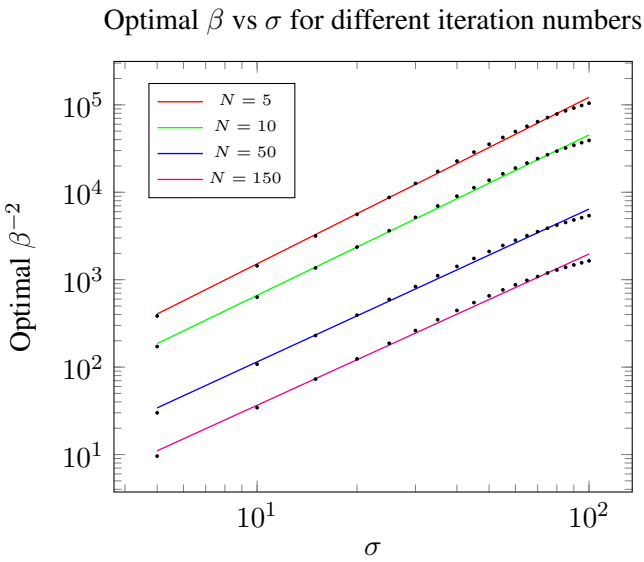


Figure 4.6: This graph shows that there exists an approximately linear relationship in log-space between $\beta^{-}2$ and σ for achieving optimal $PSNR$ values for a new image. The plot shows the relationship for different numbers of iterations.

model in log space for values of σ up to 100^2 by the simple relation

$$\log(\beta^{-2}) = a\log(\sigma) + b. \tag{4.22}$$

Figure 4.6 shows this relationship graphically for different iteration values N. To obtain this graph we ran a grid search to find good β values and then used those to initialize a Nelder-Mead downhill simplex search to find the optimal β for every σ for a given iteration count N. It is then straightforward to find the parameters which fit the model in (4.22). As shown in Figure 4.4 the global $PSNR$ maximum is obtained for almost the same β for different images corrupted by additive zero mean Gaussian noise with the same σ. This fact is critical and illustrates that a and b obtained from the log-linear model only need to be calculated once for a predetermined window size, color type and iteration count. They can then be used to prescribe a nearly optimal β for any given σ for any image. Alternatively, a more complicated model or a densely populated look-up table can be generated to relate the two for even greater accuracy.

A naive single threaded implementation of the image denoising framework has time complexity $O(NW^2R^2)$ where N is the number of iterations, W is the width of an image (assuming it is square) and $R = 2r + 1$ for a patch radius r. However, when implementing the Patch-Beltrami flow we can take advantage of the fact that the summations in (4.21) are over square regions and can be seen as convolutions with unitary weights over their input fields. We take advantage of this property and compute a summed area table I_Σ otherwise known as an integral image. Doing so requires W^2 sum operations and memory and thereafter the sum of any patch from (4.21) can be calulcated using $sum(x, y) = I_\Sigma(x-r, y-r)+I_\Sigma(x+r, y+r)-I_\Sigma(x+r, y-r)-I_\Sigma(x-r, y+r)$. This eliminates the r^2 component and

[2]Pixel intensity values range from 0 to 255.

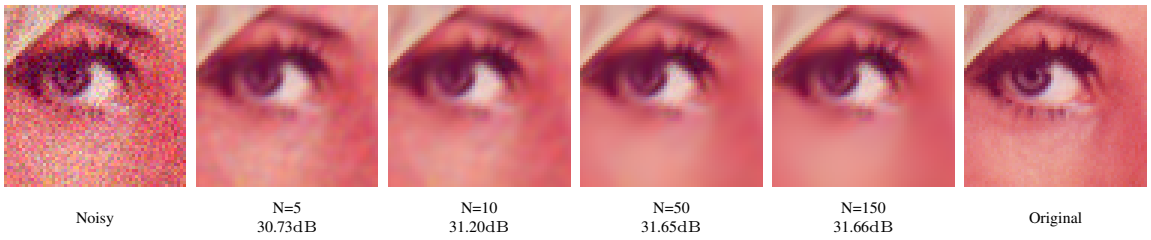


Figure 4.7: From left to right a) Noisy image at $\sigma = 20$ b) Denoised with 10 iterations, $PSNR = 29.21$ c) Denoised with 150 iterations $PSNR = 29.35$ d) Original image

yields running time complexity of $O(NW^2)$. While this approach is effective for improving computational efficiency in a serial pipeline it can in fact be replaced by a parallel implementation because all filter operations are per pixel and use information obtained from a previous iteration. We can therefore take advantage of fast parallel hardware available today in desktop and mobile GPUs which are specifically optimized to perform highly parallel computations. Surprizingly, implementing integral images on parallel hardware requires considerable engineering effort. This is even more true for mobile GPUs such as that on the iPhone 5S and 6S. It would appear that doing regular convolutions is a far better implementation option. Once again, we use the unitary weights to recognize that our convolution kernel is separable and can therefore be performed in a horizontal and vertical pass for each pixel. This reduces the per pixel time complexity from $O(NR^2)$ to $O(NR)$ which is significant for patch radiuses of $r > 1$.

Depending on an application's speed requirements, N can be further reduced with a gradual decrease in output quality as shown in Figure 4.7, where it is seen that after $N = 10$ there is virtually no visual improvement. The denoising properties of the Patch-Beltrami flow are indeed qualitativly reasonable for a small number of iterations, however higher quality visual results require more iterations and therefore longer computation times. We found that grayscale images are best denoised by $N = 50$ iterations and $r = 2$, whereas color images require only $N = 10$ iterations at a window size of $r = 3$. Running times for different parameter settings and computational environments are shown in Table 4.1. Our implementation uses a modified version of the powerful GPUImage library [81] for OSX and although this effectively uses the GPU our code is not optimized. Despite this, we can achieve close to real time denoising on 256×256 color images on an iPhone 6S. We show a timing comparison table in Table 4.1. Using the optimally chosen parameters, the denoising process can now be used automatically with the only input parameter being σ as is the norm for denoising images. This allows a visual feedback mechanism for a user who may wish to denoise or smooth her image. Together these properties indicates that the filter could be used in a real denoising environment. Furthermore, directly implementing the filter in hardware such as FPGA is likely to produce a faster and more energy efficient result although we have not explored that avenue here.

Table 4.1: Comparison of denoising times for different computational platforms and iterations. The MacAir and iPhone implementation use GPUImage while the M6700 CPU and GPU implementations use Matlab array and gpuArray implementations with little optimization.

		256×256				512×512				1080×1920			
		M6700 CPU	M6700 GPU	Mac Air GPU	iPhone 6S GPU	M6700 CPU	M6700 GPU	Mac Air GPU	iPhone 6S GPU	M6700 CPU	M6700 GPU	Mac Air GPU	iPhone 6S GPU
$N = 5$	1×1	31.6	17.5	5.2	11.1	85.2	34.0	15.3	18.9	744	207	110	123
	7×7	30.6	16.4	6.3	15.3	100	40.0	26.5	47.9	863	243	235	352
$N =$ 10	1×1	52.7	30.0	10.5	21.5	179	60.7	32.2	37.3	1435	407	257	248
	7×7	63.4	29.4	14.1	29.3	225	68.4	59.3	95	1787	471	579	709
$N =$ 50	1×1	238	104	49	103	860	276	175	186	8554	2074	1401	1266
	7×7	288	119	75.4	141	1133	352	316	474	9154	2363	3445	3661

4.3.3 Experimental results

Here we describe the results of applying the described denoising framework to a number of images that have been polluted with additive, zero mean, Gaussian noise. We use some represenative images to compare our approach numerically to the original Beltrami flow filter as well as the well known Non-Local means and BM3D [34] patch based denoising filters. For each image we use the known value of σ to prescribe the optimized parameters. Table 4.3 reveals that the optimized Patch-Beltrami filter significantly outperforms the original Beltrami flow. However, despite this improvement the method is not competitive with BM3D and other recent methods and in Section 4.5 we will do a more thorough quantitave analysis and show our Deep Patch-Beltrami extension significantly outperforms BM3D and compares favourably to the state of the art.

Table 4.2: $PSNR$ results from denoising of the Cameraman image corrupted with AGWN for $\sigma = 20$ using optimized β. Values are in dB.

Window$\rightarrow$ Iteration$\downarrow$	0	1	2	3	4
10	28.04	29.11	**29.21**	29.09	28.96
50	27.58	29.04	**29.37**	29.27	29.15
100	27.35	28.94	**29.36**	29.28	29.16
150	27.22	28.88	**29.35**	29.28	29.16

Table 4.3: $PSNR$ comparison between regular Beltrami and Patch-Beltrami filtering for some standard grayscale images. All values are in dB.

	Cameraman			Lena			Barbara			House		
Noise $\sigma \rightarrow$	5	25	50	5	25	50	5	25	50	5	25	50
Belt	37.0	26.7	22.8	37.0	28.4	25.3	35.9	24.8	22.2	37.1	28.4	24.7
Belt-P	**37.5**	**28.2**	**24.8**	**38.2**	**31.2**	**27.5**	**37.0**	**27.6**	**24.0**	**38.3**	**31.0**	**27.0**

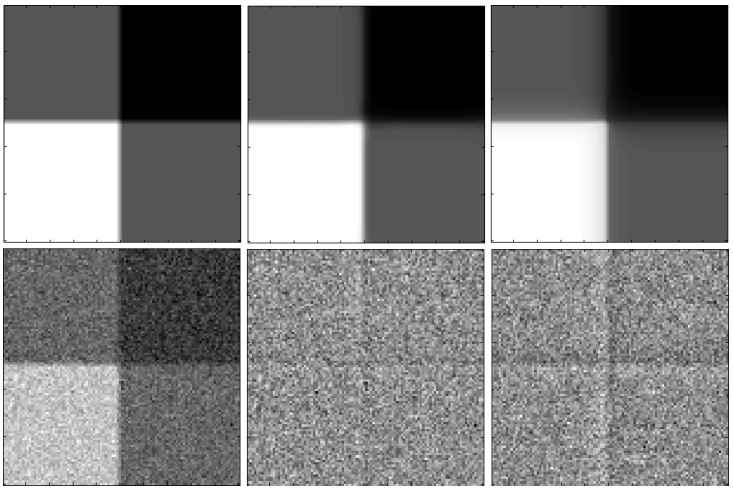


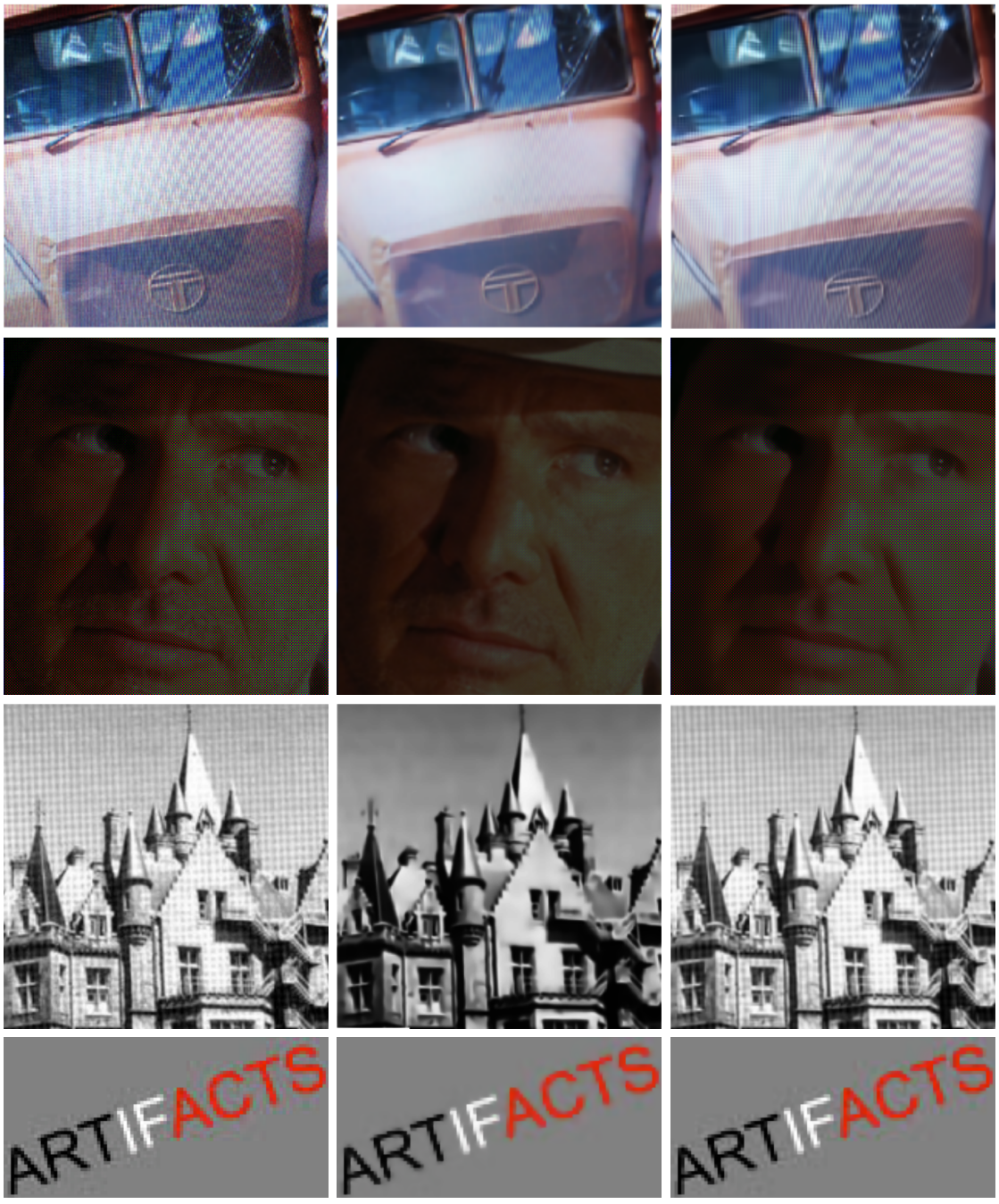
Figure 4.8: Method noise. **From top to bottom, left to right** a) Original image b) Noisy image with $\sigma = 20$ c) Patch-Beltrami denoising. 150 iterations, $w = 2$ d) Method noise for Patch-Beltrami denoising e) Regular Beltrami denoising f) Method noise for regular Beltrami denoising.

4.3.4 Residual noise

One way to compare the effectiveness of a denoising process is by visually evaluating the residual as noise as introduced by Baudes et al. in [17]. Here, we look at the differences between the estimated output image and the noisy input image. Ideally, the resulting difference image should also appear as Gaussian white noise. Figure 4.8 shows that patch based Beltrami flow produces significantly less structure in the difference image compared to the original single pixel based version.

4.3.5 Non-Gaussian denoising

The Patch-Beltrami filter has additional desirable properties which are not inherrently designed but instead emergent characteristics of the approach. The process tends to align colors along boundaries which can be used to improve aliasing artifacts of jagged edges which typically happen when a camera photographs an active screen. Another fundamental characteristic of the method is the traversal of a scale space which flattens out smooth, weakly textured objects. We have determined that this property even seems to be invariant to reduced image and can also handle mild compression artifacts despite no direct modification or specific training to do so. Examples of the effects of applying the Patch-Beltrami filter on different types of examples is shown in Figure 4.9. It is interesting to note that BM3D copes very poorly with these two situations probably because it is optimized for Gaussian noise removal alone.



Original Patch-Beltrami BM3D

Figure 4.9: Removal of aliasing and block textures. From left to right, top to bottom a) Photograph of truck with aliasing. b) Patch-Beltrami denoising, $\sigma = 20$ c) CBM3D denoising, $\sigma = 20$ d) Photograph of castle with weak block textures e) Patch-Beltrami denoising, $\sigma = 20$ f) BM3D denoising, $\sigma = 20$

Range images

The Patch-Beltrami filter is not restricted to only grayscale or color images. In fact it is indeed well suited for dealing with the problem of depth map denoising. For the case of a single height field produced by a depth scanning device such as a RealSense 3D camera we can apply the denoising procedure directly to the data. In this case the depth denoising takes only a single depth channel as input and the results can be seen applied to real data as shown in Figure 4.10. It is interesting to note that the data from a real sensor does not in fact represent a true Gaussian noise distribution and so when comparing to BM3D from a qualitative perspective our approach provides a more natural denoising effect. Furthermore, at the time of performing this experiment the available implementation of BM3D was not able to handle missing values which results in corruption of the denoised result around edges. Many structured light scanners tend to inject artifacts along the boundaries of the projected light patterns of the scanner. These artifacts can be observed as horizontal wrinkles in Figure 4.10 and are preserved by BM3D whereas our method removes the artifacts while leaving the main components of the scan intact.

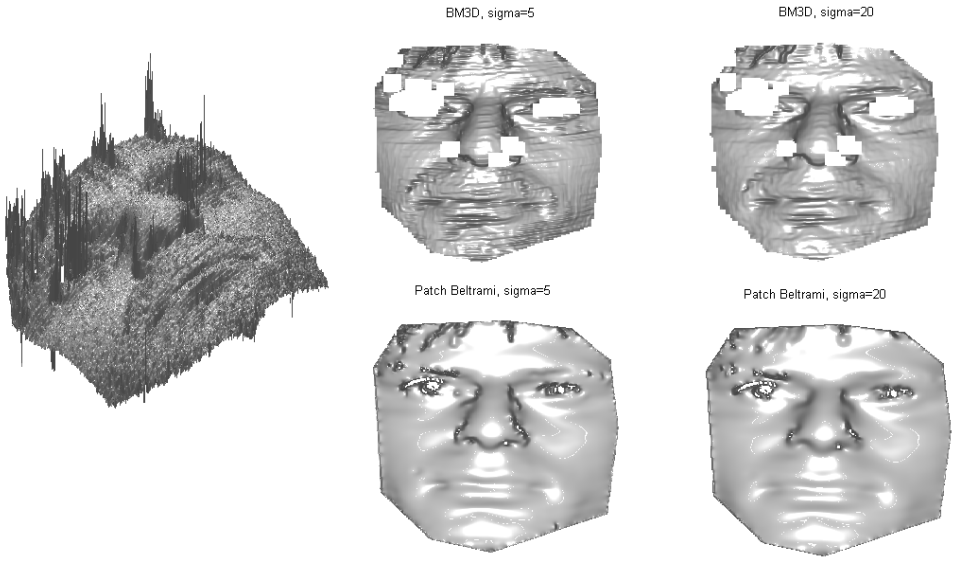


Figure 4.10: **Far left:** Original noisy scan. **Top row:** BM3D filtering for $\sigma = 5$ and $\sigma = 20$. **Bottom row:** Patch-Beltrami filtering for $\sigma = 5$ and $\sigma = 20$.

Considering the importance of facial scanning we also performed an experiment to determine whether a considerable amount of noise would destroy the underlying facial features. Figure 4.11 illustrates this experiment. Doing an extensive validation of this behavior remains as future work but it appears that the Patch-Beltrami approach preserves the key features.

It is also interesting to note that while we have only attempted denoising of depth images using a single channel depth input, there is in fact much recent work [103] to suggest that depth images can be jointly improved by also using the chromatic RGB channels as an additional signal for smoothness. A fruitful future direction to pursue is verifying that a combined RGB-D input could be used to provide additional denoising benefit for the case of depth data.

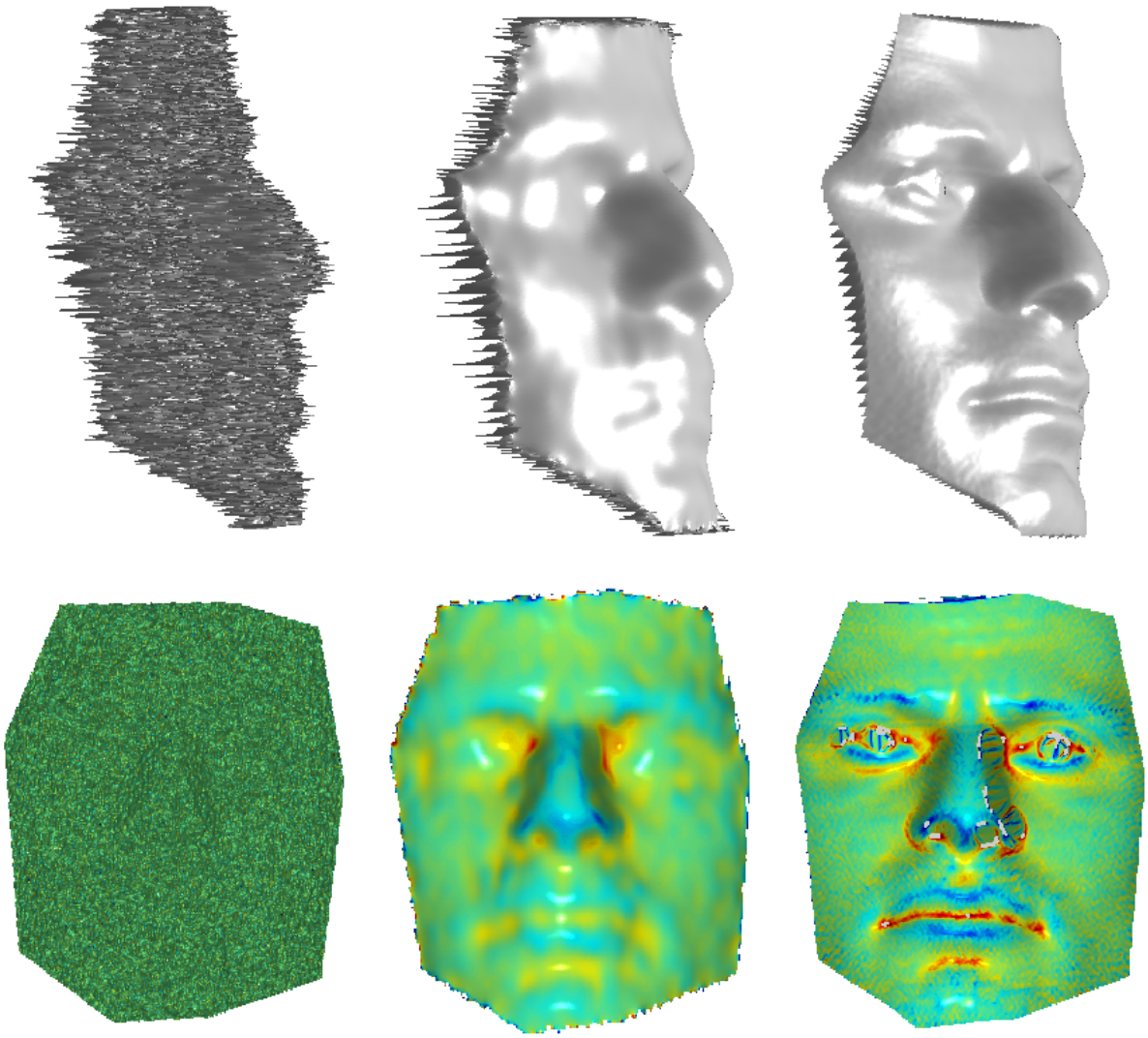


Figure 4.11: **Top row:** Side render of scanned face with significant artificial noise, our denoised result, original face. **Bottom row:** Respective mean curvature of noisy, denoised and original face. It appears that despite the disruptive noise, the denoising process retains the main features present in the face as the curvature coloring illustrates.

4.3.6 Discussion

We have shown that the extension of the original Beltrami filter with a more general metric produces significantly better results than the original Beltrami filter. The number of iterations required for denoising color images is $K \backsimeq 10$ resulting in a relatively fast algorithm of time complexity $O(KN^2)$ permitting an efficient implementation on a modern smartphone. We have also experimentally determined the relationship between the image intensities and their coordinates as posed in [128]. The proposed method produces $PSNR$ values close to state of the art techniques such as BM3D [34]. In addition to Gaussian denoising, the process accurately removes weak textures and aliasing while preserving the fine structure of the edges in images that other methods are not capable of dealing with.

While this work is focused exclusively on denoising it would seem reasonable to further study other applications of the operator in patch-space such as inverse diffusion and other processes which control the

eigen-values of the local diffusion operator. Many of these techniques have already been developed for the original Beltrami flow such as the FAB diffusion method as described by Gilboa et al. [129] and the short time Beltrami kernel as described by Spira et al. in [130] where it is approximated by finding local geodesic distances on the manifold via the fast marching method and the local metric tensor. The analysis and implementation of these same procedures would be fruitful directions for patch-space based flows.

Additional benefits of the Patch-Beltrami approach can be seen when applied to depth maps with noise models which are not perfectly Gaussian. Indeed the noise model for range scanners is mainly observed to be offset in the direction of the capturing device's focal point. These issues present a hurdle for classical image denoising algorithms due to the fact that they are mostly tuned for optimal removal of additive white noise. And while depth maps can be considered a stepping stone to dull 3D geometry, extending our approach to point clouds requires a somewhat different treatment. In the following section we will discuss a straightforward adaptation of the method to operation on unstructured 3D data.

4.4 Extension to point clouds

Denoising of point clouds, or point set fairing as referred to in the computer graphics literature, is an important post-processing step performed on potentially noisy data obtained from a 3D scanner. Given the drastic increase in the availability of commodity depth cameras the research being carried out in this field is becoming increasingly important. There are several different solutions to the problem of surface denoising depending on the kind of data. If the scanned data is in the form of a depth image, then many of the methods developed over the years by the image processing community for the purpose of denoising of regular images [148][109][135][6][35] are immediately applicable and require no special treatment apart from possibly scaling the height values. In a similar manner many of the same algorithms have been reformulated for use on surface data that is in the form of triangulated meshes and unstructured point sets. Yoshizawa [150] adapted the non-local means algorithm [6] to operate on meshes. Similarly Fleishman [50] showed how the bilateral filter [148][135] could be applied to meshes. For point sets the method of Moving Least Squares (MLS, [85]) is often used to perform smoothing on scanned data. More recently Rosman *et al*let@tokeneonedot [115] developed a patch-collaborative method for point set denoising based on the state of the art BM3D [35] algorithm. More methods for surface fairing are based on diffusion processes. For triangulated meshes, a common diffusion operator is the discrete Laplace-Beltrami cotangent weights formulation of [111], used to perform mean-curvature flow of the surface, including extensions to implicit numerical schemes [39]. For point sets, an anisotropic smoothing process was recently suggested by Lange and Polthier [80]. The method relies on a novel formulation of the underlying surface's Weingarten map which they developed specifically for point clouds. Other efforts [116],[141] have shown that denoising of images or depth scans can significantly benefit from operating

on so called patch-space within the image. We now show how the Patch-Beltrami flow described in the previous sections operating in patch-space can be used as an anistropic diffusion process that denoises noisy point cloud data. We do this by

- adapting the application of the Patch-Beltrami method onto unstructured point clouds,
- adopting a variation of the definition of patches on point cloud data and its use in the context of patch-space Beltrami filtering.
- investigating the role of normal estimation, and the choice of robust normal estimation [49] as opposed to PCA.

Section 4.4.1 describes the implementation of patch-space Beltrami filtering in point clouds, and details some of the aspects involved therein. Then in Section 4.4.6 we demonstrate the results of our algorithm on both real and synthetic noise cases.

4.4.1 Redefining the update for point-clouds

We now describe an intuitive and straightforward way in which the Patch-Beltrami filter can be applied to a point cloud to perform denoising. We consider a point cloud to be simply a set of points which have been noisly sampled from an underlying manifold where each point has been sampled above the tangent plane to that manifold. This perspective immediately leads to a procedure by which we can adapt our method for denoising. If we can approximate the tangent plane and the local neighborhood then we can directly apply a single "pixel" denoising update step as if we were dealing with a grayscale image or depth map. In other words for every point we approximate its local geometry, perform an update and then map the update back onto the point and repeat until convergence. A more explicit algorithmic description is shown in algorithm 4.1 and we will expand on the relevant steps of this algorithm in the coming sections.

Algorithm 4.1 Patch-Beltrami point set denoising algorithm

procedure CLOUD PATCH DENOISE($\mathcal{N}$)
  for iterations $t = 1, 2, \ldots, T$ **do**
    for $i = 1, 2, \ldots, N$ **do**
      Compute point neighborhood $\mathcal{N}_i$.
      Estimate normal direction n_i.
      Use the projection of points in $\mathcal{N}_i$ in the normal direction to obtain a local parameteric surface representation I_i.
      Smooth the neighborhood's representation using the patch-space Beltrami flow.
      Update point i according to its change in I_i in the normal direction.
    end for
  end for
end procedure

4.4.2 Local neighborhood

The Patch-Beltrami approach uses the regularity of a discretized image grid to obtain the explicit neighborhood relations of surrounding pixels for the sake of obtaining a patch which describes the current pixel. In a point cloud setting we do not have the true neighborhood relations for each point. Our first step in adapting the approach is therefore choosing a neighbourhood definition. Taking inspiration from [115] we define a neighborhood $\mathcal{N}_i$ of a point p_i to be the set of its k-nearest neighbors intersected with the set of points that lie within a distance R where distance is measured in Euclidean space using the L^2 norm. The main question that arizes now is how to choose R consistently and effectively especially considering that the same underlying surface may have been sampled more or less densely. We will use $\mathcal{N}_i$ to define a support from which we construct a local frame around p_i. We note that choosing too-big an R will cause the tangent plane estimation to be inaccurate. However choosing R to be too small, it is possible that the noise will dominate the chosen neighborhood leading to an unreliable frame extraction.

In order to mitigate these problems we fix a minimum number of nearest neighbors k_{min} that we would like to have in any neighborhood. We then traverse the point cloud and estimate the average distance $r_{k_{min}}$ to the k_{min}^{th} nearest neighbor of every point. As in almost all denoising algorithms we assume we have some estimate of the level of noise σ that has corrupted the data. Ideally we would like a neighborhood to span a larger domain than the strength of the signal and so we define the neighborhood radius R to be

$$R = max\left(5\sigma, r_{k_{min}}\right), \tag{4.23}$$

which essentially guarantees that we will have at least k_{min} points in any given neighborhood. The number k of neighborhood points is adjusted to reflect the value of R so we set

$$k = \lceil k_{min} \frac{R^2}{r_{k_{min}}^2} \rceil \tag{4.24}$$

where the squared powers are a result of the very rough assumption that points are distributed more or less evenly per unit area of the tangent plane to each point. In practice the value for k has to be bounded from above because the even distribution assumption is only valid locally and often too many points may be included in some neighborhoods. We note that the neighborhood is kept constant throughout all iterations for computational reasons, but this also helps avoid clustering of points together, as noted by Lange and Polthier [80]. Figure 4.12 demonstrates the selection of a local neighborhood and the accompanying choice of local frame.

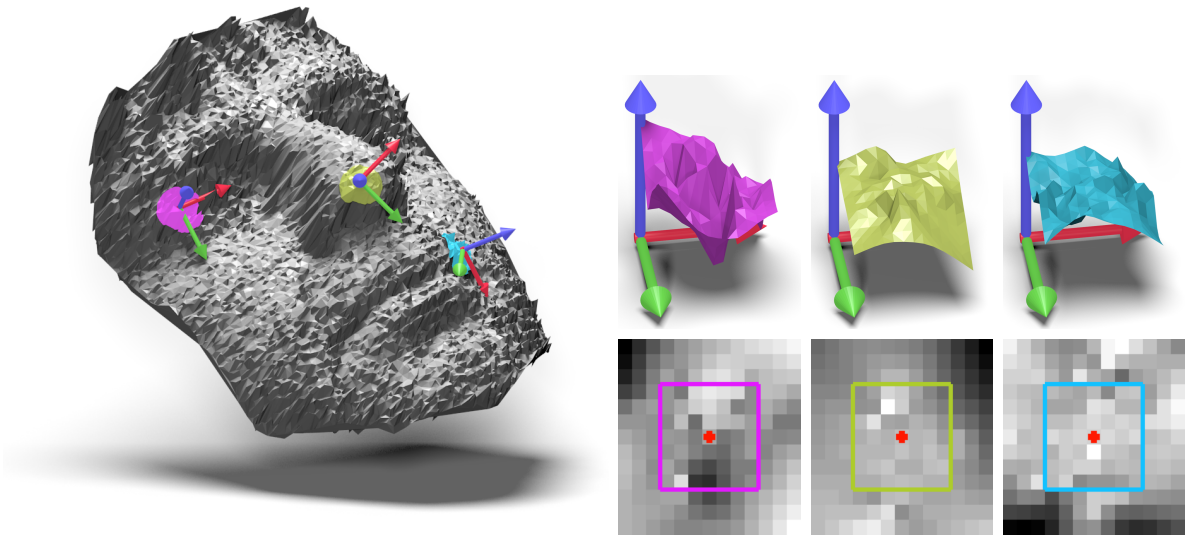


Figure 4.12: **Left:** a facial scan from a depth scanner. The image is rendered using the underlying geometry available from the scanning grid however we only use the point cloud without the mesh connectivity. **Right:** example patches extracted from the surface and used to denoise specific vertex locations. The grayscale images with colored rectangles indicate the interpolating cloud patches and the region used by the Patch-Beltrami denoising update step.

4.4.3 Frame estimation

For point p_i we would like to use $\mathcal{N}_i$ to help us approximate the noisy implicit surface on which p_i is placed. The denoising filter described earlier requires a grid to operate over and as such we need to interpolate the points in $\mathcal{N}_i$ over a discretized domain that spans an approximation of p_i's tangent plane. Approximating the tangent plane requires determining its normal vector n_i. One simple and well established method is to take the third principal component of $\mathcal{N}_i$'s coordinate matrix by performing principal components analysis (PCA) on the local patch. PCA is a linear method and is prone to outlier corruption. In fact the problem of robust normal estimation based on a point's neighborhood is non-trivial and is one of the major influencing factors in the efficacy of any method which requires point normals. Fleishman *et al*let@tokeneonedot [49] describe a robust normal estimation procedure which we adopt and discuss its benefits and application later on. Assuming that we have a method to estimate p_i's normal, we define two additional arbitrary vectors u_i, v_i that span the tangent plane and are orthnormal to n_i and to each other. All three vectors are orthonormal and as such form a local tangent frame. We use these vectors and the position of the point p_i to obtain a transformation

$$T_i = \begin{pmatrix} u_i^T & -(p_i)_u \\ v_i^T & -(p_i)_v \\ n_i^T & -(p_i)_n \\ 0\,0\,0 & 1 \end{pmatrix}, \tag{4.25}$$

which will take any point in $\mathcal{N}_i$ in its homogenous representation and transform it into a canonical frame. Provided that the surface does not fold in on itself, the points in this new canonical frame can be

interpolated over the domain $[-R, R] \times [-R, R]$ to form a height map I_i for point p_i. In our examples we found a simple bilinear interpolation to be sufficient. Figure 4.12 illustrates a number of patches transformed into a canonical shape using their detected frames.

4.4.4 Updating along the normal

The two previous processing steps have transformed $\mathcal{N}_i$ into I_i. In order to update p_i we perform a single Euler step along a discrete version of the flow described by (4.15), over I_i. We then extract the change in I_i for the central pixel, δ_i, as the central pixel corresponds to p_i. We update point i by the explicit step

$$p_i^{t+1} = p_i^t + \delta_i^t n_i^t, \tag{4.26}$$

where t indicates the global iteration step. When computing the discrete flow it is important to note that the scaling factor β from the patch-space metric H should have a fixed ratio proportional to the dimensions of the grid and be inversely proportional to the patch radius R because I_i's domain was defined to span twice the patch radius. However this is not necessarily the case in practice so we leave β as a somewhat flexible parameter to the method. We note that for a specific choice of β, the height map update will result in a variant of the mean-curvature flow. Such a flow does not preserve surface edges as is often desired in surface processing. The choice of a high β value, however, along with robust estimation of the surface normals, results in a highly nonlinear flow that preserves geometric edges while removing surface noise.

4.4.5 Robust normal estimation

The efficacy of the proposed method is largely dependent on how well we can estimate the normals at every point. While PCA-based normal estimation may be sufficient in some cases, robust normal estimation is desirable. One such method is the method of Fleishman et al. [49] introduced in the context of moving-least-squares approximation. The gist of the procedure is an iterative RANSAC-like process that builds and tests support sets of points from a neighborhood to try to find the normal vector with the least number of outliers. We found that this method preserved the corners more effectively compared to regular PCA. An example of the difference in the results can be seen in Figure 4.13

4.4.6 Results

We implemented the main algorithm in Matlab and used functionality from the open source PCL library [118], such as nearest-neighbor indexing. For all experiments we chose a grid size of 21×21 and a window size of 7×7. In principle the grid size should be dependent on the spacing of the data but we found for the data in our experiments it was sufficient to leave it constant. We used a number of the test shapes

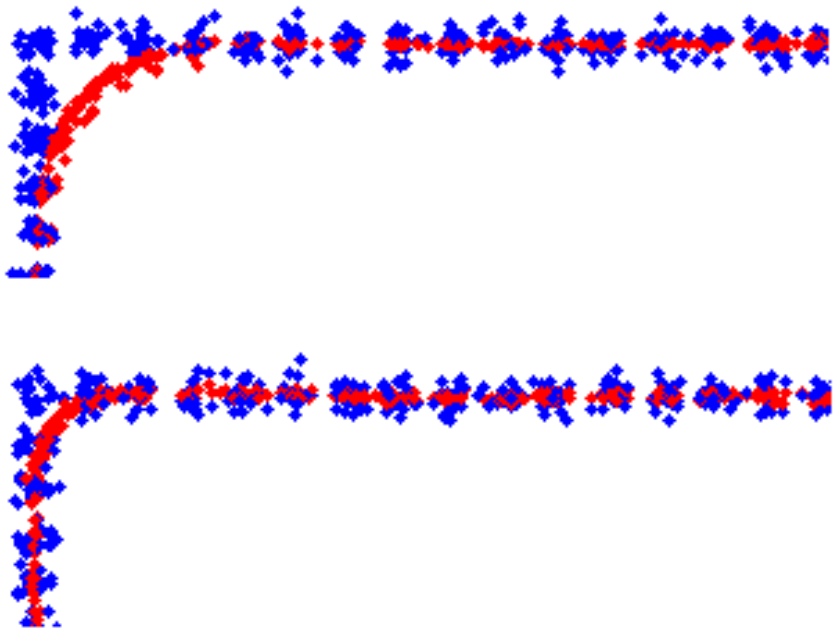


Figure 4.13: A comparison of PCA-based and robust normal estimation, in terms of smoothing results. **Top:** Smoothing results using PCA for normal estimation. **Bottom:** Results using robust normal estimation.

and error metrics from [154]. Specifically we use the normal angle error δ which measures the average difference between the direction of the resulting normals of the ground truth surface and denoised cloud. In addition to this we use the average point-to-surface error $\mathcal{D}_{mean}$ and the maximum point-to-surface error $\mathcal{D}_{max}$. Because each method has a different set of parameters which can be adjusted we quote the best result by performing a parameter search for each method. Our method requires defining the number of nearest neighbors to use (we used between 5 to 30) and the number of update iterations (we specific between 1 to 10). The results are shown in Table 4.4 and are compared to both Moving Least Squares [85] and Bilateral Mesh Filtering [50]. The results illustrate that our straightforward method for applying Patch-Beltrami to a point cloud is on par with or better than the popular MLS method. Furthermore we demonstrate that despite not having access to meshing information our approach nonetheless is still competitive with the popular mesh based approach of [50]. A qualitative visual display of the denoised shapes from Table 4.4 can be seen in Figure 4.15. The results are displayed as meshes because the ground truth connectivty is available but it was not used at any stage of the denoising. The resulting surfaces are smoothed while preserving the primary features despite the strong levels of noise. These results are encouraging and given the improvements to the Patch-Beltrami method that we discuss in the next section, it seems likely that training specifically for a deeper neural model may boost the results considerably. We have left this investigation for future work and believe that even more elaborate variations on the Patch-Beltrami model as opposed to our straightforward adaptation could potentially yield even more significant improvements.

In all our experiments we kept the neighborhoods of each point constant over all iterations of the

Table 4.4: Results from denoising of a set of standard objects represented as point clouds using the Patch-Beltrami filter. Our method compares favourably to the well known Moving Least Squares method of [85] and Bilateral Mesh Filtering [50] despite the fact that we do not using the structure of the underlying mesh. Figure 4.15 shows the shapes.

		Ours	MLS [85]	FDC [50]
Fandisk $\sigma = 0.7$	δ	**22.40**	23.50	22.50
	$\mathcal{D}_{mean}$	**2.69E-02**	2.93E-02	2.91E-02
	$\mathcal{D}_{max}$	4.26E-01	**4.16E-01**	4.74E-01
Sphere $\sigma = 0.3$	δ	15.90	15.20	**12.60**
	$\mathcal{D}_{mean}$	6.91E-02	**6.78E-02**	1.08E-01
	$\mathcal{D}_{max}$	**6.34E-01**	7.48E-01	8.38E-01
Bunny $\sigma = 0.2$	δ	**5.92**	6.35	6.93
	$\mathcal{D}_{mean}$	6.86E-04	**6.75E-04**	1.20E-03
	$\mathcal{D}_{max}$	**7.20E-02**	1.08E-01	1.47E-01
Julius $\sigma = 0.2$	δ	**6.64**	8.00	7.70
	$\mathcal{D}_{mean}$	**5.11E-04**	5.55E-04	8.44E-04
	$\mathcal{D}_{max}$	**5.36E-02**	6.29E-02	7.78E-02
Block $\sigma = 0.4$	δ	13.40	14.00	**12.70**
	$\mathcal{D}_{mean}$	**9.65E-02**	1.00E-01	1.43E-01
	$\mathcal{D}_{max}$	6.85E-01	**6.80E-01**	9.16E-01
Nicolo $\sigma = 0.2$	δ	8.44	**8.01**	8.88
	$\mathcal{D}_{mean}$	2.69E-01	**2.41E-01**	3.47E-01
	$\mathcal{D}_{max}$	**1.19E+00**	1.24E+00	1.77E+00

algorithm. This dramatically improves the run time of the algorithm and is justified by the fact that we do not expect the neighborhood of a point to change dramatically during the proposed denoising process.

4.5 Operator tuning: a deep perspective

The previous sections have discussed the Patch-Beltrami filter and it's extension to manifolds. However, despite the powerful modelling approach of dealing with images in a high dimensional spatial-spectral embedding, the quantitative results are still not on par with the state of the art. To rectify this we show in this section that a conceptual leap is required and we describe how it is possible to modify the framework to have a large number of trainable parameters. Doing so yields the ability to optimize for these parameters over a sufficiently well sized data set.

Our main observation is that by combining the advantages of PDE based image restoration procedures which have been studied over decades together with the modern tools of deep learning we are able to achieve results that are comparable to or exceed the state of the art. We indicate how it is feasible to directly model and include the Patch-Beltrami approach in an end-to-end trainable deep neural network pipeline using current tools and without the need for heavy theoretical or computational machinery. In this work we focus on the Laplace-Beltrami operator and the associated denoising PDE but we believe that the methodology we describe can and should be implemented for different problem domains.

The primary contribution of our Deep Patch-Beltrami approach is the development of deep neural network who's architecture is based on the Patch-Beltrami method. Within this context two important

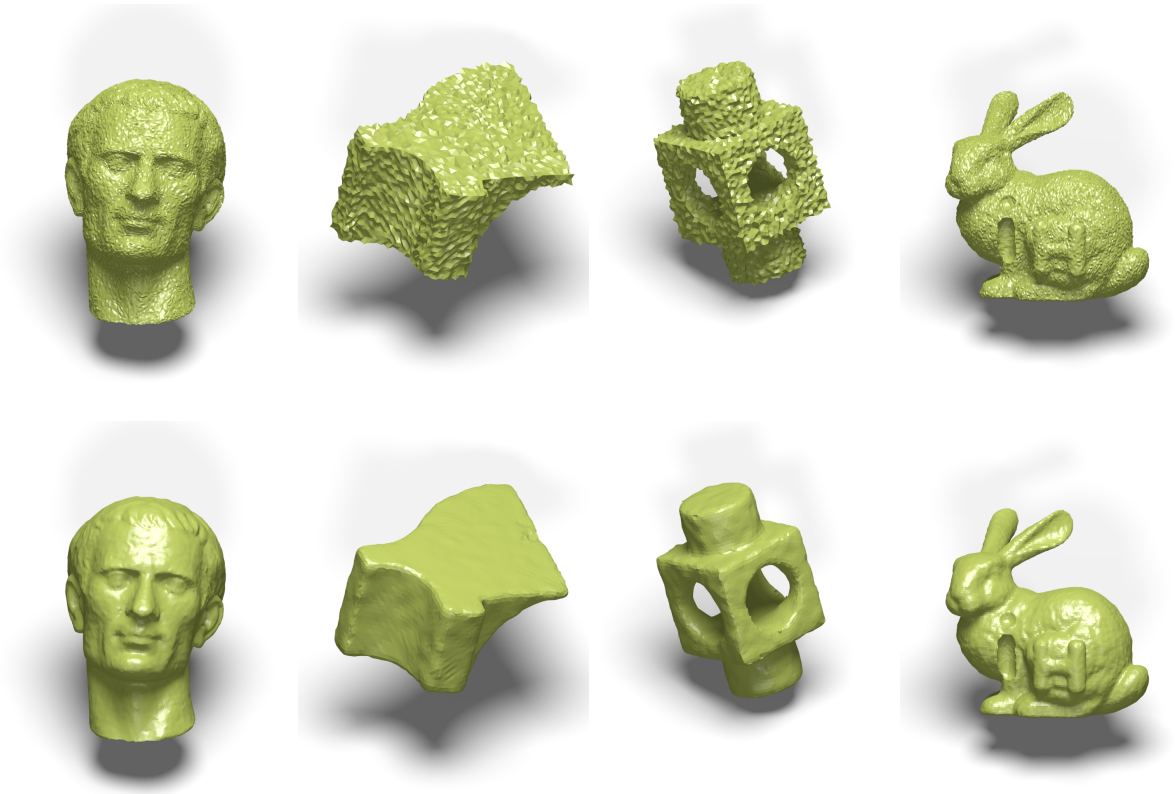
Figure 4.14: **Left:** A noisy point cloud obtained from a structured light scanner. **Right:** The corresponding point cloud after passing through the Patch-Beltrami filter operating on patch-space using the proposed method.

components are in the implementation and training procedure. For implementation we developed an open source tool [60] to enable easier visual development of network graphs. For training we show how we can take advantage of backpropogation to mitigate the complexities of explicitly computing the analytical gradient. We believe that this modelling process will be practically useful for other practioners in the field when solving similar problems.

4.5.1 Explicit schemes as recurrent tunable architectures

The core principle of our approach is to consider the iterative Patch-Beltrami explicit scheme shown in Section 4.2.2 as a recurrent neural network (RNN) and the structure can be observed in Figure 4.16. The key property which makes a neural network recurrent is that it feeds the output of a unit of computation back into the same unit. This matches the idea of iterated gradient descent for the explicit Patch-Beltrami scheme. Our approach is straightforward: each RNN stage will learn the optimal gradient step towards the solution. This is in contrast to the original scheme defined in Equation (4.15) where the descent step size and direction are defined entirely by the Laplace Beltrami operator and CFL conditions [75].

The idea of framing iterated supervised descent algoriths in this way has been around for some time in various guises [55, 33, 147] however the first significant use of modern deep learning approaches was suggested by [157]. There, the authors converted the iterative meanfield approximation for fully connected Gaussian CRF [78] into a recurrent neural network and achieved state-of-the-art image segmentation results at the time. They also found that by including the CRF as an iterative process within and end-to-end learning context they were able to use fewer meanfield iterations while maintaining good accuracy and reducing computational complexity. In other words, this approach took a powerful filtering process and incorporated its behavior so that a recurrent neural network could benefit from an inbuilt edge aware



Julius $\sigma = 0.2$ Fandisk $\sigma = 0.7$ Block $\sigma = 0.4$ Bunny $\sigma = 0.2$

Figure 4.15: Qualitative comparison of shapes denoised by the point cloud Patch-Beltrami method. The shapes are rendered using the underlying mesh for visual clarity however the denoising method does not use connectivity information. **Top row:** Noisy point clouds. **Bottom row:** Denoised point clouds.

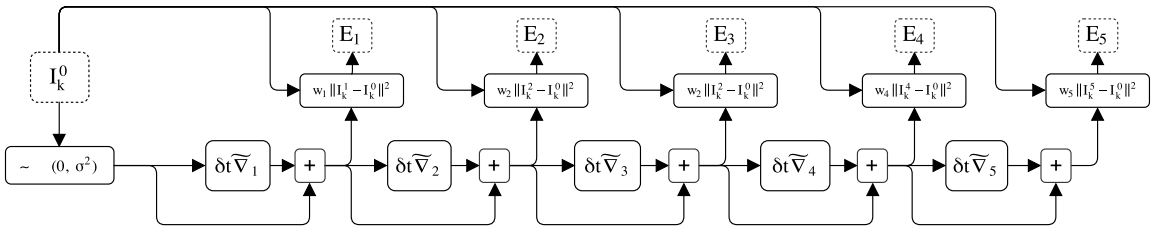


Figure 4.16: This is the computation graph for the Deep Patch-Beltrami with 5 stages.

denoising process. In the field of image restoration, the work of Chen *et al*let@tokeneonedot [28] is most similar to this approach. There they consider the Perona-Malik diffusion model as a recurrent network. They specifically modify the model to include paramterizable influence filters and non-linearities using RBF kernels.

Similar to the above methods, our approach to modifying the Patch-Beltrami to become a trainable recurrent architecture is to retain the structure of the computation in Equation 4.15 and reconstruct the computational graph as a set of neural network layers with each iteration unrolled into additional stages in the network. We constrain ourselves to N iterations and the recurrence relation from Equation 4.15 becomes an unrolled recurrent network and can be written explicitly as

$$I_k^{n+1} = I_k^n + \widetilde{\nabla} I\left(I_k^n; \Phi_n\right). \tag{4.27}$$

Crucially, we rewrite the differential operators 4.19 in the form of convolutions with adjustable kernels. To tune these kernels we define an optimization objective over the whole network to minimize the $L2$ loss between clean ground truth images y and the output of the network y_{est}. The inputs to the network are the noisy versions of y which we denote as y_σ which have additive Gaussian white noise at a predefined σ. This optimization objective is thus written as

$$E = \frac{1}{K} \sum_{k=1}^{K} \sum_{n=1}^{N} w_n \left\| I_k^{n+1} - I_k^{GT} \right\|^2 \tag{4.28}$$

Minimizing this loss essentially minimizes the mean squared error (MSE) which is equivalent to maximizing the PSNR. As noted, the only change we need make to our recurrent network which deviates from the original Patch-Beltrami is to convert every use of a differential operator into a trainable convolutional kernel. Apart from the operator structure and behavior this also differs from the RNN of [28] where in addition to convolutional differential operators the authors need to explicitly add a paramterizable RBF influence function to obtain richer non-linearities. The Patch-Beltrami does not need to explicitly add additional non-linearities because the behavior of the metric tensor provides this function. Taking this idea even further, we can interpret the process as multiple metric tensors for multiple separate embeddings that are then computed and merged together by learnt fusion paramaters. This is very reminiscent of ensemble learning where each weak-learner contributes when it is most relevant to the example at hand. Furthermore, these high dimensional embeddings create a rich feature space which enables the denoising behavior which we show in Section 4.5.3. This can be better understood by recognizing that the power of the original Beltrami filter came from the fact that it embeds an image into a spatial-spectral hyperspace. The Patch-Beltrami then increases the size of that hyperspace by including neighboring pixels. It thus appears that this new Deep Patch-Beltrami variation learns its own high dimensional embedding as opposed to

predefining it and in doing so achieves superior results.

4.5.2 Implementation

To implement the Deep Patch-Beltrami (DPB) neural network we make the following observation: regular Patch-Beltrami (PB) benefited from a fixed number of flow steps. Optimizing the parameters α and β for image denoising in Equation 4.20 using Nelder Mead yields fast convergence for such a few number of parameters. In DPB our aims is to allow a much greater model capacity by increasing the number of parameters while ensuring that we can find a good local optimum for our loss function over a given dataset. Thus for each unrolled iteration iteration in Equation 4.27 we are in effect learning optimal gradient descent steps towards the solution for the minimzation of the original Beltrami functional 4.7. Although not developed explicitly as such, training the parameters of the unrolled RNN in Equation 4.27 can be seen as a variation of the "learning to learn" framework [5]. In the language of that work the **optimizee** is our denoising functional and the **optmizer** is our parameterized RNN.

If we were to naively implement an optimization scheme to train our RNN **optimizer** to minimize 4.28 over the parameters Φ we would need to provide the analytically computed gradient of the loss for every subset Φ_n related to each iteration step n. Computing gradients is clearly an important element of any optimization scheme but it is often a complex and error prone task. The work of [28] directly compute the analytical gradient with respect to their stage parameters. They then use an L-BFGS optimizer to train their weights in a multiple stage training process.

Our approach contrasts with this. Instead of explicitly computing gradients we implement our RNN network as a computational graph in Caffe [70] using pre-existing network layers as well as some customized modifications. This relieves us from the burden of manually deriving gradient expressions with respect to parameters for each flow step and instead puts the main focus on designing the forward pass algorithmic steps. This is particularly flexible as we do not need to recompute a new anlytical gradient form for every modification we make. To perform this step we take advantage of the error backpropagation update functions for these layers which is none other than the chain rule. The computational graph for each layer can be seen in Figure 4.16.

We found that to further enable flexible modifications of different parts of the framework it was necessary to see the visible structure of the computation graph as opposed to generating it purely programmatically. To support this we developed a neural network graph editor on top of the popular Blender node graph [52] using open source code [60]. An example of the full design of our network using this tool can be seen in the screenshot in Figure 4.17. This development tool enabled fast modification of graph connections and information flow. It also enabled easily replicating or removing stages and changing the behavior of shared weights and filter channel outputs with minimal effort during training.

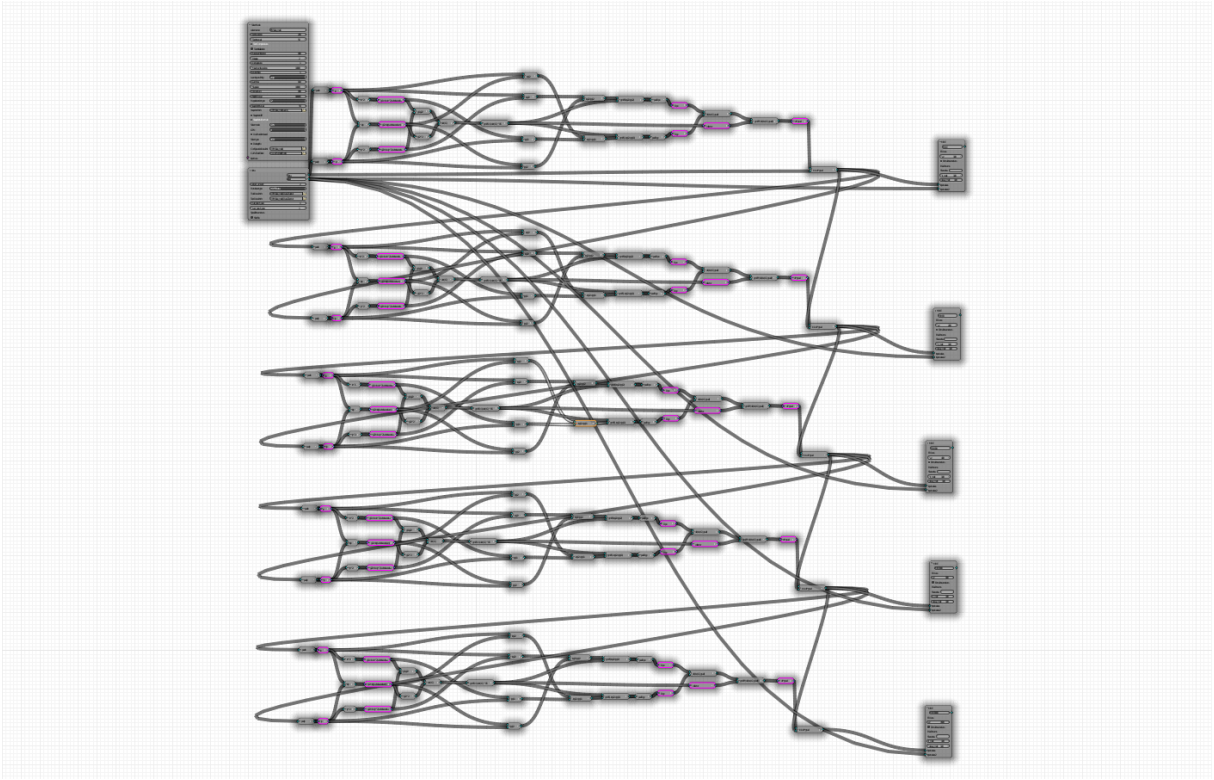


Figure 4.17: Blender design tool example for DPB. The software enables drag and drop of network nodes and adding different parameters for each node. The visual layout with different colors and hidden parameters is critical in designing new network architectures.

The tool is extensible so that new layers can be added as part of the workflow without great additional effort.

4.5.3 Experiments

We test our approach on two datasets and compare to [28] which is the closest to ours and considered state-of-the-art[3]. We use the Berkeley dataset for train training and testing data. We also introduce the use of a second dataset purely for testing. This is the Adobe portrait dataset [126] which contains only images of people in portraits and is a highly representative of a large portion of the kind of digital photography content produced by modern use of cellphones. This dataset is aimed at testing generalizability into a subset of the space of natural images (people).

Training

There are 400 images in the Berkley training set and 68 for testing. We selected 92 images arbitrarily from the original Adobe portrait dataset. The set of test images can be seen in Figure 4.6

[3] as noted before, many similar approaches have been suggested since these experiments were performed

Table 4.5: Test set: 92 face portraits

Method→ Noise $\sigma \downarrow$	NLM	BM3D	NRD-7x7_5	NRD-7x7_8	Belt	PB (ours)	DPB (ours)
5	38.15	39.78	39.83	–	37.76	39.07	**39.84**
10	34.94	36.19	–	–	33.54	35.38	**36.30**
15	33.21	34.21	34.35	34.37	31.23	33.36	**34.38**
20	31.85	32.83	–	–	29.67	31.96	**33.03**
25	30.72	31.79	31.92	31.97	28.49	30.86	**31.98**
30	29.77	30.93	–	–	27.59	29.95	**31.15**
40	28.22	29.48	–	–	26.22	28.43	**29.81**
50	27.04	28.63	28.7	28.81	25.18	27.13	**28.83**

Table 4.6: Test set: 68 images of natural scenes

Method→ Noise $\sigma \downarrow$	NLM	BM3D	NRD-7x7_5	NRD-7x7_8	Belt	PB (ours)	DPB (ours)
5	35.94	37.59	**37.74**	-	36.25	37.14	37.70
10	31.35	33.32	-	-	31.48	32.8	**33.52**
15	29.72	31.08	**31.43**	**31.43**	28.98	30.53	31.37
20	28.55	29.62	-	-	27.37	29.08	**29.92**
25	27.61	28.58	28.92	**28.96**	26.22	28.03	28.87
30	26.81	27.75	-	-	25.34	27.19	**28.04**
40	25.56	26.47	-	-	24.06	25.9	**26.81**
50	24.6	25.62	25.97	**26.02**	23.15	24.86	25.95

and Figure 4.5 . We add Gaussian white noise to all the images at the following standard deviations: $\sigma = 2, 10, 15, 20, 25, 30, 40, 50$. To train our network, we cut out 1600 random patches of size 100×100 from the original training images. Using the graph builder tool mentioned in the previous section we generate a Caffe network and fed the network the noisy patches and their clean counterparts. As is typical for denoising methods we assume that noise magnitude is known apriori and build separate denoising networks for each individual σ. All training was performed on an NVidia $K80$ with an Intel i7 motherboard and 32 GB of RAM. To train our multistage RNN we first tried to train stage by stage and then end-to-end. We tried a number of different parameter settings and tried different approaches i.e. tying all weights, untying some weights, changing and increasing number of stages and the number of parameters. Using SGD we struggled to get the network to converge but eventually did manage by using a batch size of 3 and a learning rate of 10^{-11} together with scheduled learning rate reduction. Eventually we switched to Adagrad [42] which allowed much faster convergence and experimentation. It also enabled us to train end-to-end without having to perform individual training of each stage. When the metric tensor weights are not shared within a stage then the format of the metric tensor is destroyed and the requirement for semi-positive definite (SPD) metric tensors is violated. We considered adding an energy term to enforce SPD but ultimately decided to tie the weights directly to ensure SPD of the embedding tensors.

In addition to the tunable parameters we tried a number of different boundary conditions for the

convolutions. This was achieved simply by changing the behavior of the padding. Although clamped or reflection padding initially seemed promising, eventually constant padding with a value of 128 was the most succesful. Alternatively the same results were achieved by first centering every input value around 0 by subtracting 128 and then using 0 padding throughout the network. The choice of padding had a surprizingly large impact on the final PSNR results often improving by up to $1\mathrm{dB}$.

We found that our best networks for all values of σ were in the form of a 5 stage network, untied weights between stages, tied metric tensor weights within each channel, 7×7 convolution kernels with 80 output channels for each convolution, constant 128 border padding, 100000 Adagrad iterations at a learning rate of 0.001, a batch size of 3 and a training patch size of 100×100. Changing any of these values had a negative impact on the training results. Although it may seem apprioriate to train more stages and larger filter sizes we were limited by memory restrictions during training. We found that a good way to verify the progress of the training was to visually inspect the appearance of the first stage filters. Filters which resembled oriented edge or textures and contained low levels of noise indicated good convergence progress. An example of the first stage filters resulting from the training process can be seen in Figure 4.18.

Results

Visual comparisons between the behavior of the Deep Patch-Beltrami with the regular Beltrami and Patch-Beltrami filters can be seen in Figure 4.20 and Figure 4.19. A synthetic example to illustrate the visual noise reduction compared between the three variations can be seen Figure 4.21. In Table 4.6 and Table 4.5 we compare the original Beltrami filter, the Patch-Beltrami filter, Deep Patch-Beltrami filter, the $TRD7 \times 7$ method of [28] and the classic $BM3D$ method [34]. On the Berkley test set our 5 stage DPB outperforms $BM3D$, is on par with the 5 stage $TRD7 \times 7$ network and very close to the 8 stage $TRD7 \times 7$ variation. This is perhaps unsurprizing as the methods were trained on the same dataset and share similar kernel sizes and capacity. Interestingly however, exactly the same 5 stage DPB network outperforms the $TRD7 \times 7$ for both 5 and 8 stages on the Adobe faces dataset. This result suggests that despite all the networks being trained on exactly the same data and never having seen any of the images or image characteristics of the Adobe faces dataset, the DPB approach is generalizing better. We believe the reason for this is that the TRD method is overfitting on the underlying data distributions of the Berkley dataset. We posit that the DPB does not suffer from this issue because it retains it's geometric characteristics through the SPD metric tensors which are a strong natural prior in the high dimensional embedding space generated by the patch based embedding.

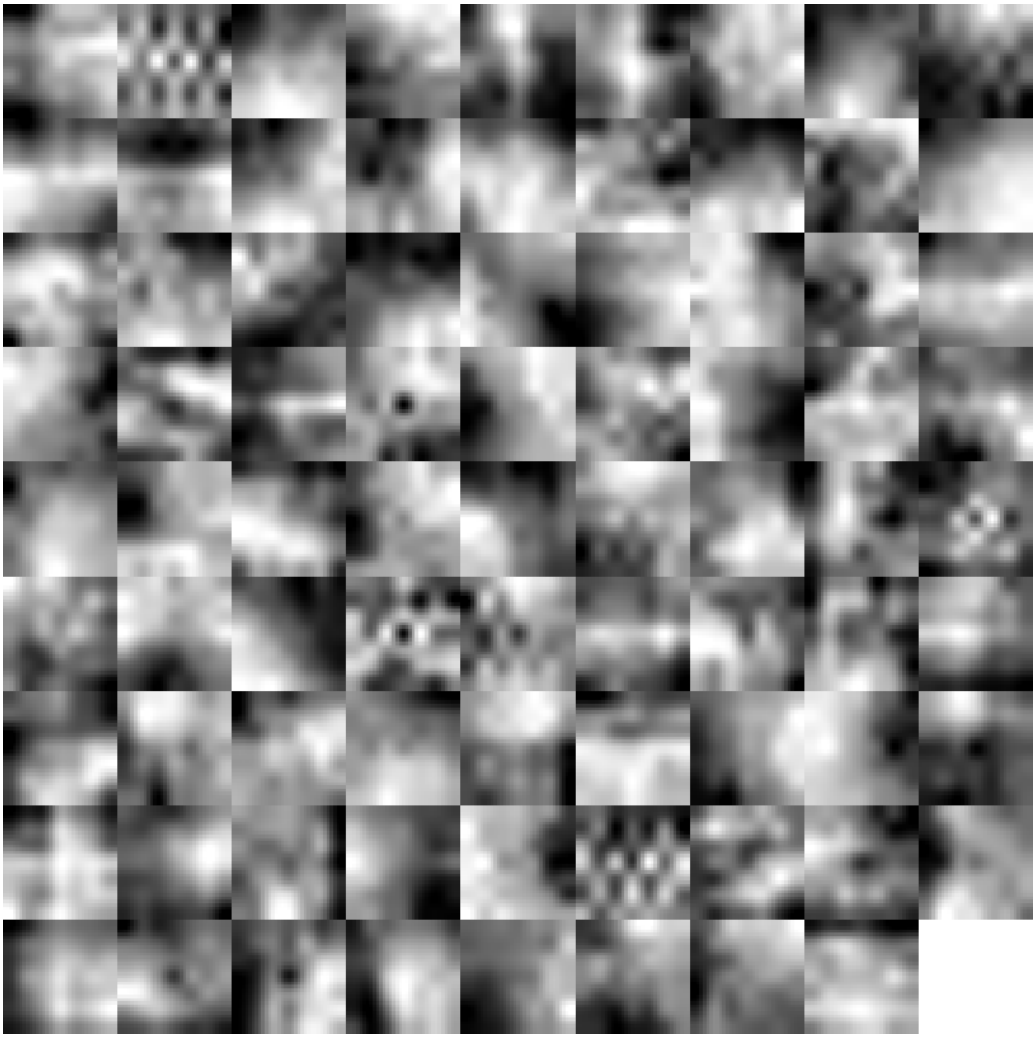


Figure 4.18: Montage of first stage learnt filters for the first DPB operator. Various common patterns such as corners and edges are visible along with less interpretable visual shapes. The patches have been rescaled to enhance visibility.

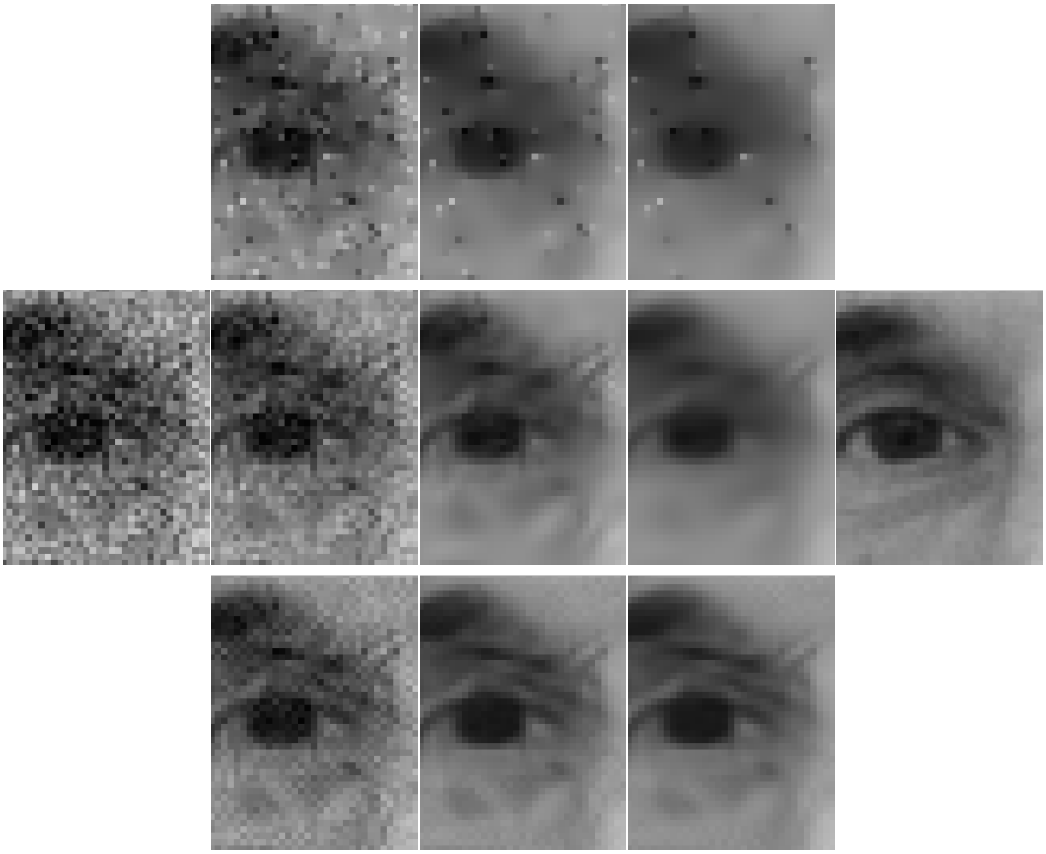


Figure 4.19: **Left image:** noisy image of Brian Cox' eye. **Right image:** clean image of Brian Cox' eye. **First row, left to right:** $20, 40, 50$ iterations of Beltrami filtering. **Second row, middle three images:** $20, 40, 50$ iterations of Patch-Beltrami filtering. **Last row, left to right:** $2, 4, 5$ iterations of Deep Patch-Beltrami. All images are zoomed in to highlight differences in denoising fidelity.

Figure 4.20: **Left image:** noisy image of Brian Cox, $\sigma = 30$. **Right image:** clean image of Brian Cox. **First row, left to right:** $20, 40, 50$ iterations of Beltrami filtering. **Second row, middle three images:** $20, 40, 50$ iterations of Patch-Beltrami filtering. **Last row, left to right:** $2, 4, 5$ iterations of Deep Patch-Beltrami. See Figure 4.19 for zoomed in versions which highlight denoising fidelity.

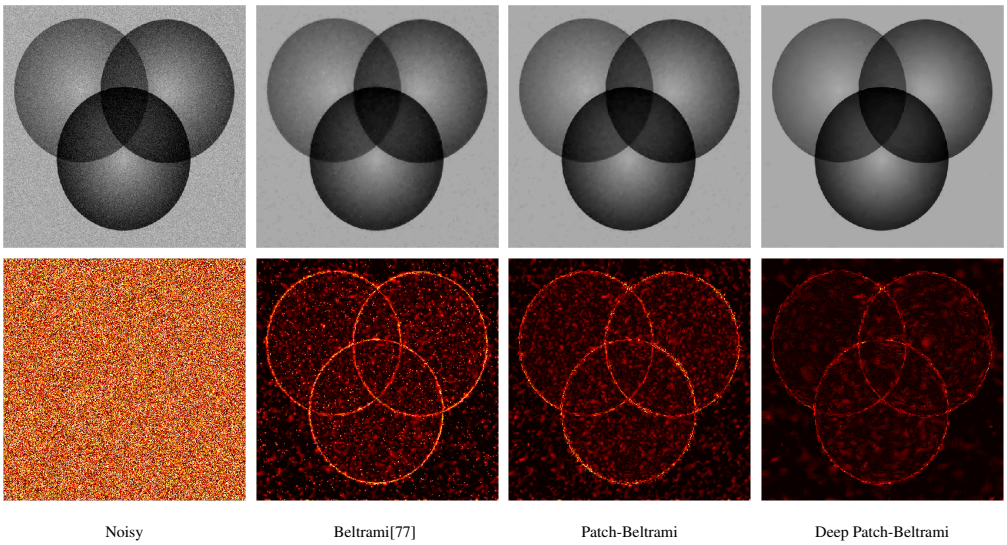


Figure 4.21: Comparison of noise for the different Beltrami methods. **Top row, left to right:** Noisy Venn diagram, denoised using Beltrami filtering, denoised using Patch-Beltrami filtering, denoised using Deep Patch-Beltrami. **Bottom row, left to right:** Residual noise after subtracting from the original noise free Venn diagram. This figure highlight the dramatic advantage the Deep Patch-Beltrami has over the regular versions. Noise along edges and in open areas is significantly reduced.

4.6 Discussion and conclusions

While the current implementation of the Deep Patch-Beltrami takes around $1\,\mathrm{s}$ on the GPU to perform denoising, this is primarily because it has been implemented to enable research flexibility as opposed to run time efficiency. However an order of magnitude or more improvement in speed would be possible because compressing the layers into single computation kernels, as was done for the mobile GPUImage Patch-Beltrami implementation, should have a dramatic impact on speed. In fact possibly the most interesting application of this kind of powerful denoising framework is for improving mobile photography so efficiency should be a first class topic in future work.

Given the fairly general approach taken here, it would appear that future promising avenues of research would extend to more complicated domains within the field of classic geometric vision. For example the geodesic active contour model of [41, 20] uses a proxy levelset to implictly represent a surface undergoing geometric flows. The updates to this levelset are driven by a gradient descent process determined by the Euler-Lagrange obtained from the original continuous formulation. This is very similar to the way in which we derived the Patch-Beltrami. Numerous other PDEs could also be approached including sharpening, image completion, superresolution, image impainting and more by directly converting the iterative processes used in classical approaches into recurrent networks with relaxed convolutional differential operators. In fact, it appears that in the modern age of deep learning, classical variational methods and their associated PDEs may potentially find a renewed use within learning based approaches.

In conclusion, in the chapter we have demonstrated a flexible pipeline which we have called the Patch-Beltrami Framework. The system uses geometrical reasoning over local neighborhoods combined with trainable parameters within a deep neural architecture paradigm to produce a powerful denoising framework which can be straightforwardly adapted to different domains. We hope this work will help others in the community by laying some of the groundwork for greater integration of powerful classic methods with modern data based approaches.

Part III

CLASSIFYING DEPTH

Chapter 5

Randomized decision trees on tera-feature scale data for vision applications

In this chapter we discuss a novel method of parallelizing the construction of a randomized decision tree on non-homogeneous streaming multiprocessor architectures, that is multiple possibly different GPUs in a single workstation. The driving motivation behind our effort is to produce powerful classification trees for use in computer vision applications that require very few decisions per output pixel prediction. We use an efficient node indexing and partitioning scheme that allows maintaining the data points ordered according to which node they belong to thus permitting cache locality and improving data throughput on modern GPUs. We design an algorithm which constructs a tree in breadth first order and uses two different GPU kernels for building histograms depending on the current depth of the tree. To mitigate the potential differences in computational ability between available worker GPUs, we employ a consumer-producer work distribution model which eliminates the need for intelligent load balancing. Unlike other existing classification tree training techniques, ours can handle billions of training examples on a single modern GPU with the number of attributes limited only by disk storage capacity. This research was initially motivated by the need to handle very larg quantities of training data produced when training hand part detectors for depth image classification. However, the method is general in its applicability and its out-of-core approach means that it can be applied to any large training without necessitating to the use of a large computing cluster.

5.1 Motivation

The use of cascaded feature based decisions for realtime computer vision was popularized by the now classical boosted Haar cascades for face detection of Viola and Jones [138]. As a result of the success of that approach many vision researchers and practitioners began to use tree and boosting based classification

for efficiently performing computer vision problems such as object detection [152], tracking [71], human pose recognition [47, 127], face landmark tracking [72] and orientation regression [66] and many others. The random forest approach for human pose detection by Shotton *et al*let@tokeneonedot [127] was arguably the most influential in the last decade in further promoting the use of decision tree based vision systems. In order to produce a highly compact super-realtime pose recognition system, the authors of that work chose to train a random forest on a large scale dataset. To do so they needed to develop a massively parallel decision tree learning system that operated on a 1000 core computing cluster while customizing an internal distributed system called DryadLinq [18]. The success of their approach stemmed from the fact that they could quickly train and test new models and easily scale up to hundreds of millions of training examples and the Kinect from Microsoft was the end result. This cycle of iterative research and training is crucial to the success of machine learning systems that operate on large datasets. The success of modern deep-learning based approaches is said to have been enabled by the fact that the use of GPUs brought down training times on neural architectures from weeks or months to hours or days thus allowing fast iterative training cycles. We believe this is the reason that despite the success of the approach taken in [127], the use of decision trees for vision based applications has been largely surplanted by deep learning based methods. While there are numerous available distributed tree learning systems [94, 151, 104, 105, 18, 95, 26] thay can operate on large datasets they all have non-trivial barriers to entry int erms of setup costs, complexity and scalability. While the more recent gradient boosting frameworks such as XGboost [26] and LightGBM [95] can take advantage of GPU acceleration and are highly effective for regression and binary classification, they are not naturally designed to hand multi-class classification.Instead classification is typically performed as a 1-vs-N problem with N binary classifiers trained. This immediately results in a runtime which increases in direct proportion with the number of classes. In other, all other factors being similar, a 50 class boosted gradient system will run 10 times slower than a 5 class system. Indeed most approaches are designed to efficiently build multiple shallow trees as opposed to a small number of deep trees. Both large ensembles of shallow trees and small ensembles of deep trees have the same runtime complexity of $O(ND)$ where N is the number of trees and D is the maximum depth of the trees. However the first is typically charactersized by $N >> D$ (eg 500 trees, depth 8) resulting in approximately $O(N)$ inference and the latter by $D >> N$ (eg 1 tree, depth 21) resulting in approximately $O(D)$ inference. Deep tree ensembles are therefore inherrently more computationally efficient at runtime and especially useful for per pixel predictions. These deficiencies of modern classification tree builders have led us to design a new parallel approach for the construction of a randomized decision trees that can run on single node workstations while taking advantage of powerful modern GPU processors.

5.2 A brief overview of decision trees

A decision tree is a hierarchical tree data structure which performs tests at each tree node to determine the validity of a query on a data sample which has traversed the tree down to that node. One can think of a tree as a predetermined set of questions which are asked sequentially and conditionally on the answer to the previous question in the same fashion as the game *20 questions*. The answer to the series of questions results in a data sample following a node trajectory down the branches of the tree until eventually arriving at a terminal leaf node with no children. Having arrived at a leaf, previously stored information such as the posterior distribution over the question responses can be used to make a decision about the nature of the data query which has arrived. Intuitively, this means that data that responds the same way to the same series of questions is likely to be similar if the questions are sufficiently relevant. The problem of finding a set of optimal queries with which to build a tree is an NP complete [67] problem and therefore trees are built from the root down using an information theoretic approach with greedy optimization.

Trees are built from a training set $\mathcal{D} \subseteq \mathcal{X} \times \mathcal{Y} = \left\{(x_i, y_i)_{i=1:|\mathcal{D}|}\right\}$. Here each pair (x_i, y_i) in the training set is comprised of an attribute space vector $x_i \in \mathcal{X}$ and of an output space label or vector $y_i \in \mathcal{Y}$. For both classification and regression building the tree involves finding a splitting function f for each tree node that results in the maximization of some criterion for an optimal split defined over the subset of $\mathcal{D}$ that has traversed to that node from the root. Decision trees usually involve binary split nodes and as such the splitting functions are often from a family of parameterized predicates of the form

$$f\left(x; \theta, \tau\right) = \left[\theta\left(x\right) > \tau\right], \tag{5.1}$$

where $[.]$ is the indicator function. The function θ is chosen per node from some set of K functions $\Theta = \{\theta_k\}$ which can be applied to x. A common choice is

$$\theta_k\left(x\right) = x^T e_k, k = 1, \ldots, \dim\left(x\right), \tag{5.2}$$

which results in axis aligned splits where e_k is a unit vector along the k_{th} axis.

Training the tree involves determining for each node j an optimal split pair $\left(\theta_k, \tau\right)_j$ by applying f to the subset $\mathcal{D}_j \subseteq \mathcal{D}$ that has arrived at j yielding a tuple comprised of the training samples at $\mathcal{D}_j$ and the child subsets $\mathcal{D}_j^L$ and $\mathcal{D}_j^R$. If we have a general measure of impurity I then we would like to decrease impurity between a parent and its children so we are looking for the change in impurity ΔI such that

$$\theta^*, \tau^* = \arg\max_{\theta,\tau} \Delta I. \tag{5.3}$$

The form of ΔI is *(parent impurity) - (Σ weighted child impurity)*. For a multi-split node this will be

given by

$$\Delta I = \left(I\left(\mathcal{D}\right)\right) - \left(\sum_i \frac{\left|\mathcal{D}^i\right|}{|\mathcal{D}|} I\left(\mathcal{D}^i\right)\right). \tag{5.4}$$

For the case of a binary split the right (or left) set is implicitly defined by its sibling and parent such that

$$\Delta I = I\left(\mathcal{D}\right) - \frac{1}{|\mathcal{D}|}\left(\left|\mathcal{D}^L\right| I\left(\mathcal{D}^L\right) + \left(|\mathcal{D}| - \left|\mathcal{D}^L\right|\right) I\left(\mathcal{D}/\mathcal{D}^L\right)\right), \tag{5.5}$$

where $|.|$ counts the members of a set.

The choice of I is chosen according to the given problem domain. In a classification setting the entropy impurity is often used where

$$I_{entropy}\left(\mathcal{D}\right) = -\sum_c p_{\mathcal{D}}^c \log_2\left(p_{\mathcal{D}}^c\right). \tag{5.6}$$

Here $p_{\mathcal{D}}^c$ is the empirical probability of label c in data set $\mathcal{D}$ when the output space $\mathcal{Y}$ is a set of discrete labels. Another option is the Gini impurity

$$I_{gini}\left(\mathcal{D}\right) = 1 - \sum_c \left(p_{\mathcal{D}}^c\right)^2, \tag{5.7}$$

which is somewhatslightly cheaper to compute than the entropy measure.

The effect of these impurity measures is to progressively reduce the per node variability of the class distribution of arriving data points. The same is true in a regression framework but variability is measured according to standard deviation for non-interacting attributes such that

$$I_\sigma\left(\mathcal{D}\right) = \sum_k \sigma_k^2, \tag{5.8}$$

where σ_k is the standard deviation of the k_{th} attribute. Alternatively, interacting attributes with partial or full covariance can be used by setting

$$I_\Sigma(\mathcal{D}) = \log\left(|\Sigma|\right). \tag{5.9}$$

Computing the full data covariance matrix Σ at every node for more than a few attributes however is often prohibitively expensive and is therefore less common to do so in practice.

The impurity measure can be considered to be a potential energy and so any model fit to the data at each node which reduces a fitting energy is also an acceptable alternative to the uni-modal Gaussian assumption implied by variance minimization. For example fitting a Gaussian mixture model to data at the nodes would be an expensive but equally valid way to measure the quality of a node split. In the

same line of reasoning multiple different objectives can be combined and weighted to control the growth and structure of a tree. Once a tree t has been built the distributions stored at each leaf can be used for inference on new data because they represent the empirical estimate of the posterior probability $p_t(c|x)$ for tree t. For regression this is usually replaced with a compressed representation of the posterior by using the mean of the output samples that have arrived at the leaf which we will denote $\overline{y}_t(x)$.

The scenario in which there are multiple trees is known as a tree ensemble. Classification of a data point using the tree ensemble is performed by passing it down each tree and then finding the most probable label by averaging the posterior probabilities over all trees at each leaf node and choosing the most likely label c^* where

$$c^* = \arg\max_c \frac{1}{T} \sum_{t=1}^{T} p_t(c|x). \tag{5.10}$$

Regression is performed in a similar fashion using

$$y^* = \frac{1}{T} \sum_{t=1}^{T} \overline{y}_t(x). \tag{5.11}$$

5.3 Past work on randomized tree building

Decision trees have proven to be a powerful tool in many fields of applied computer science such as machine learning and pattern recognition. However they have an inherent tendency to over-fit training data. This has led researchers to develop more robust training algorithms [13][4][14][54] that sub-sample and randomize the splitting features tested at each decision node and generate randomized tree ensembles that have greater generalization power and demonstrate max-margin properties. We are specifically interested in efficient induction algorithms for building randomized tree ensembles which in many cases share properties with building regular decision trees.

Single node parallelized tree induction

The C4.5 decision tree algorithm of [113] is a classic tree builder where each attribute list is first sorted and the values used as thresholds to determine the best split for each node. The sorting is done so that the complexity of rebuilding partition statistics is BigO(1) . Extending this to perform randomized tests is straightforward but still requires sequentially passing over each attribute array. A parallelized version of the C4.5 algorithm is described by Narlikar [97] which uses posix threads and a parallel quicksort to gain parallelism. However the quicksort parallelization is not effective for randomized splitting tests because typically the number of splits that are checked are far smaller than the size of the data set thus not warranting the time used to perform the sorting repeatedly. However the fine grained parallelism using posix threads holds merit and as we will describe in Section 5.4, our method uses a similar approach.

Another approach called SLIQ [92] sorts every attribute only once and for each attribute stores a full index back to an unsorted version of the output class list. Every entry in the class list contains an index into the tree node that it has fallen to. This way the need to perform sorting for all the data at every stage is performed only once as a preprocessing step and thereafter replaced by an indexed lookup into the relevant tree node and subsequent update of tree node index and partition statistics. The method permits out-of-core data sets and breadth first tree building with single passes over the data at each stage. However, it is limited by its memory requirements resulting from double indexing which also cause highly scattered memory access patterns. A similar algorithm called SPRINT [124] improves on SLIQ and does not depend on a centralized class list and parallelizes by distributing samples across multiple processors. This requires maintaining a hash lookup table and all-to-all broadcasts between all processors incurring significant overhead for disk writes when saving lists. An improvement over these algorithms was achieved by Rainforest [57] by not requiring storing of sorted attribute lists and instead storing so called AVC groups which are sufficient to compute the split node parameters. However the method does not scale well to large numbers of numerical attributes and classes and can incur prohibitively expensive disk reads and writes.

Cluster-based tree induction

Increased networking speeds and processing power have resulted in large clusters becoming viable computing platforms. Using the MapReduce [38] programming model Panda et al. designed PLANET [105] to handle very large data sets across a distributed and scalable cluster built on commodity hardware. The method builds trees breadth first and sub-samples the range of possible split thresholds to test. The map phase distributes training records to Mappers which scan the data set and outputs $\langle key, value \rangle$ pairs to be processed by Reducers. The reduction phase aggregates the $\langle key, value \rangle$ pairs, computes splits and outputs the best split seen by each Reducer which are then fed back to the Controller which in turn updates the model as is standard in MapReduce frameworks. The platform intelligently distributes work using a complicated but robust scheduling policy. Although the procedure is scalable the law of marginal returns means that PLANET does not scale linearly past a few hundred machines as a result of various runtime overheads. Using 200 machines on 315 million records with 10 attributes they train a tree to depth 13 in $1000\,\mathrm{s}$. Using a similar MapReduce framework called DryadLINQ, Budiu et al. [18] demonstrate a cluster based implementation for training the Kinect body part detector. They use 235 quad-core machines to train a randomized tree on 600 million data points with 31 classes and 2000 attributes in a day. In general, cluster based solutions require advanced networking backbones, multiple machines, server rooms, additional engineers and are generally inefficient and expensive to run. Intra-cluster communication and distributed file system speeds are also often a speed bottleneck. However, cluster based solutions are

appealing because they enable parallelization over hundreds or thousands of processors.

GPU-based tree induction

In a similar line of reasoning, modern streaming multiprocessors such as NVidia or ATI GPUs also contain thousands of computing cores and are therefore highly efficient parallel computation platforms. One such example, ForestFire [125], uses onboard graphic texture memory for computer vision related feature generation. The node histograms are stored efficiently in memory but in general the method does not map well to general streaming architectures in its reported implementation. It is limited in the depth of the trees it can build and the data is fully on-core which means the method is handicapped in terms of the quantity of data it can handle. Another fully on-core CUDA-based [98] method called CudaRF [58] maps the building of each tree in a decision ensemble to a single CUDA thread. The method cannot take into account tree level parallelism and is therefore wasteful for small ensembles on a kilo-thread GPU. It is thus apparent that an algorithm that can effectively take advantage of modern GPUs or highly parallel CPUs for the computationally expensive but highly parallelizable training stage for randomized decision tree induction on single node workstations has been lacking and we now present our novel induction algorithm.

5.4 Novel approach to randomized tree induction

We take inspiration from PLANET [105] and SLIQ [92] to develop a novel mapping of the building of randomized decision trees on streaming multiprocessor architectures and specifically single workstation CUDA enabled GPUs. Randomized decision trees are typically used in ensembles to create a random forest but we are interested in building deep trees as opposed to many shallow trees as is typical for random forests. However the process can be repeated multiple times to populate a forest of trees if necessary. Here we also note that our randomized decision trees consider a predefined number of random thresholds to obtain split candidates and we use all the data for assessing every new split instead of subsampling. We will now describe the method we propose with which we train unpruned, deep trees on parallel hardware.

Feature parallel learning

The primary design decision upon which our method is based is the vertical split of our data matrix into attribute lists which are then stored on out-of-core media. In other words we explicitly split our data matrix into a set of files which store individual attributes. As a result of this architectural choice we focus on using only a single feature per tree node which is typically referred to as an axis aligned split node and we use Equation 5.2 to perform the split decision. This further allows us to consider only a

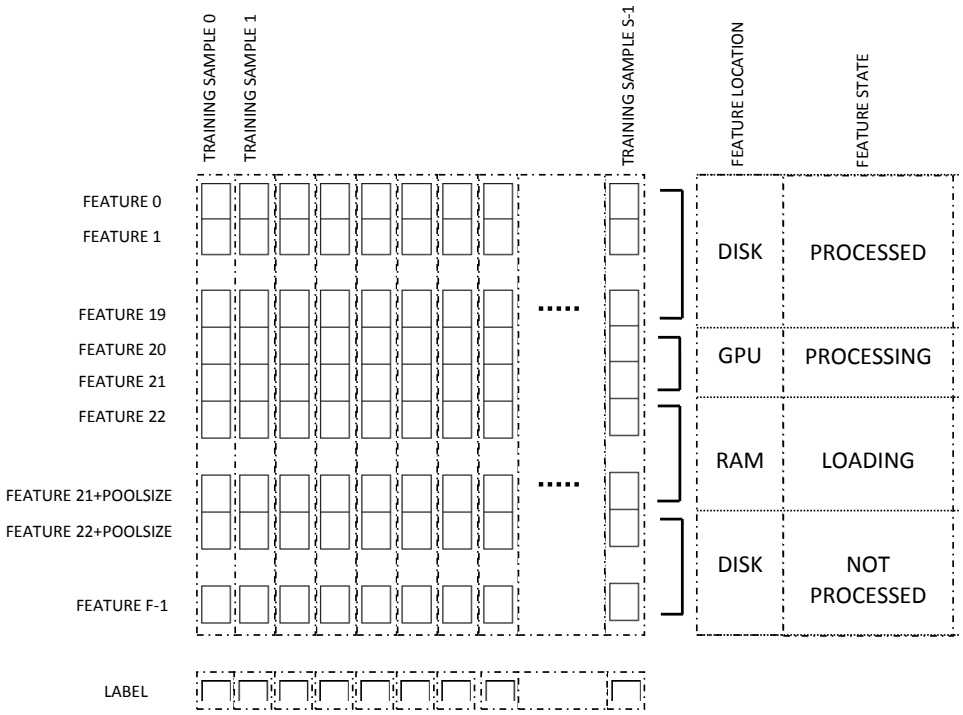


Figure 5.1: Data preloading pattern for multiple features. Every feature resides on disk. A specified number are preloaded before the start of each tree level building round. Then they are processed on the GPU workers. While they are processed the next feature are being loaded and reside in system RAM. These RAM locations are constantly being replaced with new features as the ones being processed on the GPU no longer need to reside in system RAM.

single atrribute at a time thus enabling operation on tera-scale data sets while retaining only a fraction of the data set on the GPU and CPU. It also allows us to distribute different features to different GPUs for multi-GPU parallelism. Off-core attributes are loaded into memory by a background process and then mapped to worker threads. The choice of the number of features to preload into the pool of features available directly in host memory is a techical parameter of our method. Figure 5.1 indicates the state of data during execution of a tree induction round.

To build a tree we start from the root node and build breadth first: we construct each layer of the tree fully before moving to the next layer. Each tree node needs to test all the possible feature and threshold pairs in order to select the best split according to the chosen information theoretic objective gain. This process can be done completely independently for each such pair and so the split tests are mapped to multiple GPUs using a producer-consumer model for work distribution. For every split pair tested at a given tree depth, the best pair is constantly updated on each GPU individually and only when the split tests are finished do all the GPUs send their individual best split pairs back to the main managing process which in turn selects the best overall split pair per node. The results are then pulled back to the host CPU from the worker GPUs, their optimal splits reduced into the global tree model. In a multi-workstation network configuration this process might be communication intensive but for GPUs on a common interface board within a single workstation the data transmission cost is negligible effectively means that speed

Algorithm 5.1 Work dispatch

Start background attribute loader
for Depth $d = 0, \ldots, d_{max}$ **do**
    Send Samples to all Workers via fwd
    while $fID \geq 0$ **do**
        $wID \leftarrow$ get WorkerID token
        $fID \leftarrow$ get FeatureID token
        Launch Worker process
    end while
    Pull best TreeNodes from Workers
    $rev \leftarrow$Sort trajectories with partition step
    Update TreeNodes' $start$,end indices
    Update fwd from rev
    Output current tree
end for

Algorithm 5.2 Worker process

$\delta_d \leftarrow 2^d - 1$
$h_{type} \leftarrow$ Run histogram probe
for Node $j = 0, j \leq 2^d, j \leftarrow j + \delta_{folding}$ **do**
    $j_{start} \leftarrow j + \delta_d$
    $j_{end} \leftarrow j + \delta_{folding} - 1 + \delta_d$
    Build $p^c_{D_{j_{start} \cdots, j_{end}}}$
    loop N_{thresh} times
        $\tau \leftarrow$random value
        Build $p^c_{D^L_{j_{start} \cdots, j_{end}}}$ for (θ_{fID}, τ)
        Compute Equation 5.5
        Update $j_{start} \ldots, j_{end}$
    end loop
    Update Samples' flags
end for

scales according to available worker GPU hardware with minimal impact from tree node update and transmission overhead. Once the best split pairs have been decided for a tree level according to for example Equation (5.6), the data needs to be partitioned according to the splits. So for example when tree level 3 is complete there will be 8 partitions of the data with each partition corresponding to the specific tree node each example has arrived at. Once the data has been correctly partitioned, the whole sequence is then repeated for the next levels of the tree until completion. As an overview to guide understanding, pseudocode for the whole algorithm can be seen in algorithm 5.1.

The process just described is somewhat different to typical decision tree induction which often first sorts and then sequentially searches for the best split point for a given attribute. The reason for this is that we are specifically interested in handling billions of examples at up deep tree levels without incurring exponetial penalties. While parallel radix sorts on the best GPUs can sort billions of keys per second for a single data partition, their efficiency drops dramatically when there are fewer keys to sort. By way of example, an NVidia GTX1080 GPU takes 86 seconds to sort 100 floating point numbers $100k$ times but only 0.019 seconds to sort $10m$ floats once. This implies that for the first few tree levels sorting may be practical but at deeper levels it will quickly become the bottleneck. Furthermore and equally importantly, sorting an attribute on the GPU would also require storing the unsorted index map so that each example can be correctly mapped back into the correct partition after splitting. Doing so would be prohibitively memory intensive as $1b$ examples would require an integer index taking an additional 4 GB on each GPU. The following sections will describe how we solve these and other issues in order to efficiently determine optimal splits and maintain sorted data partitions.

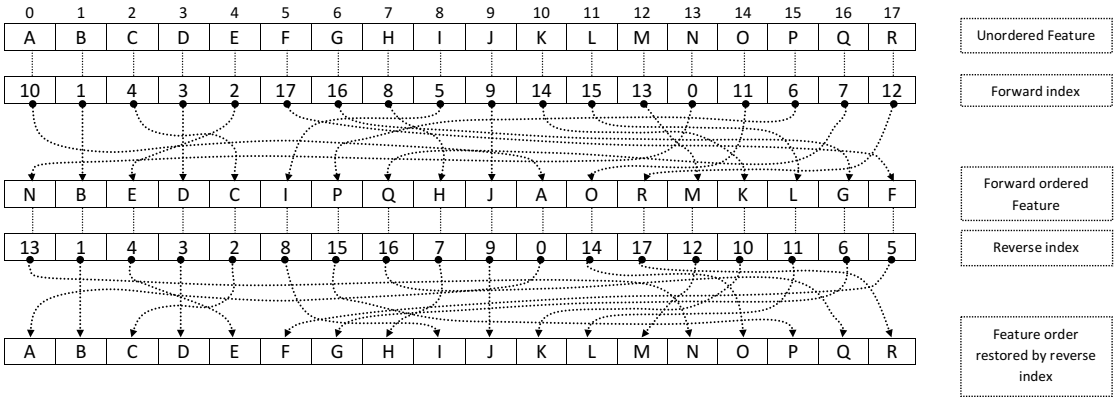


Figure 5.2: Forward-reverse indexing. A feature loaded from disk must be loaded to the GPU in the correct order according to which examples have landed in which node. This figure shows how it is achieved: a single global pair of arrays called forward and reverse arrays. The forward array stores the location to which an example must be mapped once its loaded from disk. This mapping is performed when uploading to the GPU. Once the full tree level has been processed and the samples have been classified into left or right children nodes the reverse and forward arrays are updated with the new mapping.

5.4.1 Data partitioning

The key issue which guides our method is that of data transfer and storage efficiency which is realized by only maintaining a single on-core attribute array on each of the worker GPU devices. This reduces overhead associated with repeated memory transfers and disk access across slow data media. By minimizing host to slave transfers we are effectively removing a major bottleneck which could potentially destroy any other speed improvements. Data to be read or written is located sequentially in memory or at least in close proximity so that caching mechanisms can be activated. A naive implementation would have fully coherent read access but increasingly incoherent write access because each sample maps to a different tree node. To overcome this problem we perform sample reordering after every tree level using a single step binary partition function which results in sequential samples for every data partition associated to a specific tree node. The data structures to support this are a single bit flag indicating last tree level split direction per example and single pair of forward and reverse index arrays on the host device. The associated traversal direction (left or right leaf) is used to indicate which side of the partition for each tree node subset a sample should go. Figure 5.2 indicates how the mapping using forward and reverse index arrays works. When an atrribute file is loaded from disk and then mapped to a GPU worker the mapping process sends the atrribute through the forward index mapping so that the data is stored sequentially by partition on each GPU. This functionality entirely removes the need to store the index map on the GPU. The reverse index can be easily used to undo the mapping direction at a later stage on the host.

Figure 5.3 shows an example of how data is reordered after the best splits are found for every tree level induction round. As can be seen in that figure, by design data ends up stored sequentially per tree node. Although not shown in the figure this means that a simple, cheap binary search easily allows us to

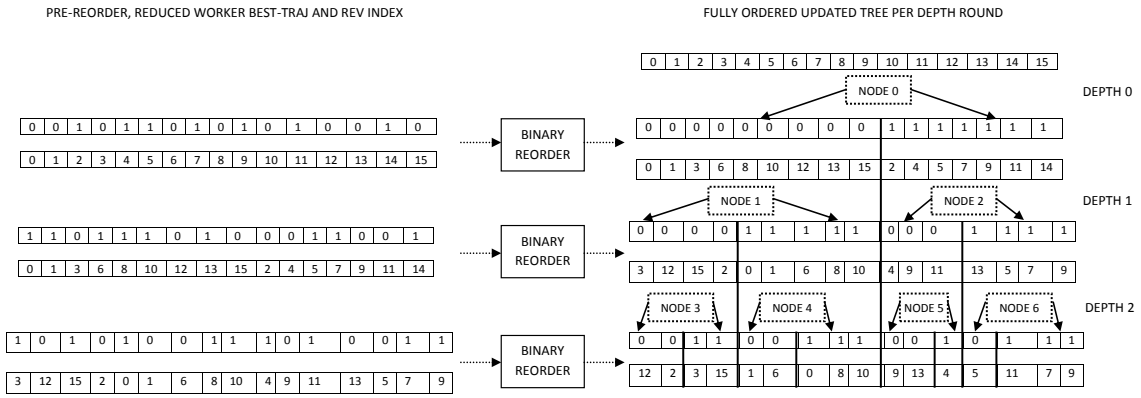


Figure 5.3: Sample reordering using a binary partition pivot after every depth level completion step. **Left:** Indicates the state of the forward array and matching child nodes (0=left, 1=right) before reordering. **Right:** Shows the sequential ordering after performing sample reordering using a binary partition pivot. The node rectangles are shown above the samples which have traversed into them.

map from a data sample index into its specific tree node. In other words the location index of a sample in its sequential list can be used in a binary search in the GPU array which stores the tree node data ranges. To understand why, one should note that each tree node j maintains an index to the start index and end index of its data partition $\mathcal{D}_j$. For very large data sets this enables a factor of two or more saving in GPU memory usage and transfer to and from the host CPU with only a small penalty in runtime. As we will see in the next section on histogram creation, this seemingly simple binary search actually enables us to create the histograms for all the nodes in a tree in a single pass over the data without launching expontially increasing numbers of GPU kernels.

Furthermore, if there are many possible classes and the tree depth is deep then tree data structures can take up too much space as a result of exponentially increasing numbers of tree nodes. The described sequential data partitioning enables splitting of the data passes over subtrees (instead of over all the nodes at a given tree level) at a predefined *folding depth* $\delta_{folding}$. This saves memory and maintains all of the cache coherent characteristics and efficient parallel passes over the data with only a small increased runtime unless the final tree depth is considerably deeper than $\delta_{folding}$.

5.4.2 Histograms for optimal splits

Once a full attribute array has been passed to a worker device a worker CPU process cycles through a fixed number of random thresholds thereby generating a sequence of potential split node parameter pairs. This is inline with how trees in a random forest are typically built. For each such pair (θ, τ) per node we would like to construct 3 histograms: the current histogram (parent) representing the distribution of class labels before splitting and the two child histograms representing the distribution for those examples which go left in the split and those which go right. For efficiency we need only compute the parent histogram once before starting to enumerate the different thresholds. Additionally, computing the histogram for one

of the children allows the other child to be inferred by subtracting from the parent. In order to compute histograms efficiently while take advantage of GPU parallelism we note that there are two different possible ways to construct a histogram. The first and most striaghtforward is to perform a pass over the data and use *atomic-add* write primitives to accumulate data per class in a parallel runtime context (we shall denote this as HIST-ATOMIC). While simple, this has the significant disadvantage in that every *atomic-add* essentially blocks all other parallel execution when simultaneous writes occur at the same histogram bin. The alternative approach is to eliminate all use of *atomic-add* primitives and use a divide and reduce approach where subsections of data are accumulated into separate bins and then all the bins are reduced into one final cumulative bin for each class. This is efficiently implemented in NVidia's *HIST64* and *HIST256* CUDA sample modules [99]. The advantage of these methods is their tremendous speed as a result of full occpuancy of all the GPU cores. Their disadvantage however is that they require extensive memory overhead to run and are limited in how many classes they can handle (up to 64 and 256 respectively). This is the direct inverse of the benefits of *HIST-ATOMIC*: it requires little overhead to run and can operate on virtually unlimited classes but is computationally inefficient. The result is that the only effective way to use the *HIST64* and *HIST256* methods is to run them individually per data partition for each node and update each node histogram individually. In direct contrast, the only way to effectively use the *HIST-ATOMIC* approach is to run it in a single pass over the full reordered data and update all tree level histograms simultaneously in this single pass. The question then immediately arises as to which method is the most appropriate. The answer, perhaps unsurprizingly, is that neither is optimal for every depth level. Instead, at the start of every tree induction level we perform a probe pass where we check which method will be faster. We start by assuming the *HIST64* or *HIST256* (depends how many classes we have) are faster. However if the probe shows that it is more than 10% slower than the *HIST-ATOMIC* method then we switch to that. The results of this probing process can be seen in Figure 5.4. In order to understand why the *HIST64* and *HIST256* methods are exponentially more expensive for the deeper tree levels one only needs to look at the time it takes to launch a GPU kernel. For instance on a GTX580 it takes approximately $7\mu s$ to launch a single kernel. This means that even if the kernel was empty, at tree depth 15 it would take approximately $0.22\,\mathrm{s}$ to launch all the kernels for a single (θ, τ) pair. Thus the use of these specialized histogram kernels is untenable at the deeper tree levels and the histogram generation switches automatically to the *HIST-ATOMIC* method. Interestingly, this method is more efficient at the deeper tree levels because there are far fewer atomic clashes. The end result is a highly efficient histogram construction process which is the core factor in enabling the speed and scalability of our algorithm. It should also be noted that building a per node histogram requires the same type of computation as maintaining running sums used for Gaussian based regression trees: summation and reduction. The method is therefore applicable to both classification and regression trees. After computing

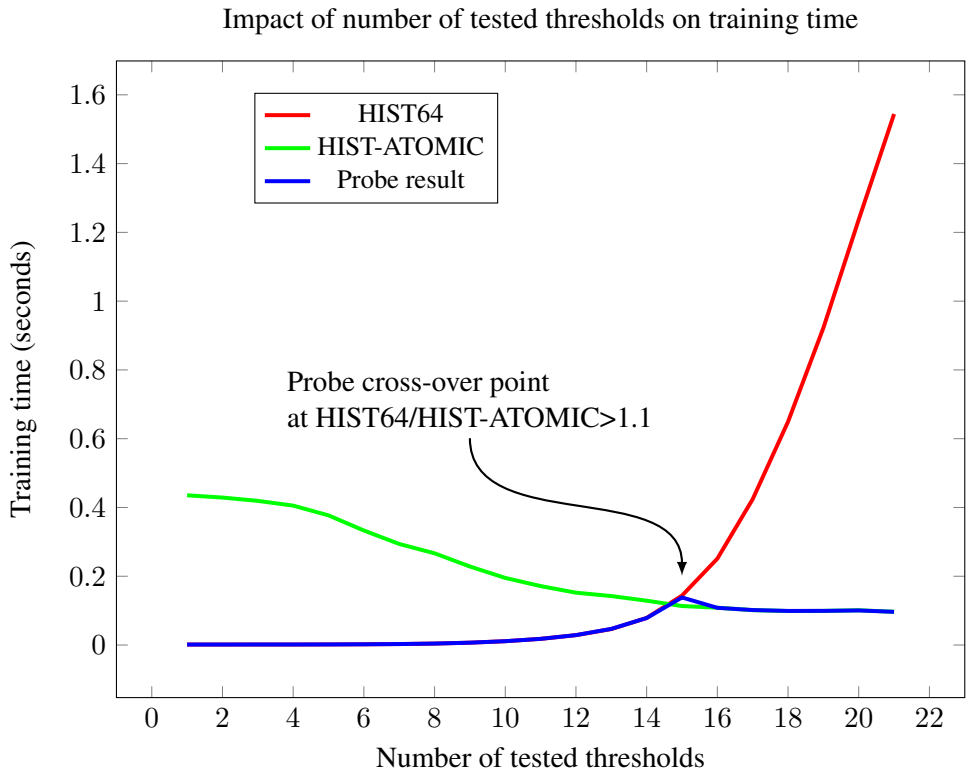


Figure 5.4: The following graph shows the effects on run time of combining GPU kernels. *Red* - Regular histogram builder using atomic add. *Magenta* - Histogram builder using a variant of CUDA's HIST64. *Green* - Speed probe applied. The experiments were conducted on a classification dataset with 100 million samples, 10 numerical attributes of type *INT8* and 8 classes. The run time is the number of seconds for a full execution of a Worker process that tests 5 random thresholds on an NVidia GTX580.

partition statistics induced by (θ, τ), the relevant impurity measure (e.g. equations 5.6,5.7,5.8,5.9) is also computed in parallel on the GPU worker for all tree nodes. The whole GPU worker process including the histogram construction can be seen in algorithm 5.2.

5.5 Experiments

We choose the HandNet [143] dataset to perform our experiments on as the data is publicly available and easy to sample more or less densely to produce different sized datasets. We do a number of different experiments to illustrate the training time behavior. The experiments are conducted using a range of NVidia GPUs with varying hard disk and CPU processor configurations. These can be seen in Table 5.1.

Accuracy versus number of inference decisions. The primary motivation of our proposed training framework is to provide a GPU accelerated, single workstation tree induction pipeline that can train on terascale sized data sets and produce trees for fast inference. Obtaining the lowest number of tree decisions required to reach a prediction while achieving high accuracy for every input sample is the critical factor which can enable realtime computer vision. Unfortunately attempting to perform a direct comparison

Table 5.1: This table shows the system setups for each of the machines that we perform experiments on. The experiments show consistently that using the GTX1080 provides the best speed and scaling behaviour. Note that all these configurations are on separate workstations.

GPU →	K80	K5000M	TitanZ	GTX1080	GTX580
CPU		i7-3940XM CPU @ 3.00GHz	i7-4770 CPU @ 3.40GHz	i7-6900K CPU @ 3.20GHz	Xeon(R) CPU E5645 @ 2.40GHz
CPU RAM (GB)	64	32	16	32	49
GPUs (#)	2	1	2	3	4
GPU RAM (GB)	11.4	4	6	8	1.5
GPU max power (W)	149	149	189	180	244
Cuda cores/GPU (#)	2496	1366	2280	2560	512
Core speed (MHz)	1760	706	875	1607	1544
Disk	HDD	SSD	HDD	HDD	SSD
Disk read (MB/s)	166	440	160	173	564
Disk write (MB/s)	178	391	145	141	105
Disk cache (GB/s)	5	9.2	9	9	7.5

to other available GPU based tree induction pipelines is challenging. For example CudaTree [86] is not suitable for comparison as it requires the entire dataset to be held in GPU memory which severely limits the number of samples which can be used. A relatively small dataset (relative to what we can handle) of $20m$ samples with 300 features stored as *int8* format would require $20m * 300 + 2 * 20m * 300 * ceil(log2(20m)/8) + 20m * 300 = 601$ GB of GPU memory for CudaTree whereas ours requires only 2301 MB to build a tree to depth 20. Similarly, while GPU accelerated, CudaRF [58] and Curfil [123] do not appear to be able to handle more than a few hundreds of thousands of samples. Instead, we compare to a number of different training configurations in LightGBM [95] which is a powerful GPU accelerated gradient boosting tree induction framework. LightGBM can train trees to different depths and chain them in a boosted gradient framework. This gives us a useful way to compare both the accuracy and capabilities of LightGBM versus our method despite their training objectives being different.

Table 5.2: This table shows the different experiment configurations we used for comparing the number of tree decisions required for our method versus those of LightGBM. The behavior of each setup is shown with different training set sizes. Apart from our method required significantly fewer tree decisions, we were also able to train on more data on each workstation. This is unlike LightGBM which requires both more decisions and could not train on the large $500m$ sample dataset because it could not be stored on-core.

Method →	Ours				LightGBM			
Samples, features ↓								
500m, 1000	80.4	81.2	82	82.8				
500m, 300	77.3	78	78.6	79.3				
100m, 1000	79.8	80.5	80.9	81.3				
100m, 300	77	77.5	77.8	78.1	69.6	78	75.6	79.8
20m, 2000	79.9	80.2	80.4	80.3	70.3	78.3	76.7	81.1
20m, 1000	78.2	78.3	78.2	77.9	69.6	76.5	75.2	79
20m, 300	75.6	75.7	75.5	75.3	69.5	74	73	75.5
Depth →	17	18	19	20	3	10	22	22
Leaves →	131k	262k	524k	1m	112	14k	1.8k	36k
Decisions →	17	18	19	20	28	140	140	2800

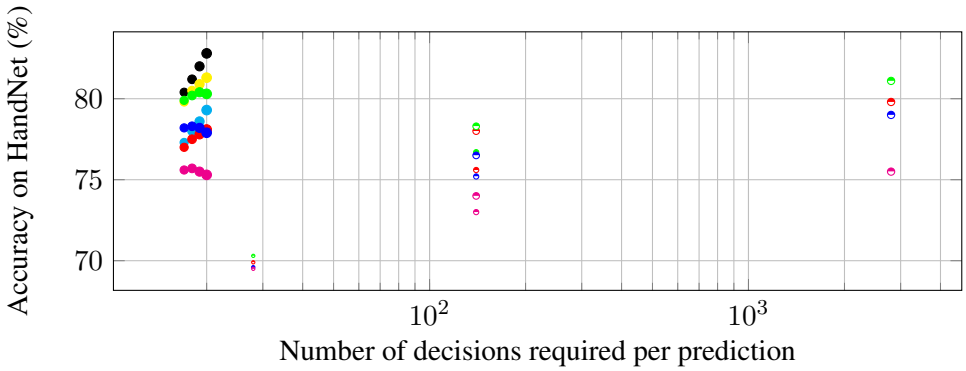


Figure 5.5: This plot shows our method compared to LightGBM for different training set sizes and decision numbers. Each circle represents a different experiment. Circles with the same color were trained on the same data. The specific experiment data can be seen in table 5.2. A solid color indicates an experiment using the classification tree from this work. A half circle indicates training using LightGBM gradient boosted trees. The area of the circle is proportional to the total number of leaves used in each method. Our method uses considerably fewer decisions while being capable of training on more data thus yielding higher accuracy.

Table 5.2 (which is visually displayed in Figure 5.5) shows the experiments we performed. We were not able to run any experiments with LightGBM in which the dataset occupied more than the memory available on the host CPU which limited the maximum number of samples and features we could attempt. The experiments illustrate that while LightGBM can obtain better accuracy than our method when trained on the same data, our method is capable of handling significantly more data in a single workstation environment while also preserving extremely low decisions required per prediction. The reason for this is that despite the success of gradient boosting methods, their inherrent boosting property requires multiple boosting iterations to allow the method to work properly and also requires multiple trees for multi-class classification.

Data types. The type of data that is used for storing the attributes has a significant impact on training time. Figure 5.6 shows that the runtime execution increases more or less linearly according to the number of bytes in the chosen datatype. Unsurprizingly this means that using *int8* data to store attributes is the optimal setting. Various kinds of quantization can be used to compress single precision data down to *int8* and categorical data with up to 256 categories can also be stored effectively at this data type size.

Number of thresholds. For each tree node our method passes over all available features and generates a sequence of random thresholds for optimal split testing. We show in Figure 5.7 that training time increases in almost direct proportion with the number of thresholds. Interestingly however the training time is not drastically impacted for very few thresholds up to around 10. The reason for this is that other factors such as data transmission and other overheads are considerably more time consuming. It is possible in some training scenarios that 10 threshold tests would be sufficient beyond which there are few improvements in accuracy.

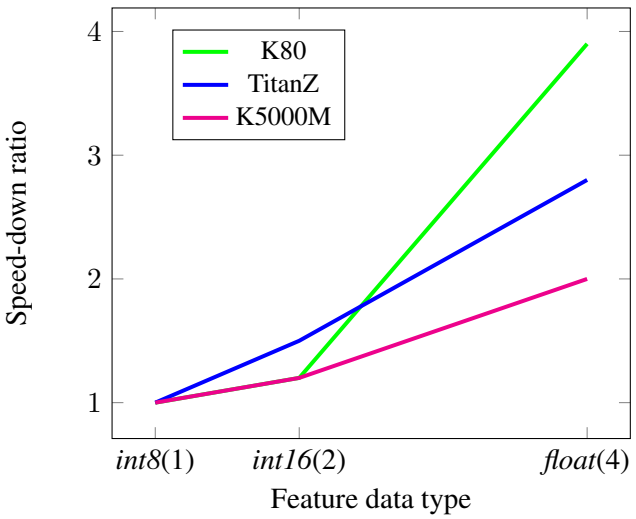


Figure 5.6: This graph shows the impact in training time on using different datatypes (number of bytes) on different GPUs relative to using *int8*. This is important because it shows how using more quantized data can lead to significant improvements in speed. Params: $100m$ samples, 100 features, 7 classes, 20 preloaded, 10 thresholds, 17 depth levels.

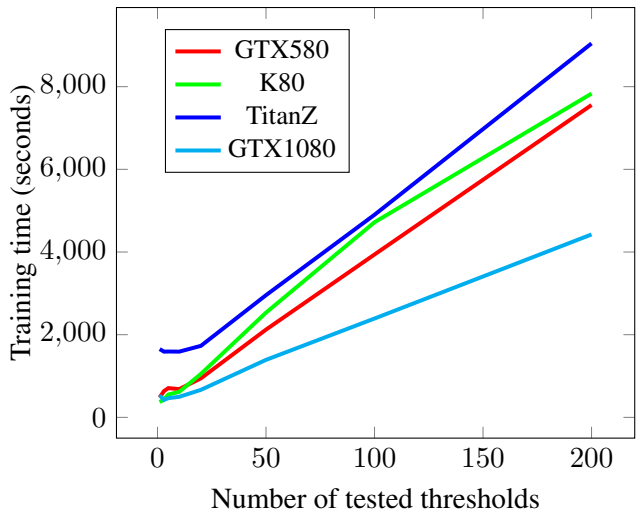


Figure 5.7: Experiments showing impact of different numbers of testing thresholds for different GPUs. The initial region for up to 20 thresholds is more less dominated by other factors. Beyond 20 thresholds the increase in time complexity is linear. Params: $100m$ samples, 200 features, 7 classes, 20 preloaded, 15 depth levels.

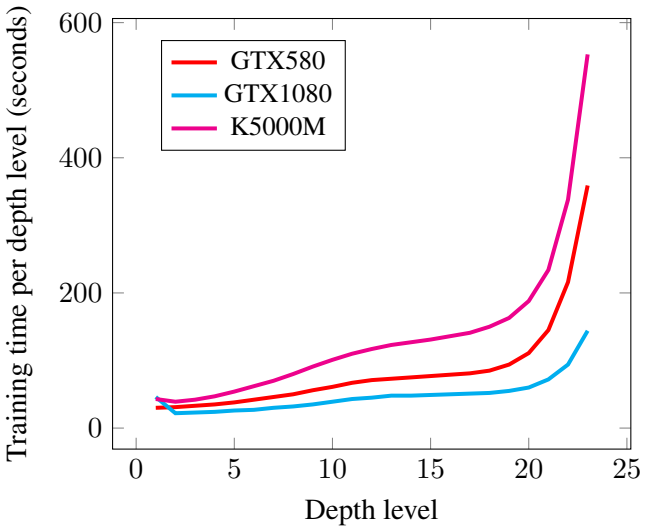


Figure 5.8: Experiments showing the different runtimes for each level of tree construction on different GPUs up to depth level 23. Params: $100m$ samples, 200 features, 7 classes, 20 preloaded, 10 thresholds.

Training time per level. The time taken to train each individual tree level is an important consideration when deciding how deep to train a tree. The deeper levels typically give less return on investment of training time and thus for fast iterative research it is often sufficient to train to shallower levels until the final model needs to be produced. Figure 5.8 illustrates how long training each depth level takes. Exponential increases in training times start occuring for all GPUs after around depth 20 because the size of the tree structure itself becomes very large very quickly and begins to incur exponential data transfer time penalties.

Number of GPUs. An important aspect of our framework is the fact that it can scale to multiple GPUs on a single workstation. It can be seen in Figure 5.9 that for the somewhat older cards the speed up is signifcant but not 1-to-1 whereas for the GTX1080 the training time speed-up is almost perfectly matched to the number of GPUs used. It is likely that for future cards the ratio will continue to hold which implies that even more cards could bring an even more significant speedup.

Preloading data. It was initially expected that enabling preloaded attributes to reside on host memory would hide data latency from the disk. However as Figure 5.10 illustrates, this is not the case. We discovered that our training procedure is mostly memory bound as opposed to computationally bound and thus tasks are consumed faster than they are produced which yields a bottleneck. The only truly effective way to eliminate this bottleneck is to fully preload all the available attributes to the host memory in advance. While this can yield dramatic speed improvements it is impossible for datasets which are larger than available host memory which is precisely the reason that we implemented the preloading mechanism: to enable out-of-core training.

Number of classes. The number of classes in a multi-class dataset effects the training time in a non-trivial fashion as a result of using the different histogram construction kernels mentioned in Section 5.4.2.

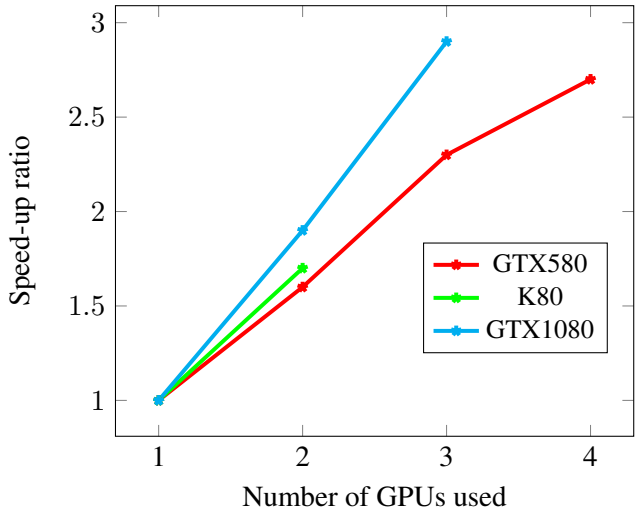


Figure 5.9: This figure shows the training time speed-up which is achievable by using multiple GPUs. Different GPUs have different behaviors however the GTX1080 displays nearly perfect parallelism when adding more cards. This illustrates that using the right hardware training time improvements scale linearly. Params: $100m$ samples, 200 features, 7 classes, 20 preloaded, 10 thresholds, 21 depth levels.

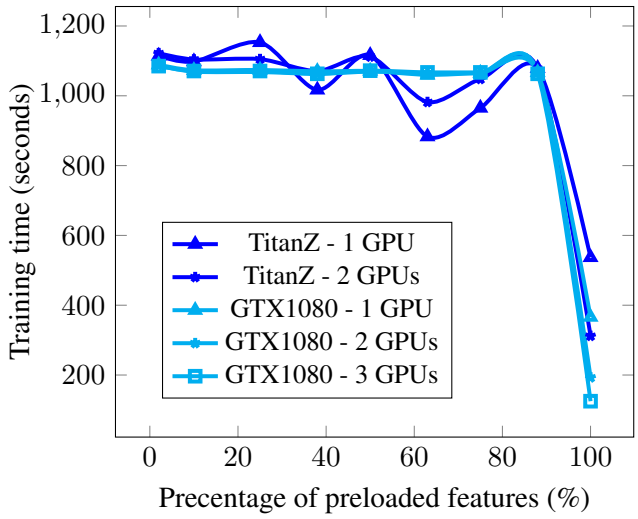


Figure 5.10: Experiments on different number of GPUs showing impact of ratio of preloaded features on training time. Apart from fully loading all the training data, preloadeding features has virtually no impact. Params: $54m$ samples, 200 features, 7 classes, 17 depth levels, 10 thresholds.

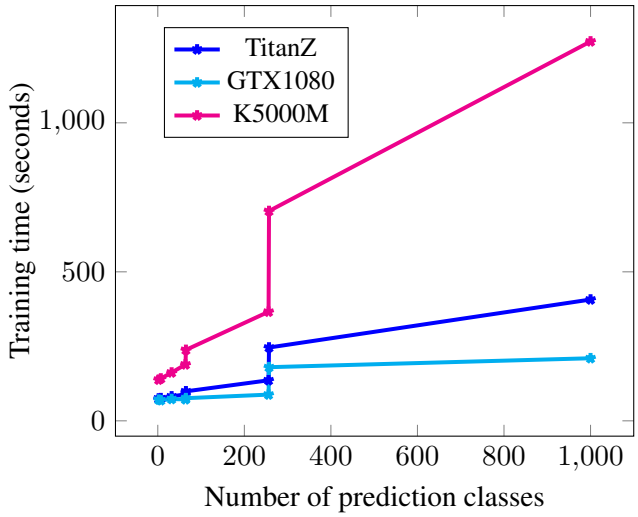


Figure 5.11: This plot shows the impact of the number of classes used on training time. The two mechanisms by which the number of classe have impact are histogram kernel method and memory use. The plot shows that there is a gradual increase in complexity for each kernel type. The kernel types are *HIST64* for $1-64$ classes, *HIST256* for $65-256$ classes, *HIST-ATOMIC* for more than 256 classes. Jumps in runtime can be seen for each crossover point in these ranges. Params: $100m$ samples, 20 features, 20 preloaded, 10 thresholds, 19 depth levels.

Figure 5.11 shows this behavior clearly. The apparent steps in the timing graph occur at 65 classes and 257 classes as these are when HIST256 is selected over HIST64 and HIST-ATOMIC is selected over HIST256 respectively.

5.6 Runtime inference

While the main focus of this work is on performing tree induction it is instructive to note how the output of our method can be used for inference. Performing inference on the CPU is relatively straightforward as it simply requires traversing the tree or forest based on the tree nodes. One of the msot significant advantages of tree based prediction models is that for axis aligned splits one only needs to compute a set of features, the size of which is proportional to the depth of the tree. This is very efficient within a computer vision context as it means no unecessary features are computed. Performing inference on the GPU is also possible and we refer the reader to [123, 125] for good examples of ways to do this. Instead of focusing on these, we describe here our approach for performing inference using a tree produced by our induction algorithm on a readily available, small, commodity FPGA hardware platform.

5.6.1 FPGA pixel flow

Performing FPGA based inference efficiently is the first step in enabling a power efficient and fast hardware solution for executing randomized trees on low power mobile devices. This kind of context for prediction is becoming more ubiquitous with the continued rise of mobile computing and photography and

the need for real-time computer vision. While there are other methods for performing tree inference on FPGAs [100, 132, 133] we describe our own approach for implementing on an FPGA realtime semantic segmentation using a single 21-level deep tree trained on the 7-class HandNet dataset. We developed our inference algorithm using Xilinx VIVADO and emulated on the Ztex 2.13d [1] evaluation board which has a mounted Xilinx Artix 7 FPGA XC7A100T. Initially, the logic design was developed using Vivado HLS toolbox but we found it to be insufficient in terms of cycle optimization, therefore the implementation was redesigned using System Verilog. The initialization stage of the FPGA includes the transfer of the decision tree to the DDR3 SDRAM on board the Ztex card and initializing several 128×128 BRAM blocks to be ready to store pixel values and the decision tree branch addresses. At runtime a new image is transferred via USB to the card. Once a full input image has been stored the FPGA iterates over each pixel in a loop that starts at the root of the tree. It fetches the required feature type and decision threshold and then computes the feature by fetching values from the DDR. To do this the FPGA needs to read the values from the DDR by making a request to the DRAM controller. Since fetching data from the DRAM access requires an order of magnitude more clock cycles than local BRAM access each pixel loop is asynchornous with every other and the FPGA continues to the next pixel while the DRAM controller processes the memory request. The loop state of every pixel is stored on the BRAM which allows the FPGA to execute a fully pipelined, asynchronous approach in order to increase pixel throughput and decrease idle time while waiting for the data to return from the DRAM. The FPGA repeats this process for every pixel until all pixels have reached their final leaf. Once all leaves have been encountered the FPGA sends a request to load all the relevant histograms from the DRAM and performs the final pixel predictions. Since different pixels are required in each branch calculation, three 128×128 BRAM blocks are used for pixel storage to allow parallel access. Two 24×16384 BRAM blocks were used to store the branch information which was crucial in order to achieve a one cycle throughput of the pipeline as each in-flight pixel is potentially at a different tree depth. The latency of the image processing was tested using a Java program which connected to the Ztex board and transferred a stream of 128×128 images. Using this method we determines that the average time from the start of the transfer of the first bit of the image until the last result bit was returned by the FPGA was $12\,\mathrm{ms}$ with a $40\,\mathrm{MHz}$ system clock and the total number of clock cycles of the FPGA system clock was 481927 cycles. This ammounts to a total framerate of 83 frames per second which is considered to be well above real-time (30 fps). Of course it does not include various other system elements such as preprocessing, postprocessing and capture time and there are numerous optimizations which are still available. However it indicates that the approach described can be an effective baseline for implementing inference using the randomized decision trees produced by our induction method as well as potentially trees generated by other training processes.

[1] `https://www.ztex.de/usb-fpga-2/usb-fpga-2.13.e.html`

5.7 Conclusions

In this chapter we have described a novel approach to training randomized decision trees on individual workstations with one or more GPUs to help fully parallize the training process. By doing so we are essentially replacing the need to use computing core clusters in a distributed network with the very dense computing capabilities available on modern GPUs. Our approach is designed to enable datasets which cannot be held in main memory of either the host machine or the GPUs themselves. This guiding principle is the key enabler to our approach. The training process as well as the FPGA based inference pipeline we describe can be modified to train/run a random forest by accumulating histograms over each tree. In this case one would typically use many shallow trees or only a few deep trees. However in our experience, unitary deep trees which are trained on hundreds of millions or billions of examples and thousands of features are often able to perform similarly to multiple trees trained on less data to lower depths as shown for example in Figure 5.5. Ensembles increase runtime linearly according to the number of trees in the ensemble and thus the only way to avoid increasing runtimes is to collect and train more data. This is precisely the scenario which our tree induction algorithm is trying to serve by enabling training on terascale datasets. To enable future development, we have made code for both training and inference publicly available[2]. In conclusion, we believe that using the approach described here researchers and practitioners potentially have a new tool which can enable iterative research and applicative utility that was not previously possible.

[2] github.com/twerdster/tea.git

Chapter 6

Fingertip detection and identification

We discuss various aspects along the computational pipeline of hand detection and processing. Our primary investigation is focused on a novel global orientation regression approach for articulated objects using a deep convolutional neural network. This is integrated with an in-plane image derotation scheme which we call DeROT, to tackle the problem of per-frame fingertip detection in depth images. The method reduces the complexity of learning in the space of articulated poses which is demonstrated by using two distinct state-of-the-art learning based hand pose estimation methods applied to fingertip detection. Significant classification improvements are shown over the baseline implementation. Our framework involves no tracking, kinematic constraints or explicit prior model of the articulated object in hand. To support our approach we describe a new pipeline for high accuracy magnetic annotation and labeling of objects imaged by a depth camera. We also briefly cover one possible approach for using the results of the fingertip pipeline in a hand model and pose matching process.

6.1 Introduction

In this chapter we primarily focus on a method for normalizing out the effects of rotation on highly articulated motion of deforming geometric surfaces such as hands observed by a depth camera. Changing the global rotation of an object directly increases the variation in appearance of the object parts. The work of [74] physically removes this variability with a wristworn camera and samples only a single 3D point on each finger to perform full hand pose estimation. For markerless situations, removing variability through partial canonization can significantly reduce the space of possible images used for pose learning instead of trying to explicitly learn the variability through data augmentation. In [82] the authors show that learning a derotated 2D patch instead of the original one around a feature point dramatically reduces the learning capacity required and improves the classification results while using fewer randomized trees. To develop our method we use fingertip detection as a challenging representative

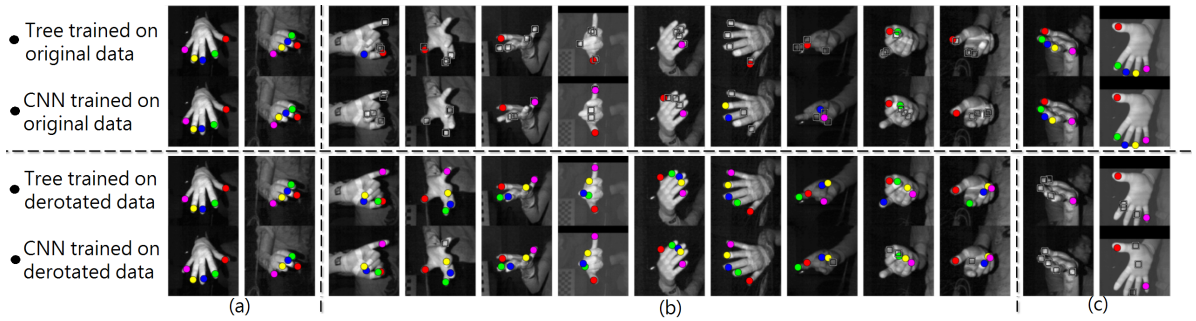


Figure 6.1: Examples from HandNet test set detections. The colors represent fingertips that are correctly located and identified. The white boxes indicate false detections with the error threshold chosen to be 1 cm. **Top two rows:** Trained and tested on non-derotated data. **Bottom two:** Trained and tested on derotated data and then rotated back to the non-derotated space. The detections are overlaid on the IR image from the camera which is not part of the classification process. a) Successful examples for all methods. b) Representative challenging examples for which derotation enables better performance. c) Failure cases where derotation fails to improve the results.

scenario with a propensity for self occlusion and high rotational variability relative to an imaging sensor. Many approaches in the literature use fingertip or hand part detection towards the goal of full hand pose (*e.g.*let@tokeneonedot[73],[25],[136],[140]) however, they all approach the problem by trying to learn on datasets by augmenting rotational variability. Instead, we propose to remove this hand space variability during both the training phase and run-time. To this end we propose to learn the rotation using a deep convolutional neural network (CNN) in a regression context based on a network similar to that of [136]. We show how this can be used to predict full three degrees of freedom (DOF) orientation information on a database of hand images captured by a depth sensor. We combine the predicted orientation with a novel in-plane derotation scheme. The "Rule of thumb" is derived from the following insight: there is almost always an in-plane rotation which can be applied to an image of the hand which forces the base of the thumb to be on the right side of the image. This implies that the ambiguity inherrent in rotationally variant features can be overcome by derotating the hand image to a canonical pose instead of augmenting a dataset with all variations of the rotational degrees of freedom as is commonly done. Figure 6.1 shows examples of extensive pose variation that can benefit from our approach [1].

No currently available hand datasets (*e.g.*let@tokeneonedot[156],[25],[136]) include accurate full 3 DOF ground truth hand orientations on a large database of real depth images. Using joint location data from NYUHands [136] it is possible to extract a global hand orientation per pose. However, we found that the size of this dataset and rotational variability are not optimal for learning to predict 3 DOF orientation. A significant contribution of this paper is therefore the creation of a new, large-scale database of fully annotated depth images with 212928 unique hand poses captured by an Intel RealSense camera that we call HandNet[2]. For the purpose of effectively annotating such a large dataset we describe a novel

[1] All graphs and images in this paper are best viewed in color.

[2] To advance research in the field this database and relevant code is available at `www.cs.technion.ac.il/~twerd/`

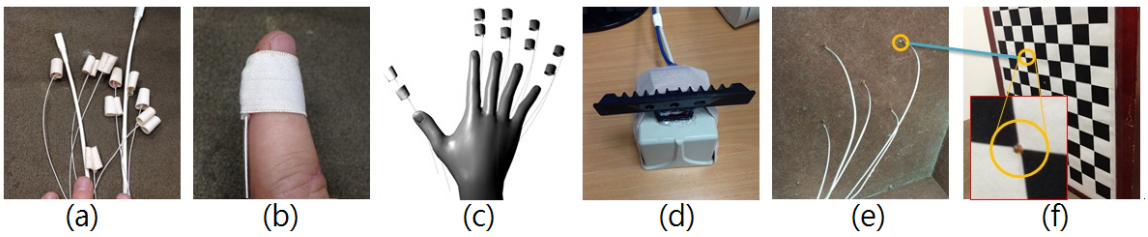


Figure 6.2: The data capture setup. a) 2mm magnetic sensors. The larger rectangular sensors are not used. b) A fingertip sensor inside the inner seam. c) Virtual model used for planning a multi-sensor setup. We only use 5 sensors. d) The RealSense camera rigidly fixed to the TrakStar transmitter. e) The back of the wooden calibration board where the glass sensor housings are firmly pushed through. f) The front of the calibration board where the glass sensor housings are visible on the corners as seen in the inset.

image annotation technique. To overcome the severe occlusion inherrent in such a process we use DC magnetic trackers which are surprizingly sparsely used by the vision community considering their high accuracy, speed and robustness to occlusions. Using our deep derotation method (DeROT) we show up to 20.5% improvement in mean average precision (mAP) over our baseline results for two state-of-the-art approaches for fingertip detection in depth images, namely, a random decision tree [73] (RDT) and a deep convolutional neural network [136] (CNN). We compare our results to a non-learning based method similar to PCA and show that it produces inferior results, further supporting the proposed use of DeROT. Using the results of the fingertip detector we explore one avenue for generating hand pose and matching a hand mesh to an input depth frame.

The following sections will describe the processes we arrived at and we will start by covering the most challenging practical aspect: collecting and annotating data.

6.2 Building HandNet: Creation and annotation

Synthetic databases such as those created using [139] have a severe disadvantage in that they cannot accurately account for natural hand motion, occlusions and noise characteristics of real depth cameras. The creation of a large hand pose database of real depth images with consistent annotations is therefore of great importance, but beyond the capability of human annotators. The NYUHands database [136] uses a full model of the hand and a three-camera setup to annotate hand joint locations. There are instances where fingers are obstructed and accurate orientation information is not reliable. Similarly the method of [140] uses inverse kinematics coupled with a colored glove which also has the disadvantage of not having explicitly measured orientation as well as fingertip locations which are obstructed from the depth camera. An alternative to model based systems are sparse marker systems such as those used by [156], however, the excessive cost of a modern mocap setup such as Vicon as well as the occlusion problem make such an approach unattractive. In contrast, modern DC magnetic trackers like the TrakStar [1] are robust to

HandNet/

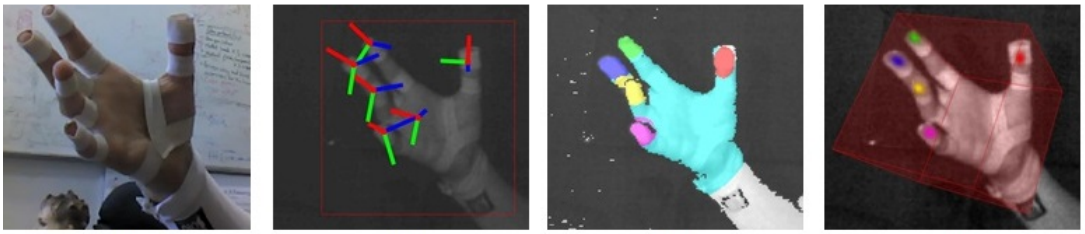


Figure 6.3: The available data annotations after calibration. a) Color image. Illustrates a full hand setup for this work. The color is not used. b) The RGB axes indicate the measured location and orientation of each fingertip and the back of the palm. c) IR image(not used) overlaid with the labels generated from the raycasting described in Section 6.2. d) IR image overlaid with the generated heatmaps per fingertip and the global orientation of the hand represented as an oriented bounding box (not used).

metallic interference and obstruction by non-ferrous metals, and provide sub-millimeter and sub-degree accuracy for location and orientation relative to a fixed based station. Despite their almost non-existent use in modern computer vision literature, we have found them to be an excellent measurement and annotation tool.

Sensors. To build and annotate our HandNet database we use a RealSense camera combined with $2mm$ TrakStar magnetic trackers. We affix the sensors to a user's hand and fingertips by using tight elastic loops with sensors in sewn seam pockets. This prevents lateral and medial movement along the finger. This can be seen in Figure 6.2. The skin tight elastic loops have an additional significant benefit over gloves in that the depth profile and hand movements are not affected by the attached sensors and thus do not pollute the data.

Callibration. Camera callibration with known correspondences is a well studied problem [155]. However, in our case we need to callibrate between a camera and a sensor frame. We do this by positioning the magnetic sensors on the corners of a checkerboard pattern thereby creating physical correspondence between the detected corner locations and the actual sensors. This setup can be seen in Figure 6.2. We use the extracted 2D locations of the corner points on the callibration board [12] together with the sampled sensor 3D locations to perform EPnP [83] to determine the extrinsic configuration between the devices.

Annotation. We model each sensor as a 3D oriented ellipsoid. We then raycast the ellipsoid into the camera frame and set the label to be the identity of the ellipsoid closest to the camera for every pixel. We also create a heatmap h_i for each fingertip i using the same technique but setting the value per pixel to be Gaussian over the distance to the projected sensor location. An example of both types of annotation can be seen in Figure 6.3.

Recording the database. The database is created from 10 participants (half male, half female, different hand sizes) who perform random hand motions with extensive pose variation while wearing the magnetic sensors. The RealSense camera operates at 58fps producing 640×480 depth maps which

we reduce to 320×240. The TrakStar samples measurements at a rate of 720Hz. In total we recorded 256987 images. A portion of these images were removed because of low quality. The final dataset is 212928 frontal images including full annotation of the position and orientation of each fingertip and the back of the palm. After recording each participant we used a software utility to add offsets to the rotation and location of each sensor to adjust for greater consistency in positioning across subjects.

6.3 Hand localization

While large ammounts of data are crucial for a succesful fingertip detection pipeline we will also need a method to find the location of a user's hand in a frame. We would like to enable the possibility of robust per frame hand detection, stable tracking and robust re-initialization on tracking failure. Furthermore, the method needs to have a minimal computational requirement so as to not be a bottleneck of the system. The large database of hands in different situations provides sufficiently rich data for building a supervised learning problem in which our input examples are hand pixels with a binary label i.e. hand or not hand. For each pixel we need to generate attributes that are potentially useful and then train our detector based on these features. In their work on keypoint recognition Lepetit *et al*let@tokeneonedot [82] and Calonder *et al*let@tokeneonedot [19] suggest taking the difference between random paired locations in a fixed size window around each sample pixel. These seemingly weak and arbitrary features proved to be reliable and demonstrated robustness to different environmental lighting conditions. Shotton *et al*let@tokeneonedot [127] adapted the features to be depth invariant by taking advantage of image depth data. An attribute vector with K features for any given pixel is therefore defined as being $y = (y_1, y_2 \dots y_K)$ such that

$$y_k = z\left(q + \frac{o_{k,1}}{z\left(q\right)}\right) - z\left(q + \frac{o_{k,2}}{z\left(q\right)}\right) \tag{6.1}$$

where $q \in \mathbb{N}^2$ is an image coordinate. $z : \mathbb{N}^2 \rightarrow \mathbb{R}$ is the scene depth map supplied by the camera and $o_{k,[1:2]} \in \mathbb{N}^2$ are the two offsets defining the k_{th} attribute. Instead of taking uniformly distributed random pair locations we instead sample the set of first coordinates from close to the center of the window and the second coordinates at increasing gaps from the center. This pattern of sensors can be seen in the retina of animals and a similar distribution of features was recently demonstrated in FREAK [3] to provide robust features for state of the art keypoint matching. The sampling pattern can be seen in Figure 6.4. Attributes with their binary labels (hand/not hand) are then extracted from examples in the training set and passed to the randomized tree builder described in the previous chapter. In our experiments we found that a single shallow tree was sufficient to provide reasonable estimates of which pixels belong to hands. This is a somewhat surprising result considering the size of the database and the rich variety of possible hand appearances. However on further inspection it is clear that the vast majority of hand poses involve

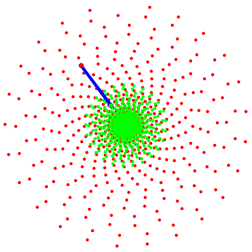


Figure 6.4: Normalized coordinates of feature pair locations. Each red location $o_{k,1}$ is paired with a green location $o_{k,2}$ and the after scaling the difference from Equation 6.1 defines the k_{th} attribute value. The blue line indicates an example pairing.

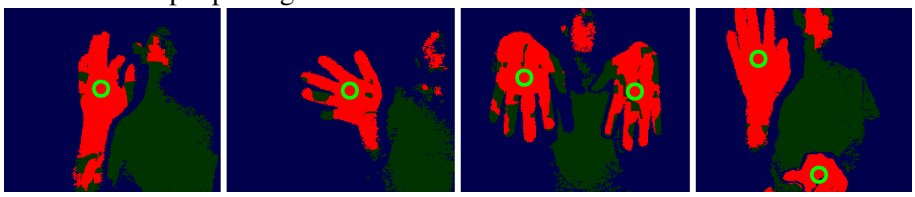


Figure 6.5: Examples of hand detections in a raw depth stream. For sequential tracking of individual blobs we use a simple motion model and a Kalman filter.

a blob of pixels in front of a separate background. It is likely that this blob is what is currently being identified by the tree ensemble. In principle lens distortion could be accounted for in the training database but we have not found this necessary. We use the intrinsic camera matrix $K_{cam} \in \mathbb{R}^{3\times3}$ to unproject each detected hand pixel coordinate q to its camera space coordinate $v \in \mathbb{R}^3$ such that

$$v = K_{cam}^{-1}(q_0, q_1, 1)^T z\,(q)\,. \tag{6.2}$$

We collect these as a point cloud $\mathcal{V} = \{v_{i=1:V}\}$ with V points and process $\mathcal{V}$ with a kernel density estimator. By doing so we are implicitly treating $\mathcal{V}$ as comprising a random sample of an underlying probability distribution of which we would like to find the modes. We apply the meanshift algorithm [56] to extract the modes with a natural bandwidth set to be the average width of a hand. Every sufficiently supported mode is considered a hand candidate. An example set of detections and their relevant segmentations can be seen in Figure 6.5. It is important to note that this system is reliable enough to operate without adding a tracking framework but that in order to improve stability a first or second order Kalman filter should be added to ensure that erroneous trajectories are avoided.

6.4 Fingertip detection

Although there are many non-learning based hand pose methods that can produce fingertip locations (*e.g.*let@tokeneonedot[121, 101, 93, 8]), they use kinematic and frame to frame constraints coupled with hand modelling. In contrast, here we specifically focus on per frame fingertip detection in depth images without either tracking or kinematic modelling. For our pipeline we first segment the target hand from the

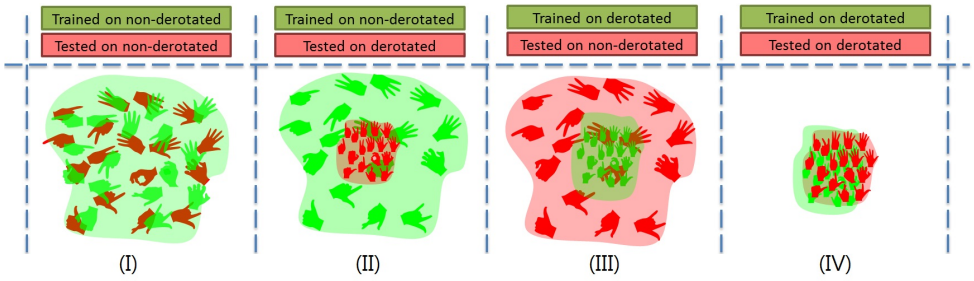


Figure 6.6: Understanding derotation: We represent the space of poses by a non-uniform 2D region with representative hand poses. Red and green represent the pose-space covered by training images and testing images respectively. Each I,II,III,IV indicates one of the 4 possible combinations of training and testing for a machine learning method where the database remains fixed in size. The larger region indicates greater pose variability while the smaller represents less. Intuitively, by training on a space with low variance and testing in this same space (type IV) we expect to see an improvement over the opposite (type I). Section 6.6.1 supports this intuition.

depth image using a fast depth based flood-fill method seeded either from the previous frame for real-time use and testing or from the ground truth hand location for building the database. Using the center of mass (CoM) of the segmented hand and its average depth value we define a depth dependent bounding box of size $w = \frac{50000}{z}$ for a RealSense camera (HandNet) and $w = \frac{70000}{z}$ for a Kinect camera (NYUHands) where z is the depth of the CoM of the segmented hand. We derotate the image about the CoM using an angle of rotation according to the in-plane angle produced by DeROT described in Section 6.5. This comes from the predicted full 3D orientation at run-time or from the ground truth sensor orientation for database construction or testing. We then crop the image using the bounding box. We now describe our modifications of the two different, learning-based fingertip detectors that we use in this work.

6.4.1 Random decision tree

We follow the method of Keskin *et al*let@tokeneonedot[73] where a random decision tree (RDT) ensemble learns hand part labels for every pixel in a depth image of a hand. We refer the reader to the supplementary material of our paper as well as [73, 127] for specific details of this approach. However, here we propose a number of key differences which we found specifically helpful for fingertip detection and run-time efficiency. We use the same random binary depth attributes per pixel but spatially distribute them according to an exponential sampling pattern similar to that of BRISK [131]. In addition to this, we use only a single RDT which contrasts with the common use of multiple trees in an ensemble. After training our single RDT the class distributions stored at each leaf can be used for inference because they represent the empirical estimate of the posterior probability $p\left(c|x\right)$ of hand part label c given the image evidence x. Inferring the most likely fingertip identity label is therefore simply performed pixel-wise by finding the c^* which maximizes $p\left(c|x\right)$ per pixel. However, label inference performed this way results in noisy labels as neighboring classifications do not influence one another. Without adding more trees we propose a simple

but highly effective spatial regularization: for each fingertip i we treat the posterior $p\left(c=i|x\right)$ for all pixels as an image and convolve it with a discrete 2D Gaussian smoothing kernel g_σ with blur radius σ. This has the effect of correlating the posterior label distributions of nearby pixels. Therefore every pixel q is labeled by fingertip identity (including palm and wrist labels) according to

$$c^*\left(q;x\right) = \underset{i\in\{0..6\}}{\arg\max} \left(g_\sigma * p_{c=i|x}\right)\left(q\right). \tag{6.3}$$

Finally, we found that the close proximity of fingers compromises standard mean-shift [32] clustering. Instead we detect the largest label blobs in the label image from Equation (6.3) above a certain area threshold. The 2D fingertip locations are then assigned to the blob centers and, if necessary, the average depth value for each blob can be used to generate the 3D camera-space coordinates.

Training the RDT. Training optimal decision trees is known to be NP-complete [67] and therefore trees are built from the root down using breadth-first greedy optimization over tree node impurity. We use the Gini impurity measure which is slightly cheaper to compute than the more typical entropy measure. To build our database for training an RDT we extracted 80% of the fingertip pixels in our training datasets and 50% of the non-fingertip hand pixels. For HandNet this results in a training dataset of 500 million sample pixels totaling 600GB of data for 1200 attributes. Our tree-builder trains an unpruned randomized tree on $4\times$ GTX 580 GPUs and an Intel I7 processor with 48GB of RAM in 16 hours for a tree depth of 21 with 18000 query tests per node. We are not aware of another single-workstation tree-builder capable of handling this quantity of data. The very large number of examples helps to prevent overfitting demonstrated by single RDTs.

6.4.2 Convolutional neural network

For our second evaluated method we build a CNN architecture based on Tompson *et al*let@tokeneonedot[136] to predict the location of the five fingertips by using the maximum location in a set of heat maps which implicitly represent fingertip locations. We refer the reader to that work for specific details and to our supplementary material for the explicit architecture of our implementation. This multi-layer deep approach is critical for an input space as complicated as the set of images of an articulated object and we found that the deeper convolutional layers extract feature responses on a higher semantic level such as oriented fingertips. Using the heatmap based error objective helps to spatially regularize the network during training. For input to the CNN we set D_1 to be the cropped depth resized to 96×96 pixels. We then downsample it by a factor of two twice to produce D_2 and D_3. We use a subtractive form of local contrast normalization (LCN) [136, 69] so that $D_i \leftarrow D_i - g_\sigma * D_i$ using a Gaussian smoothing kernel with $\sigma = 5$ pixels. The triplet (D_1, D_2, D_3) is then input to the network. The trained network outputs a heatmap h_i per fingertip i for new data. Our method differs for fingertip detection in that we augment the output by

a non-fingertip heatmap that is strong wherever a fingertip is not likely to be present. Also, instead of fitting a Gaussian model to the strongest mode in the low resolution heatmaps, we instead upsample each 18×18 fingertip heatmap h_i to a fixed size of 128×128 with a smoothing bi-linear interpolator. Similar to Section 6.4.1 every pixel q is labeled with fingertip identity (including a non-fingertip class)

$$c^*(q) = \underset{i \in \{0..5\}}{\arg\max}\, h_i(q)\,. \tag{6.4}$$

As in Section 6.4.1 the fingertip locations are given by the location of the largest label blob.

Training the CNN. Both the orientation regression CNN of the next Section as well as the described fingertip CNN are trained using Caffe [70] on an NVidia GTX 980 with an i7 processor and 16GB of onboard RAM. We train both with a Euclidean loss and a batch size of 100 for 100000 iterations with stochastic gradient descent. We start with a learning rate of 0.01 and reduce it by a factor of 0.2 after every ten thousand iterations. We found that repeated fine-tuning was necessary to help network convergence.

6.4.3 RDT and CNN input

Figure 6.7: a) The image shows a depth map of a hand with the overlay of the offsets (in purple) that we use when generating the attributes per pixel. The example pixel in this case is in the center of the thumb. Random, pre-computed pairs of these offsets are used to generate the 1200 features for every example in the datasets. b) The input to the CNN for both regression and heatmap prediction is a triplet (of sizes 96×96, 48×48,24×24) generated from an input depth map. Note that to highlight the difference as a result of Gaussian smoothing we have upsampled each of the three inputs to the same size.

For the features of an RDT we follow [73, 127]. An attribute vector with K features for any given pixel is defined as being $x = (x_1, x_2 \ldots x_K)$ such that $x_k = z\left(q + \frac{o_{k,1}}{z(q)}\right) - z\left(q + \frac{o_{k,2}}{z(q)}\right)$ where $q \in \mathbb{N}^2$ is an image coordinate. $z : \mathbb{N}^2 \rightarrow \mathbb{R}$ is the scene depth map supplied by the camera and $o_{k,[1:2]} \in \mathbb{N}^2$ are the two offsets defining the k_{th} attribute. Figure 6.7 shows the exponential offset distribution that we use which is based on BRISK [131]. We base our CNN on the architecture described in [136]. The authors of that work use a different training framework (Torch7 [31]) and there are likely to be differences between our architecture and theirs. However we believe our network closely approximates the one described in that paper. We use Caffe [70] for defining our two CNN networks. Figure 6.8 and Figure 6.9 show

the full structure displayed by using Caffe's network visualization capabilities. For regression output we use 9 parameters and for the heatmap output we use $18 \times 18 \times 5 = 1620$ output values. It is likely that both architectures could be updated to perform the subtractive local contrast normalization directly and therefore use only a single input channel but this has not yet been implemented.

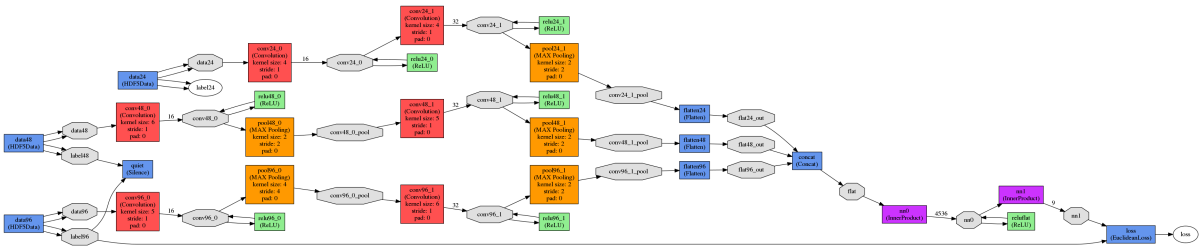


Figure 6.8: This is the network layout for the full hand orientation regression. We use Caffe's HDF5 input format as opposed to leveldb because of its simpler interface and available tools in Matlab. Label96 is used to store the 9 orientation parameters for every batch input. Label48 and Label24 are silenced.

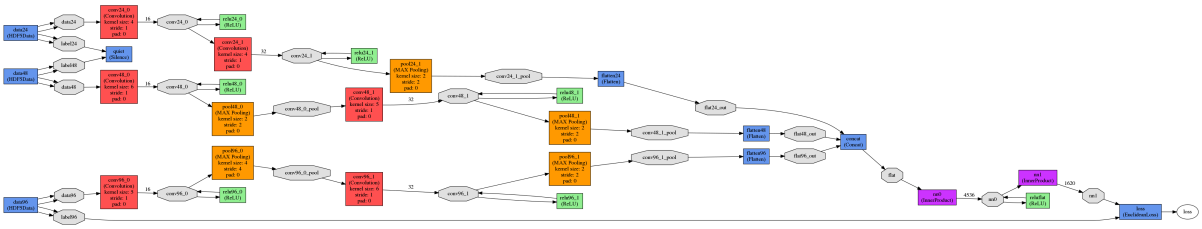


Figure 6.9: This is the network layout for a fingertip heat map predictor. We use Caffe's HDF5 input format as opposed to leveldb because of its simpler interface and available tools in Matlab. Label96 is used to store the heatmap data for every batch input. Label48 and Label24 are silenced.

6.5 Derotation

6.5.1 Orientation regression

We adapt the deep convolutional architecture from Section 6.4 to predict full 3 DOF hand orientation. Instead of a heatmap, we directly predict the 9 coefficients of the rotation matrix. There are only 3 degrees of freedom in a regular rotation but by using 9 parameters and a large database we are effectively regularizing our over-parameterized output. The representation of a rotation matrix in this way is unique in $SO\left(3\right)$ unlike quaternions and Euler angles which we found to be noisy and unreliable. This noise was most visible when trying to predict a single representative angle. For training we use Euclidian loss and do not enforce orthonormality. However, the output of this CNN is directly projected onto the closest unitary matrix using the SVD decomposition $R = USV^T$. $\hat{R} = UV^T$ then provides a least squares optimal projection into $SO\left(3\right)$, if we additionally enforce $\det\left(\hat{R}\right) = 1$. Figure 6.10 shows the result of predicting the 9 ground truth coefficients for HandNet and the full network architecture can be seen in the supplementary material.

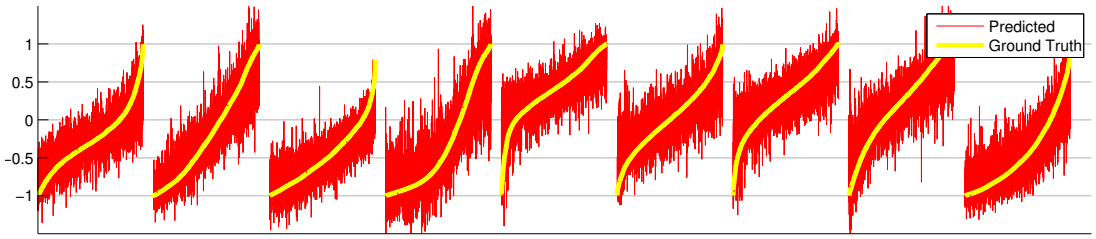


Figure 6.10: This graph shows the predicted value of all 9 coefficients of the hand orientation matrix in red relative to the ground truth in yellow. For clarity we order each ground truth coefficient monotonically and apply this reordering to the predicted results. The mean squared error for all the coefficients on the HandNet test set before and after SVD is 0.0271 and 0.0234 respectively.

6.5.2 DeROT: Designing a derotation method

Algorithm 6.1 Derotation procedure

procedure DEROTATE(R)
  $r_{align} \leftarrow \text{argmax}_{r_i \in \{r_1, r_2, r_3\}} \| (0,0,1) \cdot r_i \|$
  if r_{align} = r_2 (thumb aligned axis) **then**
    $\alpha \leftarrow atan2(r_{3x}, r_{3y}) + 90 + \begin{cases} 180 & \text{if } r_{2z} \leq 0 \\ 0 & \text{if } r_{2z} > 0 \end{cases}$
  else
    $\alpha \leftarrow atan2(r_{2x}, r_{2y}) + 90$
  end if
  Return α
end procedure

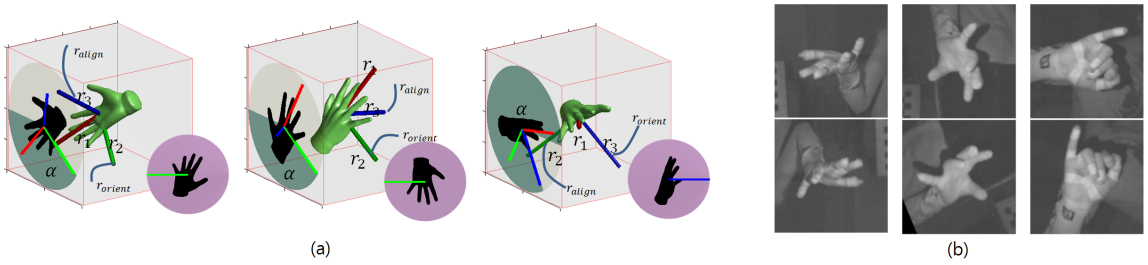


Figure 6.11: Synthetic and real examples of DeROT. a) The depth projection of the virtual hand before applying DeROT can be seen on the left wall of the cube representing the camera plane. The axis marked r_{orient} is projected onto the camera plane and used in DeROT to define the angle α. The purple circle contains the resulting image of the hand after applying derotation by angle α. b) The top row of images are un-derotated. The bottom row have been derotated by α obtained by DeROT. Note that the thumb is consistently on the right of the image.

We take advantage of the orientation prediction $\hat{R} = [r_1 r_2 r_3]$ to compute an angle α which we will use for rotating the camera image about its center. The aim of this is to reduce pose variance by heuristically forcing the thumb to be on the right side of the image. We could use a predefined axis and set the angle α with which to rotate the image to be that between the projection of this axis and the upwards image direction. Unfortunately, when this axis mostly points to or away from the camera the projection onto the screen will be small and noisy. As a simple heuristic we detect if this is the case and if so choose an alternative axis. Specifically we first determine the predicted axis r_{align} most aligned with the camera z axis as $r_{align} = \text{argmax}_{r_i \in \{r_1, r_2, r_3\}} \| (0,0,1) \cdot r_i \|$. If r_{align} is either the palm pointing direction or the direction of the extended fingers then we can be sure that the thumb direction r_2 will be non-noisy for this case and set $r_{orient} = r_2$. If the test yields instead that $r_{align} = r_2$ (i.e. thumb direction is mostly

pointing towards or away from the camera) then we instead set $r_{orient} = r_3$ which is the palm vector. This procedure is summarized in Algorithm 6.1. Synthetic and real examples can be seen in Figure 6.11. This choice is arbitrary and can be adapted for objects other than the hand. We thus define DeROT to be the combination of using the CNN from Section 6.5.1 to predict $\hat{R}$ together with this derotation heuristic.

6.5.3 Derotation with PCA and Procrustes

Instead of using DeROT, an alternative approach is to extract the principal axes of the hand silhouette using PCA and taking the rotation angle of the largest axis to the vertical image axis. We have found that a similar but more stable option is to determine an enclosing ellipse using a Procrustes like algorithm on the convex hull of the points $\mathcal{V}$ of the hand segmentation. The minimum area enclosing ellipse can be found efficiently over the points $x_i \in \text{convhull}(\mathcal{V})$ by minimizing $-\log(\det(A))$, s.t $(x_i - \overline{x}_i)^T A (x_i - \overline{x}_i) \leq 1$ for $A, \overline{x}_i$ defining the ellipse. We solve this using Khachiyan's algorithm [7]. However, as shown in Section 6.6.1 even with this added stability the method reduces performance rather than improving it.

6.6 Experiments

6.6.1 Evaluation protocol and data

Experiments. We perform our experiments using our HandNet database and the publicly available database NYUHands [136]. All experiments are performed separetly on the two databases. Our baseline results come from (I) training on non-derotated data and testing on non-derotated data. We compare this to (II) training on non-derotated data while testing with derotated data, (III) training on derotated data while testing with non-derotated data, (IV) training on derotated data while testing with derotated data.

Non-derotated data. For HandNet training we randomly select 202928 images and use the remaining 10000 images for testing. For NYUHands we use all 3 camera views (72757 images per view) for training and the frontal view for testing (8252 images). We slightly dilute the training and testing sets according to our hand segmentation pipeline which results in 184100 training images and 7241 testing images.

For experiment types (II) and (IV) we use this data to train two CNN orientation regression networks; one for each dataset. We use the same data for training the RDT and CNN fingertip detectors for experiment types (I) and (II). However, for testing the fingertip detectors in experiments (I) and (III), we rotate each testing image by uniformly random in-plane rotational offsets between -90 and 90 degrees. This further guarantees that the testing data is different from the training data.

Derotated data. Experiment types (III) and (IV) use training data which is first derotated by an Oracle which we define to be DeROT that uses the ground truth R_{gt} obtained from the magnetic sensors. With experiment types (II) and (IV) we first apply the same uniform random image rotation to the test

images exactly as for experiment types (I) and (II). We then apply one of the following: (a) Procrustes derotation, (b) DeROT using $\hat{R}$ predicted by the CNN regression network, (c) Oracle derotation with R_{gt}.

Mean precision and mean average precision. We compute precision and recall according to the protocol of [46]. We set prediction confidence as the value at the location of the fingertip detection in the 128×128 channel heatmap for each fingertip. The mean precision (mP) represents the mean precision over all fingertips at a recall rate of 100%. Mean average precision (mAP) measures the mean of all the areas under the precision-recall curves for each fingertip and takes into account the behaviour over all confidence values.

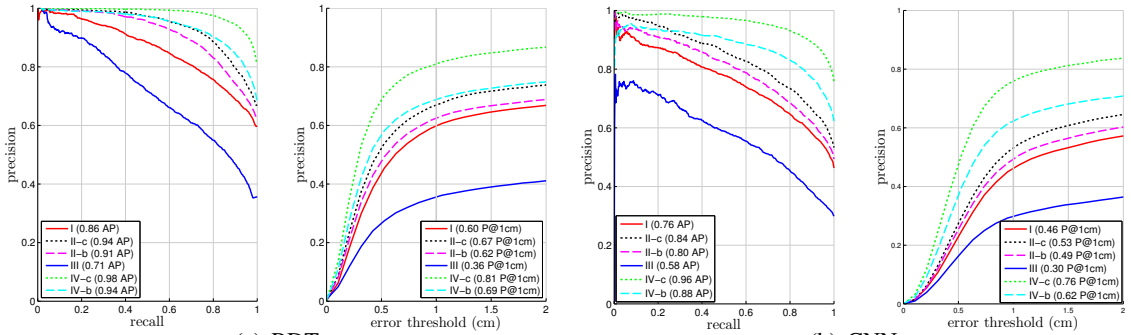
Error threshold. The error of a prediction is the distance to the ground truth location. If a fingertip is more than 6 pixels from the ground truth position it is considered a false positive. The threshold of 6 pixels roughly translates into a distance of $1\,\mathrm{cm}$ for both HandNet and NYUHands in an image patch of size 128×128 cropped according to Section 6.4. $1\,\mathrm{cm}$ is a natural threshold to choose as the distance between adjacent fingertips is over $1.6\,\mathrm{cm}$ on average [36].

Test set derotation method	None		(a) Procrustes		(b) DeROT		(c) Oracle	
	mP	mAP	mP	mAP	mP	mAP	mP	mAP
HandNet								
RDT trained on non-derotated data	0.51	0.79	0.49	0.77	**0.55**	**0.85**	**0.60**	**0.87**
RDT trained on derotated data	0.32	0.60			**0.63**	**0.88**	**0.75**	**0.95**
CNN trained on non-derotated data	0.44	0.73	0.42	0.73	**0.46**	**0.77**	**0.50**	**0.79**
CNN trained on derotated data	0.30	0.59			**0.61**	**0.88**	**0.74**	**0.95**
NYUHands								
RDT trained on non-derotated data	0.51	0.75	0.47	0.73	**0.58**	**0.84**	**0.61**	**0.86**
RDT trained on derotated data	0.35	0.58			**0.63**	**0.88**	**0.68**	**0.89**
CNN trained on non-derotated data	0.38	0.70	0.36	0.69	**0.46**	**0.80**	**0.48**	**0.81**
CNN trained on derotated data	0.23	0.42			**0.49**	**0.72**	**0.53**	**0.73**

Table 6.1: Results of our experimental evaluation for all experiment types described in Section 6.6.1. The experiment types I,II,III,IV are highlighted in red, pink, blue and cyan respectively. A result in bold indicates that it outperforms the baseline (I) shown in red. For each row pair (derotated training data vs non-derotated training), the underlined result is the better of the two. Procrustes consistently reduces the quality of fingertip detection. Conversely, DeROT outperforms the baseline *for every experiment*. For all but one experiment, this improved performance is significantly enhanced by training on derotated data instead of original data. See Section 6.6.2. The results from the Oracle serve as an upper bound achievable by derotation.

6.6.2 Discussion

The results of the experiments can be seen in Table 6.1. In Figure 6.12 we display a precision-recall curve and error threshold graph for the thumb on the HandNet test-set for all experiment types which is representative of the behavior of all fingertips. The results show that the use of DeROT improves over the baseline results *for all measurements* for both RDT and CNN for experiments on both datasets. On HandNet, when training an RDT and CNN on ground truth derotated data, we see that test-time use of DeROT yields improvement in mAP of 11.3% and 20.5% over the respective baselines. For NYUHands,



(a) RDT (b) CNN

Figure 6.12: These graphs show typical precision to recall and precision to error threshold for thumb detection (using RDT and CNN on the HandNet test set. Each line indicates an experiment which is labeled in the legend using the experiment types from Section 6.6 and the derotation types Procrustes(a), DeROT(b), Oracle(c). The baseline is in red. Training on derotated data and then applying DeROT or Oracle is in cyan or green respectively. Training on non-derotated data and then applying DeROT or Oracle is in magenta or black respectively. The average precision (AP) and precision at $1\,\mathrm{cm}$ error (P@$1\,\mathrm{cm}$) are shown for each thumb experiment.

DeROT gives an RDT a gain of 17.3% in mAP when trained on derotated data and a CNN achieves mAP gains of 14.2% when trained on underotated data but only a marginal gain of 2.5% when trained on derotated data. We found that the confidence values for this specific case were not reliable (which directly effects mAP) because of confusion between fingertips (specifically index and ring) which further justified the creation of HandNet. *For all experiments and datasets* the mP when using DeROT shows improvements of between 7.8% and 21.1% on underotated training data and between 23.5% and 38.6% for derotated training data. The simplistic Procrustes derotation negatively impacts fingertip detection relative to the baseline and we therefore chose not to build and train an RDT and CNN on Procrustes derotated versions of the two datasets. For our experiments a single RDT mostly outperforms a CNN. Although they are trained with different data and objectives it hints that there is no silver bullet to determining which machine learning approach is more appropriate. In order to further facilitate understanding the effects of derotation and its variance reduction behavior we took the full test set derotated by DeROT and compared it with the non-derotated test set. The results can be seen in Figure 6.13.

6.7 Going beyond fingertips to full hand pose

Using the discriminative fingertip classifiers described above we can achieve fast pixel classifications and joint proposals on a per frame basis and improve them by using DeROT. However these predictions still often suffer from noise, total occlusion and mismatches. Ultimately one of the best use-cases for the fingertip detection that we have described is to provide anchors for fitting and tracking a hand pose and model. To do so we need to define an energy minimization target over the required parameters using the available geometry from the visible hand surface in the depth image together with the estimated fingertip

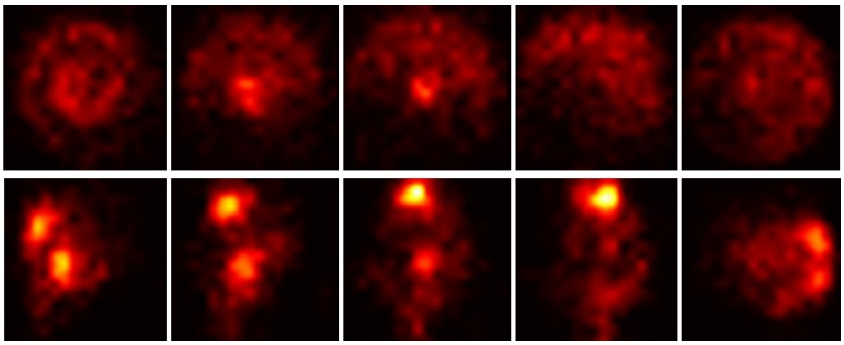


Figure 6.13: The top row from left to right shows an accumulation map for the individual finger locations (pinky, ring, middle, index, thumb) over all 10000 HandNet test images. The distribution of locations for every fingertip has high variance. The bottom row illustrates the effect of applying the proposed DeROT derotation method. The reduction of variance is noticeable in the much more concentrated distributions. It is interesting to note the bi-modal distribution for each fingertip which indicates that on average a fingertip spends most of it's time either extended (the mode further from the center) or curled (the mode close to the center). It is also useful to observe that the thumb has a more concentrated distribution to the right of it's accumulation map. This is sensible considering that the method is based on the heuristic of positioning the thumb in this fashion.

and hand points. While there are many different approaches for fitting a hand model we now describe how we adapted a model from Taylor *et al*let@tokeneonedot [134] originally used for bodies to work with hand models.

We define an inverse kinematic energy using the detected hand points and an iterative closest point like energy term along with a simple prior over the pose parameters ϕ. ϕ is parameterized using bone lengths and quaternions for bone rotations. The kinematic model with bind pose parameters ϕ_0 is skinned by a mesh with the set $\mathcal{U}^0 = \left\{u^0_{i=1:U}\right\}$ of U mesh vertices in $\mathbb{R}^3$. Linear blend skinning means that for a pose ϕ a vertex u^0_i undergoes a hierarchically chained transformation to become $u\left(\phi; \phi_0, u^0_i\right)$ which we will denote as u_i such that

$$u_i = \left[\begin{array}{cc} I & 0 \end{array}\right] \sum_{l=1}^{L} \alpha_{ik} T_l\left(\phi\right) T_l^{-1}\left(\phi_0\right) \left[\begin{array}{c} u^0_i \\ 1 \end{array}\right]. \tag{6.5}$$

Here T_l defines the homogeneous coordinate frame transformation for each kinematic joint (bone) l. The sum runs over all the L bones in the kinematic structure and each each vertex u_i is associated with a bone l and blending weight α_{il} where the weights for bone l sum to one. The weights α_{il} are usually set by an artist and are highly localized to a specific bone and smoothly become zero around the regions between bones resulting in a reasonable deformation of the skinning vertices which we shall denote by the set $\mathcal{U} = \{u_{i=1:U}\}$. We introduce a simple inverse kinematic energy which acts to create forces that pull the set of seven known salient points $\mathcal{S} = \{s_{n=1:7}\}$ locations in $\mathbb{R}^3$ to closely match the detected salient

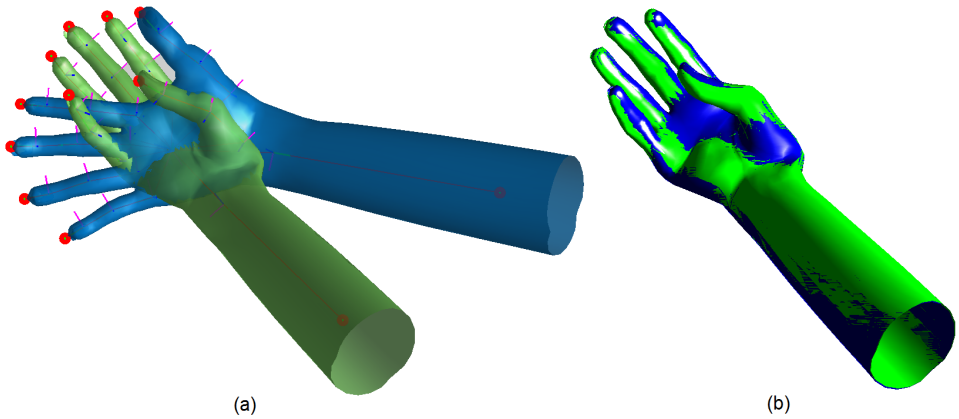


Figure 6.14: Results of matching and then fitting a source pose to a target pose. (a) The source pose is shown in blue and the target pose in green. The salient locations are illustrated with red spheres. (b) The resulting fit using the described energy minimization after 25 iterations of L-BFGS.

points denoted by $\widehat{s}_n$. This IK energy is then defined as

$$E_{IK} = \sum_{n=1}^{7} \omega_n \left\| \widehat{s}_n - s_n \left(\phi; \phi_0, h_n \right) \right\|_2^2. \tag{6.6}$$

The weights $\omega_n \in \{0, 1\}$ mask out salient points which have not been detected in the current frame. The resulting system is underdetermined and thus necessitates a prior. The prior is defined as

$$E_{prior} = \left(\phi - \overline{\phi} \right) \Sigma^{-1} \left(\phi - \overline{\phi} \right), \tag{6.7}$$

where Σ is the empirical covariance of the parameters over the training data and $\overline{\phi}$ is the mean. This prior penalizes deviation from the mean more heavily for parameters which have lower covariances. When the pose is roughly aligned with the model we can match points on the deformed mesh model to their nearest neighbors in the observed point cloud $\mathcal{V} = \{v_{k=1:V}\}$ with V detected points. Each point in the set of vertices $\mathcal{U}$ is assigned a nearest neighbor match in $\mathcal{V}$ and given a binary weight β_i indicating whether or not the match is valid according to a distance thresholding criteria. Using this scheme we can define a point-to-point and point-to-plane ICP matching energy by defining

$$E_{ICP} = \sum_{i=1}^{U} \beta_i \left(\left\| n_{v_{k(i)}}^T \left(v_{k(i)} - u_i \right) \right\|_2^2 + \lambda \left\| v_{k(i)} - u_i \right\|_2^2 \right). \tag{6.8}$$

The term $n_{v_{k(i)}}$ is the normal at the observed match location $v_{k(i)}$ and λ defines the strength of the point-to-point against the point-to-plane fitting terms. The total energy is therefore

$$E_{tot} = \gamma_{IK} E_{IK} + \gamma_{prior} E_{prior} + \gamma_{ICP} E_{ICP}. \tag{6.9}$$

The final minimization is performed by scheduling the energy weights γ so that first E_{IK} is dominant

in order to move the hand to a rough estimate of the current pose. Subsequently the importance of E_{ICP} is increased and E_{IK} reduced in order to improve the geometric data fidelity. We solve the above system using L-BFGS [87] which converges quickly to a good solution. We found that using the described model we were able to obtain qualitatively good hand poses and model matching using only fingertip and hand point detections. An example convergence can be seen in Figure 6.14. It is interesting to note that most current methods that produce hand pose estimates do so by using a full set of predictions of the locations of all the joints in a hand. While we did not pursue this direction of hand pose research extensively and it appears that further efforts to using only fingertips may be beneficial.

6.8 Conclusions and future work

In this chapter we have discussed various elements along the computational pipeline of hand processing. We described how we succesfully created a large and rich dataset of handposes and associated annotation labels. A simple approach for detecting user hands in a depth stream was then described. We also showed that by using derotation, specifically DeROT, we can significantly improve the localization ability of machine-learning based per-frame fingertip detectors by reducing the variance of the pose space. Furthermore we found that this procedure works despite the extremely high range of potential poses. We see this approach as an alternative to data augmentation and as a potentially useful additional step in pipelines dedicated to articulated object pose extraction such as hands. Finally, we briefly described one possible approach for utilizing the per-frame fingertip detections for use in a hand model and pose matching scheme. Considering the wide range of applications for hand detection and tracking it seems likely that this area of research will continue to challenge researchers in the coming years.

Chapter 7

Discussion and Conclusions

This thesis represents a stepping stone along the path of providing computers the capacity to interpret and understand some aspects of what we understand naturally about our world. We proposed to look at three primary topics along the path of computational geometric vision: reconstruction, denoising, classification. These concepts all fit along different levels of detail in the pipeline. Reconstruction and denoising can in some ways be considered low level concepts. They essentially take raw data and convert it into forms which can be ingested by other algorithms. Classification is a high level process whereby data is semantically interpreted to align with some concept which is typically easy for a human to understand. The core guiding principle throughout our research was the idea that geometric approaches that use and interpret geometry possess inherrent benefits over those that dont. Our reasoning for this has been that the world appears to be better understood through the perspective of a gemeotric framework.

We briefly touched on various ways in which the 3D world can be captured and reconstructed on mobile devices. While this was not a primary focus for our efforts it is nonetheless an instructive way to think about how future research will be influenced by the explosion in availability of mobile computation. Our discussion turned from mobile devices to a novel approach for reconstructing 3D geometry using endoscopic photometric stereo. This provided insight into how it is possible to design a direct photometric stereo method using realistic lighting and imaging conditions. We showed how our approach was supported by both theoretical and practical considerations and results.

Moving from actual 3D geometric reconstruction we described our Patch-Beltrami framework whose fundamental embedding of local space into higher dimensional patch-space embbedding provided a powerful and flexible denoising framework. We demonstrated how the framework could be extended to use deep neural architectures with many parameters. This in turn resulted in the ability to relax components of the intrinsic gemetric properties of the network and better adapt to a data driven denoising optimization scheme. We provided a number of qualitative and quantitaive experiments to ellucidate the benefits of our approach.

In the final part of this work we approached the high level problem of producing semantically meaningful output given input in the form of depth maps that capture people using their hands. We handled two different aspects of the problem. We first looked at how to efficiently create decision trees to learn on massive datasets for fast, effective machine learning train cycles and inference. We then applied this knowledge to creating various elements along a hand detection and pose estimation pipeline. Creation of a largescale hand dataset paved the way for us to build fingertip detectors using powerful machine learning tools. We additionally found that normalizing out rotation of such highly articulate objects like hands is both possible and beneficial.

It is our hope that the work covered in this thesis provides some of the groundwork for future researchers interested in exploring similar territory. To keep going forward there are still many stones left upon which to step.

Bibliography

[1] Ascension TrakStar. `http://www.ascension-tech.com/`, 2015.

[2] A. Agrawal, R. Raskar, and R. Chellappa. What is the range of surface reconstructions from a gradient field? *Computer Vision–ECCV 2006*, pages 578–591, 2006.

[3] A. Alahi, R. Ortiz, and P. Vandergheynst. Freak: Fast retina keypoint. In *CVPR*, pages 510–517, 2012.

[4] Y. Amit and D. Geman. Shape quantization and recognition with randomized trees. *Neural computation*, 9(7):1545–1588, 1997.

[5] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, and N. de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, pages 3981–3989, 2016.

[6] B. C. Antoni Buades and J.-M. Morel. A review of image denoising algorithms, with a new one. *SIAM Interdisciplinary Journal*, 4:490–530, 2005.

[7] B. Aspvall and R. E. Stone. Khachiyan's linear programming algorithm. *Journal of algorithms*, 1(1):1–13, 1980.

[8] L. Ballan, A. Taneja, J. Gall, L. V. Gool, and M. Pollefeys. Motion capture of hands in action using discriminative salient points. In *European Conference on Computer Vision (ECCV)*, Firenze, Oct. 2012.

[9] D. Barash. A fundamental relationship between bilateral filtering, adaptive smoothing and the nonlinear diffusion equation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(6):844–847, 2002.

[10] R. Basri, D. Jacobs, and I. Kemelmacher. Photometric stereo with general, unknown lighting. *International Journal of Computer Vision*, 72(3):239–257, 2007.

[11] P. J. Besl. Active optical range imaging sensors. In *Advances in machine vision*, pages 1–63. Springer, 1989.

[12] J. Bouguet. Camera calibration toolbox for matlab. 2004.

[13] L. Breiman. Bagging predictors. In *Machine Learning*, pages 123–140, 1996.

[14] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[15] M. Breuß, E. Cristiani, J.-D. Durou, M. Falcone, and O. Vogel. Perspective shape from shading: Ambiguity analysis and numerical approximations. *SIAM Journal on Imaging Sciences*, 5(1):311–342, 2012.

[16] A. M. Bronstein, M. M. Bronstein, E. Gordon, and R. Kimmel. High-resolution structured light range scanner with automatic calibration. *Techn. Report CIS-2003-06, Dept. Computer Science, Technion, Israel*, 2003.

[17] A. Buades, B. Coll, and J. M. Morel. A non-local algorithm for image denoising. In *CVPR*, pages 60–65, 2005.

[18] M. Budiu, J. Shotton, D. Murray, and M. Finocchio. Parallelizing the training of the kinect body parts labeling algorithm. In *Big Learning: Algorithms, Systems and Tools for Learning at Scale*, 2011.

[19] M. Calonder, V. Lepetit, C. Strecha, and P. Fua. Brief: Binary robust independent elementary features. In *ECCV (4)*, pages 778–792, 2010.

[20] V. Caselles, R. Kimmel, and G. Sapiro. Geodesic active contours. *International journal of computer vision*, 22(1):61–79, 1997.

[21] M. Chandraker, J. Bai, and R. Ramamoorthi. On differential photometric reconstruction for unknown, isotropic brdfs. *IEEE transactions on pattern analysis and machine intelligence*, 35(12):2941–2955, 2013.

[22] M. K. Chandraker, S. Agarwal, and D. J. Kriegman. Shadowcuts: Photometric stereo with shadows. In *CVPR*, 2007.

[23] S. G. Chang, B. Yu, and M. Vetterli. Adaptive wavelet thresholding for image denoising and compression. *IEEE transactions on image processing*, 9(9):1532–1546, 2000.

[24] P. Chatterjee and P. Milanfar. Is denoising dead? *IEEE Transactions on Image Processing*, 19(4):895–911, 2010.

[25] Q. Chen, S. Xiao, W. Yichen, T. Xiaoou, and S. Jian. Realtime and robust hand tracking from depth. In *Conference on Computer Vision and Pattern Recognition (cvpr)*, pages 1106–1113. IEEE, 2014.

[26] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794. ACM, 2016.

[27] T. Chen, K.-K. Ma, and L.-H. Chen. Tri-state median filter for image denoising. *IEEE Transactions on Image processing*, 8(12):1834–1838, 1999.

[28] Y. Chen, W. Yu, and T. Pock. On learning optimized reaction diffusion processes for effective image restoration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5261–5269, 2015.

[29] J. J. Clark and H. Pekau. An integral formulation for differential photometric stereo. In *Computer Vision and Pattern Recognition, 1999. IEEE Computer Society Conference on.*, volume 1, pages 119–124. IEEE, 1999.

[30] T. Collins and A. Bartoli. 3D reconstruction in laparoscopy with close-range photometric stereo. *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2012*, pages 634–642, 2012.

[31] R. Collobert, K. Kavukcuoglu, and C. Farabet. Torch7: A matlab-like environment for machine learning.

[32] D. Comaniciu and P. Meer. Mean shift: A robust approach toward feature space analysis. 24:603–619, 2002.

[33] T. F. Cootes, G. J. Edwards, and C. J. Taylor. Active appearance models. *IEEE Transactions on pattern analysis and machine intelligence*, 23(6):681–685, 2001.

[34] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian. Image denoising by sparse 3D transform-domain collaborative filtering. *IEEE Transactions on Image Processing*, pages 2080–2095, 2007.

[35] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian. Image restoration by sparse 3D transform-domain collaborative filtering. In J. T. Astola, K. O. Egiazarian, and E. R. Dougherty, editors, *Proc. SPIE*, volume 6812, 2008.

[36] K. Dandekar, B. Raju, and M. Srinivasan. 3D finite-element models of human and monkey fingertips to investigate the mechanics of tactile sense. 125:682–691, 2003.

[37] J. Davis, R. Ramamoorthi, and S. Rusinkiewicz. Spacetime stereo: A unifying framework for depth from triangulation. In *Computer Vision and Pattern Recognition, 2003. Proceedings. 2003 IEEE Computer Society Conference on*, volume 2, pages II–359. IEEE, 2003.

[38] J. Dean and S. Ghemawat. Mapreduce: Simplified data processing on large clusters. In *OSDI*, pages 137–150, 2004.

[39] M. Desbrun, M. Meyer, P. Schröder, and A. H. Barr. Implicit fairing of irregular meshes using diffusion and curvature flow. In *SIGGRAPH*, pages 317–324, New York, NY, USA, 1999. ACM Press/Addison-Wesley Publishing Co.

[40] D. L. Donoho, I. M. Johnstone, G. Kerkyacharian, and D. Picard. Wavelet shrinkage: asymptopia? *Journal of the Royal Statistical Society. Series B (Methodological)*, pages 301–369, 1995.

[41] A. Dubrovina, G. Rosman, and R. Kimmel. Multi-region active contours with a single level set function. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 2015.

[42] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

[43] J. D. Durou, J.-F. Aujol, and F. Courteille. Integrating the normal field of a surface in the presence of discontinuities. In *7th International Conference on Energy Minimization Methods in Computer Vision and Pattern Recognition*, 2009.

[44] M. Elad. On the origin of the bilateral filter and ways to improve it. *IEEE Transactions on Image Processing*, 11(10):1141–1151, 2002.

[45] M. Elad and M. Aharon. Image denoising via sparse and redundant representations over learned dictionaries. *IEEE Transactions on Image processing*, 15(12):3736–3745, 2006.

[46] M. Everingham, L. V. Gool, J. Winn, C. Williams, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2011 (VOC2011) Results. http://www.pascal-network.org/challenges/VOC/voc2011/workshop/index.html.

[47] G. Fanelli, J. Gall, and L. Van Gool. Real time head pose estimation with random regression forests. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 617–624. IEEE, 2011.

[48] P. Favaro and S. Soatto. A geometric approach to shape from defocus. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(3):406–417, 2005.

[49] S. Fleishman, D. Cohen-Or, and C. T. Silva. Robust moving least-squares fitting with sharp features. *SIGGRAPH*, pages 544–552, 2005.

[50] S. Fleishman, I. Drori, and D. Cohen-Or. Bilateral mesh denoising. *Transactions on Graphics*, 22(3):950–953, 2003.

[51] S. Foix, G. Alenya, and C. Torras. Lock-in time-of-flight (tof) cameras: A survey. *IEEE Sensors Journal*, 11(9):1917–1926, 2011.

[52] B. Foundation. Blender.

[53] R. T. Frankot and R. Chellappa. A method for enforcing integrability in shape from shading algorithms. 10(4):439–451, 1988.

[54] J. Friedman. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002.

[55] J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.

[56] K. Fukunaga and L. Hostetler. The estimation of the gradient of a density function, with applications in pattern recognition. *Information Theory, IEEE Transactions on*, 21(1):32–40, 1975.

[57] J. Gehrke, R. Ramakrishnan, and V. Ganti. Rainforest - a framework for fast decision tree construction of large datasets. In *VLDB*, pages 416–427, 1998.

[58] H. Grahn, N. Lavesson, M. H. Lapajne, and D. Slat. Cudarf: a CUDA-based implementation of random forests. In *Computer Systems and Applications (AICCSA), 2011 9th IEEE/ACS International Conference on*, pages 95–101. IEEE, 2011.

[59] S. Gu, L. Zhang, W. Zuo, and X. Feng. Weighted nuclear norm minimization with application to image denoising. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2862–2869, 2014.

[60] A. W. H. Tomkins, R. Herskovitz. Caffe gui tool. `https://github.com/twerdster/caffe-gui-tool`, 2015.

[61] M. Hansard, S. Lee, O. Choi, and R. P. Horaud. *Time-of-flight cameras: principles, methods and applications*. Springer Science & Business Media, 2012.

[62] C. Hernández, G. Vogiatzis, and R. Cipolla. Shadows in three-source photometric stereo. In *Eccv*, pages 290–303, 2008.

[63] H. Hirschmuller. Stereo processing by semiglobal matching and mutual information. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 30(2):328–341, 2008.

[64] B. K. Horn. Shape from shading: A method for obtaining the shape of a smooth opaque object from one view. 1970.

[65] B. K. Horn. Height and gradient from shading. *International journal of computer vision*, 5(1):37–75, 1990.

[66] C. Huang, X. Ding, and C. Fang. Head pose estimation based on random forests for multiclass classification. In *Pattern Recognition (ICPR), 2010 20th International Conference on*, pages 934–937. IEEE, 2010.

[67] L. Hyafil and R. Rivest. Constructing optimal binary decision trees is np-complete. *Information Processing Letters*, 5:15–17, 1976.

[68] Y. Iwahori, H. Sugie, and N. Ishii. Reconstructing shape from shading images under point light source illumination. In *Pattern Recognition, 1990. Proceedings., 10th International Conference on*, volume 1, pages 83–87. IEEE, 1990.

[69] K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? In *International Conference on Computer Vision (iccv)*, pages 2146–2153. IEEE, 2009.

[70] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. 2014.

[71] Z. Kalal, K. Mikolajczyk, and J. Matas. Tracking-learning-detection. 34(7):1409–1422, 2012.

[72] V. Kazemi and J. Sullivan. One millisecond face alignment with an ensemble of regression trees. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1867–1874, 2014.

[73] C. Keskin, F. Kiraç, Y. E. Kara, and L. Akarun. Real time hand pose estimation using depth sensors. In *Consumer Depth Cameras for Computer Vision*, pages 119–137. Springer, 2013.

[74] D. Kim, O. Hilliges, S. Izadi, A. Butler, J. Chen, I. Oikonomidis, and P. Olivier. Digits: freehand 3D interactions anywhere using a wrist-worn gloveless sensor. In *Proceedings of*

the symposium on User interface software and technology (UIST), pages 167–176. ACM, 2012.

[75] R. Kimmel. *Numerical geometry of images: Theory, algorithms, and applications*. Springer Science & Business Media, 2012.

[76] R. Kimmel and A. M. Bruckstein. Tracking level sets by level sets: A method for solving the shape from shading problem. *Computer Vision and Image Understanding*, 62(1):47–58, 1995.

[77] R. Kimmel, R. Malladi, and N. Sochen. Images as embedded maps and minimal surfaces: Movies, color, texture, and volumetric medical images. 39(2):111–129, 2000.

[78] P. Krähenbühl and V. Koltun. Efficient inference in fully connected crfs with gaussian edge potentials. In *Advances in neural information processing systems*, pages 109–117, 2011.

[79] N. Kutulakos and S. Seitz. A theory of shape by space carving. *IJCV*, 38(3):199–218, 2000.

[80] C. Lange and K. Polthier. Anisotropic smoothing of point sets. *Comput. Aided Geom. Des.*, 22(7):680–692, 2005.

[81] B. Larson. GPUimage. `https://github.com/BradLarson/GPUImage/`, 2016.

[82] V. Lepetit, P. Lagger, and P. Fua. Randomized trees for real-time keypoint recognition. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 775–781, 2005.

[83] V. Lepetit, F. Moreno-Noguer, and P. Fua. E-pnp: An accurate o(n) solution to the pnp problem. *International journal of computer vision*, pages 155–166, 2009.

[84] A. Levin and B. Nadler. Natural image denoising: Optimality and inherent bounds. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 2833–2840. IEEE, 2011.

[85] D. Levin. The approximation power of moving least-squares. *Math. Comput.*, 67(224):1517–1531, 1998.

[86] Y. Liao, A. Rubinsteyn, R. Power, and J. Li. Learning random forests on the GPU. *New York University, Department of Computer Science*, 2013.

[87] D. Liu and J. Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1-3):503–528, 1989.

[88] W. Lorensen and H. Cline. Marching cubes: A high resolution 3D surface construction algorithm. In *ACM siggraph computer graphics*, volume 21, pages 163–169. ACM, 1987.

[89] R. Mecca and M. Falcone. Uniqueness and approximation of a photometric shape-from-shading model. *SIAM Journal on Imaging Sciences*, 6(1):616–659, 2013.

[90] R. Mecca, A. Tankus, and A. M. Bruckstein. Two-image perspective photometric stereo using shape-from-shading. In *Accv*, page 12, 2012.

[91] R. Mecca, A. Tankus, A. Wetzler, and A. M. Bruckstein. A direct differential approach to photometric stereo with perspective viewing. *SIAM Journal on Imaging Sciences*, 7(2):579–612, 2014.

[92] M. Mehta, R. Agrawal, and J. Rissanen. Sliq: A fast scalable classifier for data mining. In *EDBT*, pages 18–32, 1996.

[93] S. Melax, L. Keselman, and S. Orsten. Dynamics based 3D skeletal hand tracking. In *I3D*, page 184, 2013.

[94] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, et al. Mllib: Machine learning in apache spark. *The Journal of Machine Learning Research*, 17(1):1235–1241, 2016.

[95] Microsoft. Lightgbm. `https://github.com/Microsoft/LightGBM`, 2016.

[96] T. Migita, K. Sogawa, and T. Shakunaga. Specular-free residual minimization for photometric stereo with unknown light sources. In *Pacific-Rim Symposium on Image and Video Technology*, pages 178–189. Springer, 2011.

[97] G. Narlikar. A parallel, multithreaded decision tree builder. Technical Report CMU-CS-98-184, Computer Science Department, Carnegie Mellon University, Dec. 1998.

[98] J. Nickolls, I. Buck, M. Garland, and K. Skadron. Scalable parallel programming with cuda. *Queue*, 6(2):40–53, 2008.

[99] NVidia. Cuda samples. `http://docs.nvidia.com/cuda/cuda-samples/index.html#cuda-histogram`, 2016.

[100] J. Oberg, K. Eguro, R. Bittner, and A. Forin. Random decision tree body part recognition using FPGAs. In *Field Programmable Logic and Applications (FPL), 2012 22nd International Conference on*, pages 330–337. IEEE, 2012.

[101] I. Oikonomidis, N. Kyriazis, and A. A. Argyros. Markerless and efficient 26-dof hand pose recovery. In *Asian Conference on Computer Vision*, pages 744–757. Springer, 2010.

[102] T. Okatani and K. Deguchi. Shape reconstruction from an endoscope image by shape from shading technique for a point light source at the projection center. *Computer vision and image understanding*, 66(2):119–131, 1997.

[103] R. Or El, G. Rosman, A. Wetzler, R. Kimmel, and A. M. Bruckstein. RGBD-Fusion: Real-time high precision depth recovery. In *Ieee Conference on Computer Vision and Pattern Recognition (cvpr)*, pages 5407–5416, 2015.

[104] S. Owen, R. Anil, T. Dunning, and E. Friedman. Mahout in action. 2011.

[105] B. Panda, J. S. Herbach, S. Basu, and R. J. Bayardo. PLANET: massively parallel learning of tree ensembles with mapreduce. *Proceedings of the VLDB Endowment*, 2(2):1426–1437, 2009.

[106] T. Papadhimitri and P. Favaro. A new perspective on uncalibrated photometric stereo. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1474–1481, 2013.

[107] J. Park, H. Kim, Y.-W. Tai, M. S. Brown, and I. Kweon. High quality depth map upsampling for 3d-tof cameras. In *Computer Vision (ICCV), 2011 IEEE International Conference on*, pages 1623–1630. IEEE, 2011.

[108] V. Parot, D. Lim, G. González, G. Traverso, N. S. Nishioka, B. J. Vakoc, and N. J. Durr. Photometric stereo endoscopy. *Journal of biomedical optics*, 18(7):076017–076017, 2013.

[109] P. Perona and J. Malik. Scale-space and edge detection using anisotropic diffusion. *IEEE Trans. Pattern Anal. Mach Intell*, pages 629–639, 1990.

[110] G. Peyre. Manifold models for signals and images. *Computer Vision and Image Understanding*, 113:249–260, Feb. 2009.

[111] U. Pinkall and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics*, 2(1):15–36, 1993.

[112] E. Prados and O. Faugeras. Shape from shading: a well-posed problem? In *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, volume 2, pages 870–877. IEEE, 2005.

[113] J. R. Quinlan. *C4.5: Programs for Machine Learning*. Elsevier, 2014.

[114] A. W. R. Slossberg and R. Kimmel. Freehand laser scanning using mobile phone. In *BMVC*, pages 88–1, 2015.

[115] G. Rosman, A. Dubrovina, and R. Kimmel. Patch-collaborative spectral surface denoising. Technical Report CIS-2012-03, Technion, Department of Computer Science, 2012.

[116] A. Roussos and P. Maragos. Tensor-based image diffusions derived from generalizations of the total variation and Beltrami functionals. In *ICIP*, Sept. 2010.

[117] O. Rubinstein, Y. Honen, A. Bronstein, M. Bronstein, and R. Kimmel. 3D-color video camera. In *Computer Vision Workshops (ICCV Workshops), 2009 IEEE 12th International Conference on*, pages 1505–1509. IEEE, 2009.

[118] R. B. Rusu and S. Cousins. 3D is here: Point cloud library (PCL). In *IEEE International Conference on Robotics and Automation*, Shanghai, China, May 2011.

[119] J. Salvi, J. Pages, and J. Batlle. Pattern codification strategies in structured light systems. *Pattern recognition*, 37(4):827–849, 2004.

[120] Y. Y. Schechner and N. Kiryati. Depth from defocus vs. stereo: How different really are they? *International Journal of Computer Vision*, 39(2):141–162, 2000.

[121] T. Schmidt, R. Newcombe, and D. Fox. Dart: Dense articulated real-time tracking. 2014.

[122] U. Schmidt and S. Roth. Shrinkage fields for effective image restoration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2774–2781, 2014.

[123] H. Schulz, B. Waldvogel, R. Sheikh, and S. Behnke. CURFIL: Random forests for image labeling on GPU. In *VISAPP (2)*, pages 156–164, 2015.

[124] J. Shafer, R. Agrawal, and M. Mehta. SPRINT: A scalable parallel classifier for data mining. In *VLDB*, pages 544–555, 1996.

[125] T. Sharp. Implementing decision trees and forests on a GPU. In *ECCV (4)*, pages 595–608, 2008.

[126] X. Shen, A. Hertzmann, J. Jia, S. Paris, B. Price, E. Shechtman, and I. Sachs. Automatic portrait segmentation for image stylization. In *Computer Graphics Forum*, volume 35, pages 93–102. Wiley Online Library, 2016.

[127] J. Shotton, A. Fitzgibbon, M. Cook, T. Sharp, M. Finocchio, R. Moore, A. Kipman, and A. Blake. Real-time human pose recognition in parts from single depth images. In *Conference on Computer Vision and Pattern Recognition (cvpr)*, pages 1297–1304, 2011.

[128] N. Sochen, R. Kimmel, and R. Malladi. A general framework for low level vision. pages 310–318, 1998.

[129] N. A. Sochen, G. Gilboa, and Y. Y. Zeevi. Color image enhancement by a forward-and-backward adaptive Beltrami flow. In *AFPAC*, pages 319–328, 2000.

[130] A. Spira, R. Kimmel, and N. A. Sochen. Efficient Beltrami flow using a short time kernel. In *Scale-Space*, pages 511–522, 2003.

[131] L. Stefan, C. Margarita, and S. R. Yves. Brisk: Binary robust invariant scalable keypoints. In *International Conference on Computer Vision (iccv)*, pages 2548–2555. IEEE, 2011.

[132] J. Struharik. Implementing decision trees in hardware. In *Intelligent Systems and Informatics (SISY), 2011 IEEE 9th International Symposium on*, pages 41–46. IEEE, 2011.

[133] R. J. Struharik and L. A. Novak. Intellectual property core implementation of decision trees. *IET computers & digital techniques*, 3(3):259–269, 2009.

[134] J. Taylor, J. Shotton, T. Sharp, and A. Fitzgibbon. The Vitruvian manifold: Inferring dense correspondences for one-shot human pose estimation. In *CVPR*, pages 103–110, 2012.

[135] C. Tomasi and R. Manduchi. Bilateral filtering for gray and color images. In *Proc. IEEE ICCV*, pages 836–846, 1998.

[136] J. Tompson, M. Stein, Y. Lecun, and K. Perlin. Real-time continuous pose recovery of human hands using convolutional networks. *ACM Transactions on Graphics (ToG)*, 33(5):169, 2014.

[137] D. Tschumperlé and L. Brun. Non-local image smoothing by applying anisotropic diffusion pde's in the space of patches. In *ICIP*, pages 2957–2960, 2009.

[138] P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *Cvpr (1)*, pages 511–518, 2001.

[139] M. Šarić. Libhand: A library for hand articulation, 2011.

[140] R. Y. Wang and J. Popović. Real-time hand-tracking with a color glove. In *ACM transactions on graphics (TOG)*, volume 28, page 63. ACM, 2009.

[141] A. Wetzler and R. Kimmel. Efficient Beltrami flow in patch-space. In *SSVM*, pages 134–143, 2011.

[142] A. Wetzler, G. Rosman, and R. Kimmel. Patch-space Beltrami denoising of 3D point clouds. In *Electrical & Electronics Engineers in Israel (IEEEI), 2012 IEEE 27th Convention of*, pages 1–5. IEEE, 2012.

[143] A. Wetzler, R. Slossberg, and R. Kimmel. Rule of thumb: Deep derotation for improved fingertip detection. In M. W. J. Xianghua Xie and G. K. L. Tam, editors, *Proceedings of the British Machine Vision Conference (BMVC)*, pages 33.1–33.12. BMVA Press, Sept. 2015.

[144] L. B. Wolff and E. Angelopoulou. 3D stereo using photometric ratios. In *European Conference on Computer Vision*, pages 247–258. Springer, 1994.

[145] R. Woodham, Y. Iwahori, and R. A. Barman. *Photometric stereo: Lambertian reflectance and light sources with unknown direction and strength*. University of British Columbia. Department of Computer Science, 1991.

[146] C. Wu, S. G. Narasimhan, and B. Jaramaz. A multi-image shape-from-shading framework for near-lighting perspective endoscopes. *International Journal of Computer Vision*, 86(2):211–228, 2010.

[147] X. Xiong and F. De la Torre. Supervised descent method and its applications to face alignment. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 532–539, 2013.

[148] L. P. Yaroslavsky. *Digital Picture Processing*. Springer Verlag New York, Inc., Secaucus, NJ, USA, 1985.

[149] S. Yeung, H.-T. Tsui, and A. Yim. Global shape from shading for an endoscope image. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 318–327. Springer, 1999.

[150] S. Yoshizawa, A. Belyaev, and H. P. Seidel. Smoothing by example: mesh denoising by averaging with similarity-based weights. *Proceedings of International Conference on Shape Modelling and Applications*, pages 38–44, 2006.

[151] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, et al. Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

[152] C. Zhang, J. C. Platt, and P. A. Viola. Multiple instance boosting for object detection. In *Advances in neural information processing systems*, pages 1417–1424, 2006.

[153] L. Zhang, B. Curless, and S. M. Seitz. Rapid shape acquisition using color structured light and multi-pass dynamic programming. In *3D Data Processing Visualization and Transmission, 2002. Proceedings. First International Symposium on*, pages 24–36. IEEE, 2002.

[154] W. Zhang, B. Deng, J. Zhang, S. Bouaziz, and L. Liu. Guided mesh normal filtering. *Computer Graphics Forum (Special Issue of Pacific Graphics 2015)*, 34:23–34, 2015.

[155] Z. Zhang. On the epipolar geometry between two images with lens distortion. In *Proceedings of: International Conference on Pattern Recognition (ICPR)*, pages 407–411, 1996.

[156] W. Zhao, J. Chai, and Y. Xu. Combining marker-based mocap and rgb-d camera for acquiring high-fidelity hand motion data. In *Symposium on Computer Animation*, pages 33–42, 2012.

[157] S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. Torr. Conditional random fields as recurrent neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1529–1537, 2015.

מודרנית. המודל הנלמד יכול להיות מיושם במהירויות גבוהות בזמן אמת על חומרה מקבילה כדי לאפשר את זיהוי האצבעות באופן יעיל. מערכת זו מהווה אבן דרך חשובה בעולם שבוא יתאפשרו אינטראקציות טבעיות יותר עם המחשב. לסיכוף תזה זו מתמקדת בחקר אלמנטים רבים של ראייה גיאומטרית חישובית.אנו מאמינים שעל ידי שילוב עקרונות גיאומטריים לניתוח צורות עם מערכי נתונים גדולים וארכיטקטורות חישוביות רבות קיים סיכוי גדול יותר לאפשר ל"עמיתינו החשמליים" להבין אותנו ואת העולם הסובב אותם.

הזמינים כדי לקבוע תחילה כמות עקיפה כגון כיוון תאורה, נורמלי שטח או פוקוס בתמונה של סצנה. לאחר מכן, יש שילוב של המידע תחת אילוצים טבעיים על מנת להגיע לתוצאה הסופית. כחלק מהעבודה בתזה זו אנו נתאר פיתוח של וריאציה ייחודית של שימוש במשוואות דיפרנציאליות חלקיות לשיחזור עומק באמצעות הסטריאו הפוטומטרי . השיטה מטפלת במקרים של צילום מקרוב ומקורות אור הנמצאים בקרבת מקום הצילום. נראה שהשיטה יכולה להתמודד באופן טבעי עם הסתרות ונחקור את התנהגות השיטה תחת מצבים שונים של רעש ותנאי נתונים חסרים. הורדת רעש חיישנים מוגבלים על ידי תוכנם ההנדסי וחוקי הפיסיקה. התוצאה היא, כי ישנם מקורות שונים של רעש שמוצאים את דרכם לתוך הפלט. חיישני אור סובלים מרמות נמוכות של אור, חום, ארכיטקטורה מסובכת, ארטיפקטים של דחיסה ועוד הרבה גורמים אחרים שיכולים להשפיע על האיכות הסופית של הנתונים המיוצרים. עשרות שנים של מחקר התמקדו באופן בו ניתן לחסל את הבעיות הללו. ההחלטה כיצד להתמודד עם הבעיה תלויה באילוצים על רמת הזמינות החישובית וביעילות של התהליך הרצוי. הפתרון הפשוט ביותר להסרת רעש מתמונה הוא להתייחס לתמונה כאל אות רועש ולבצע סינון באמצעות ממוצע או טשטוש גאוסיאני. סינון לינארי זה מתעלם מתוכן התמונה ומתייחס אליה כאל אות דו־ממדי עם רכיבי תדר שניתן להבין ולטפל בהם בכלים מאנליזה של פורייה. החיסרון הוא כי לטכניקות אלה אין יכולת מובנית שמתחשבת במבנים כמו קצוות ופינות המופיעות בתמונות טבעיות. כתוצאה מכך, פותחו טכניקות מתקדמות אשר יכולות להתגבר על המגבלות הטמונות בפעולות הטשטוש הבסיסיות. הגישות המוצלחות המרכזיות כוללות את שיטת החציון, סינון הבילטרלי ,מודלים דלילים, הצטמקות של מקדמי וויבלטים, תיקון שיתופי, מודלים דיפוזיים אנאיזוטרופיים ולאחרונה, ארכיטקטורות עצביות עמוקות. במהלך המחקר, פיתחנו פריימוורק חדש עבור הורדת רעש אשר אנו קוראים לו תיקון פאטצ בלטרמי. בהמשך תזה זו נדגים כיצד אנו נוקטים בגישה גיאומטרית עקרונית להתמודדות עם הבעיה גם בתמונות וגם בעננים של נקודות. השיטה מאפשרת הורדת רעש תוך שימוש בגישה בהשראה גיאומטרית ומניבה תוצאות המשתוות לשיטות אחרות מודרניות תוך כדי ניצול הכלים והפרספקטיבות של למידה עמוקה. הבנה: מבעד לחלון בתזה זו הרעיון העיקרי שמנחה את מאמצינו הוא שניתן להשתמש בנתונים איכותיים ומולטימודליים יחד עם אלגוריתמים חזקים כדי לאפשר לנו אינטראקציה טבעית ויעילה יותר עם המחשב. לאחר יכולת הדיבור, הידיים הן הכלי החזק ביותר שאיתו אפשר לתקשר עם הזולת. היד האנושית היא מכשיר יעיל שהתפתח במשך מיליוני שנים כדי להקנות יתרון אבולוציוני בכך שהוא מאפשר שימוש בכלים ובתקשורת אנושית־אנושית לא מילולית. עידן המחשוב המודרני מחייב מציאות בה אנו עבדים לסמן וירטואלי שנע בסביבה שטוחה. התוצאה היא שהאינטראקציה עם המכונות הפועלות בחיינו היא עדיין חיקוי חלש ומוגבל לעומת זאת הקיימת בין שני אנשים. בעידן המחשוב המודרני מוטל כעת על החוקרים לפתח שיטות אינטראקציה טבעיות ואמינות יותר, המנצלות את ההתקדמות בכוח המחשוב. הבנייה המוצלחת של מערכת הנשלטת בידיים עם פונקציונלית מלאה תהיה עם השלכות מרחיקות לכת. כדי להתקדם לקראת מערכת כזו נצטרך ליצור כמות עצומה של מידע המייצג קרוב ככל האפשר את הטווח המלא של תנועת הידיים. לאפשר למידה על כמות הדטה הגדולה נתאר מיפוי חדשני של למידה בעץ לחומרה מקבילה. לאחר מכן נבחן כיצד זה ואחרים ממנגנוני למידה רבי עוצמה יכולים לשמש כדי לאפשר הבנה סמנטית של חלקי יד כפי שהם נקלטים על ידי מצלמת עומק. נדבר על גישה חדשה לדחיסת מרחב תנוחות הידיים אשר מפשטת את המרחב שבו אנו צריכים להפעיל את הליכי הלמידה שלנו. אנחנו גם נסביר את הגישה שלנו לבניית מאגר גדול של תנוחות ידיים עם תיוג מדוייק של האצבעות. התוצאה הסופית של מאמצינו היא יכולת הלימוד של המערכת שלנו על מיליארדי דוגמאות תוך כדי שימוש בחומרה

תקציר

עבודה זו נועדה לקדם את גבולות הבנתנו על האופן בו אנו יכולים לספק למחשבים יכולת לעבד ולהבין את הגיאומטריה של העולם. לשם כך, עלינו לבנות מערך של כלים תיאורטיים וחישוביים תוך שימוש בגאומטריה כקו מנחה . אנו גם לוקחים השראה משמעותית מן הגישה המוצלחת של שימוש ברשתות עצביות עמוקות ומסדי נתונים גדולים עבור תחומים טקסטואליים וויזואולים. כדי שנוכל לשלב בין הדברים האלה נצטרך לקחת השקפה פחות טהורה. למעשה, המתמטיקאי המפורסם הנרי פוינקיר אמר פעם "גיאומטריה זה לא נכון, זה מועיל". במילים אחרות, העולם הוא מבולגן אבל הגיאומטריה מספקת עדשה טובה שדרכה אפשר לצפות בו. לאור גישה זו התחלנו את מאמצינו המדעיים על ידי שילוב של עקרונות גיאומטריים לניתוח צורה עם טכניקות חישה מודרניות. אנו מראים בעבודה זו דרכים שונות לאפשר "לעמיתים החשמליים" שלנו להבין את העולם הסובב אותם. העולם בתלת מימד העולם הפיזי שבו אנו חיים מורכב משלושה ממדים מרחביים. אולם, המידע העובר דרך העיניים שלנו ושל חיישני מצלמה הינו הטלה ממרחב תלת מימדי על מרחב דו ממדי - ייצוג שבהכרח מאבד מידע. במובן מסוים גם הביולוגיה שלנו וגם הטכנולוגיה שאנו מפתחים מוגבלים ביכולתם לייצג באופן מלא את המבנה הגיאומטרי של העולם הסובב אותנו. למרות מגבלה זו, האנושות השקיעה משאבים רבים בייצור טכנולוגיות מסך וחישה המאפשרים ייצוג מקורב של העולם. למעשה לאחרונה יכולות אלו מצאו את דרכן לתחום חישת עומק גיאומטרית ובאו לידי ביטוי במכשירי סריקה פופולריים כמו הקינקט ממיקרוסופט והרילסנס מאינטל. במהלך המחקר שבוצע כאן, ראינו עלייה דרמטית במספר ובמגוון מצלמות העומק הזמינות בשוק, כמו גם ריבוי השיטות השונות שמאפשרות שיחזור תלת מימדי מהסביבה. מרפואה, רובוטיקה, גאוגרפיה ועד בידור וממשקי מחשב - הטווח הרחב של יישומים שיכולים להשתמש בטכנולוגיות תלת מימד מכסה כמעט את כל ספקטרום הפעילות האנושית. בהינתן המצב המתקדם של שחזור עומק, מעניין כי עדיין קיימים מספר רב של מקרי שימוש אשר יכולים להפיק תועלת ממתודולוגיות משופרות או מגישות חדשות. בשנים האחרונות חלה עלייה דרמטית של כוח המחשוב, וכתוצאה מכך גם ניידות של הקיבולת החישובית. אנו נסכם בקצרה מספר ניסויים שונים שביצענו להפקת עומק ומבנה תלת ממדי בסמארטפונים ונספק חשיפה לטכניקות שונות שזמינות עבור שחזור גיאומטרי. השיטות הן מגוונות כמו גם היישומים עצמם. לכל אחד מהם יש בדרך כלל קבוצה של תכונות המגדירות את התנהגותו. תכונה אחת כזו היא האם השיטה ישירה או עקיפה. שיטות ישירות אוספות את כל המידע שהן צריכות כדי לייצר הערכה ראשונית של כמויות גיאומטריות אחרי מעבר אחד על הנתונים. הן עשויות להשתמש במעברים נוספים כדי לחדד את האומדן הראשוני, אך השיטות הישירות מייצרות למעשה את כל המידע הדרוש לאחר שכל הנתונים הזמינים נצרכו ועובדו ולפעמים גם לפני כן.

גישה הפוכה לגישה הנ"ל מיושמת על ידי משפחה של שיטות עקיפות. אלה בדרך כלל דורשות עיבוד של כל הנתונים

המחקר בוצע בהנחייתו של פרופסור רון קימל, בפקולטות למדעי המחשב והנדסת חשמל.

חלק מן התוצאות בחיבור זה פורסמו כמאמרים מאת המחבר ושותפיו למחקר בכנסים ובכתבי־עת במהלך תקופת מחקר הדוקטורט של המחבר, אשר גרסאותיהם העדכניות ביותר הינן:

Roberto Mecca, Ariel Tankus, Aaron Wetzler, and Alfred M Bruckstein. A direct differential approach to photometric stereo with perspective viewing. *SIAM Journal on Imaging Sciences*, 7(2):579–612, 2014.

Roberto Mecca, Aaron Wetzler, Alfred M Bruckstein, and Ron Kimmel. Near field photometric stereo with point light sources. *SIAM Journal on Imaging Sciences*, 7(4):2732–2770, 2014.

Roy Or-El, Rom Hershkovitz, Aaron Wetzler, Guy Rosman, Alfred M Bruckstein, and Ron Kimmel. Real-time depth refinement for specular objects. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4378–4386, 2016.

Roy Or-El, Guy Rosman, Aaron Wetzler, Ron Kimmel, and Alfred M Bruckstein. Rgbd-fusion: Real-time high precision depth recovery. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5407–5416, 2015.

Gautam Pai, Aaron Wetzler, and Ron Kimmel. Learning invariant representations of planar curves. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

Ron Slossberg, Aaron Wetzler, and Ron Kimmel. Freehand laser scanning using mobile phone. In *British Machine Vision Conference, 2015*, 2015.

Ron Slossberg, Aaron Wetzler, and Ron Kimmel. Deep stereo matching with dense CRF priors. *arXiv preprint arXiv:1612.01725*, 2016.

Aaron Wetzler, Yonathan Aflalo, Anastasia Dubrovina, and Ron Kimmel. The Laplace-Beltrami operator: a ubiquitous tool for image and shape processing. In *International Symposium on Mathematical Morphology and Its Applications to Signal and Image Processing*, pages 302–316, 2013.

Aaron Wetzler, Rom Hershkovitz, and Ron Kimmel. The Patch-Beltrami framework. submitted, 2017.

Aaron Wetzler and Ron Kimmel. Efficient Beltrami flow in patch-space. In *International Conference on Scale Space and Variational Methods in Computer Vision*, pages 134–143, 2011.

Aaron Wetzler and Ron Kimmel. Multi-GPU training for tera-feature scale, randomized decision trees. submitted, 2017.

Aaron Wetzler and Ron Kimmel. Winqi: A system for 6d localization and SLAM augmentation using wideangle optics and coded light beacons. *arXiv preprint arXiv:1708.05625*, 2017.

Aaron Wetzler, Ron Kimmel, Alfred M Bruckstein, and Roberto Mecca. Close-range photometric stereo with point light sources. In *3DV 2014*, 2014.

Aaron Wetzler, Guy Rosman, and Ron Kimmel. Patch-space Beltrami denoising of 3D point clouds. In *Electrical & Electronics Engineers in Israel (IEEEI), 2012 IEEE 27th Convention of*, pages 1–5, 2012.

Aaron Wetzler, Ron Slossberg, and Ron Kimmel. Rule of thumb: Deep derotation for improved fingertip detection. In *British Machine Vision Conference, 2015*, 2015.

Eric Yudin, Aaron Wetzler, Matan Sela, and Ron Kimmel. Improving 3D facial action unit detection with intrinsic normalization. In *Proceedings of the 1st International Workshop on DIFFerential Geometry in Computer Vision for Analysis of Shapes, Images and Trajectories (DIFF-CV 2015)*, pages 5.1–5.10, 2015.

אני מודה לטכניון על התמיכה הכספית הנדיבה בהשתלמותי.

נושאים בראיה חישובית גאומטרית

חיבור על מחקר

לשם מילוי חלקי של הדרישות לקבלת התואר

'פ''ר'' ''ש''''ש'פ

אהרון ווצלר

הוגש לסנט הטכניון --- מכון טכנולוגי לישראל

אלול תשע''ז חיפה אוגוסט 2017

נושאים בראיה חישובית גאומטרית

אהרון ווצלר